

An introduction to numerical methods for continuous optimisation

Lars A.A. Beex

Contents

1	Introduction, preliminaries, notation and tips	1
1.1	Preface	3
1.2	What is optimisation?	4
1.3	Why is continuous optimisation important?	7
1.4	Outline	8
1.5	Derivative, gradient, second-order derivative and Hessian	9
1.6	Linear algebra	12
1.7	Essential programming tips	15
2	Objective functions	17
3	Genetic algorithms	24
3.1	Elitism & recombination	25
3.2	Mutations	30
3.3	Variations	32
3.4	Questions	35
4	Gradient descent approaches	37
4.1	Steepest descent direction	39
4.2	Backtracking line search	40
4.3	Conjugate gradient direction	41
4.4	Variations	45

4.5	Final comments	50
4.6	Questions	50
5	Newton's method	52
5.1	Newton's method as a root finding technique	53
5.2	Newton's method as a minimisation technique	56
5.3	Practicalities	57
5.4	Why is the Hessian symmetric?	59
5.5	The quadratic convergence explained in detail	61
5.6	Questions	68
6	Quasi-Newton methods	70
6.1	The best known quasi-Newton method	71
6.2	Background	77
6.3	L-BFGS: Avoiding storage problems	86
6.4	Questions	92
7	Constrained minimisation	93
7.1	Equality constraints	95
7.2	Constraint counting	97
7.3	Inequality constraints	97
7.4	Questions	99
8	Constrained minimisation: the penalty method	100
8.1	Equality constraints	101
8.2	Inequality constraints	103
8.3	Final comment	105
8.4	Questions	105

9	Constrained minimisation: the method of Lagrange multipliers	107
9.1	The approach in a nutshell	109
9.2	The ansatz	111
9.3	Inequality constraints	113
9.4	Final comment	113
9.5	Questions	114
10	Solutions to the exercises	115

Introduction, preliminaries, notation and tips

1.1 Preface

From finance to physics, from manufacturing to aerospace, from biology to sociology, and from data science to the spread of infectious diseases, numerous application fields, industries, sectors and research domains employ mathematical models to computationally explain and study trends and behaviours observed in the past and to computationally predict trends and behaviours expected in the future. Where computational models are used, optimisation is essential.

First, optimisation is used to identify the parameter values of these models based on recorded data in the past. Second, optimisation (together with these models) is used to optimally adjust trends and behaviours in the future. This can range from the decision at which moment in time stocks and options should be sold in order to maximise profit, to what restrictions to human behaviour in daily life will minimise the spread of COVID-19. On top of that, some computational models themselves are formulated as optimisation problems. The field of quasi-static mechanical models, such as molecular statics and other time-independent mechanical models, is an example of this. Optimisation is even used to solve systems of large numbers of linear equations.

Two types of optimisation can be distinguished: discrete optimisation and continuous optimisation. In discrete optimisation, the variables to be optimised are integers, e.g. -1, 0, 5, 117. The variables in continuous optimization are scalars, e.g. -11.112, 1.00017, 12.958. These lecture notes focus on continuous optimisation.

These lecture notes do not focus in detail on how to formulate optimisation problems. Instead, it concentrates on a range of well-known numerical methods to perform optimisation. Unfortunately, not one numerical method outperforms the others; in some cases, one method is more likely to find the optimum faster, whereas in other cases, other methods perform better. Important for the decision which method to use is the calculation speed of the method; generally, the choice of which method to use is always a compromise between how long one wants to wait for the optimisation to be successfully finalised and the robustness of the method, i.e. the chance that the method is able to calculate the optimum. Generally, one can state that methods with a high robustness require more time than methods with a low robustness.

These days, numerous optimisation toolboxes containing numerous numerical methods exist. Many toolboxes can freely be downloaded or are included under the license of more general software packages. Optimisation toolboxes are convenient to use, because they avoid the implementation of the optimisation algorithm, their implementations are fast because they are implemented in low-level code close to the machine instructions, and the implementations are often even parallel.

Even if one only exploits an existing optimisation toolbox, a good background in numerical approaches for optimisation problems is beneficial, because if optimisation toolboxes are used in a black-box fashion, unnecessarily high computational times may occur or one may find a suboptimal solution. Understanding the pros and cons of the different types of optimisation approaches is essential to select the most suited approach for the optimisation problem of interest - in terms of speed, accuracy, but also in terms of limitations. Furthermore, the different numerical parameters that each numerical method requires must often still be selected by the user her/himself and is often crucial for the speed and success of the optimisation. On top of that, even existing optimisation toolboxes prefer you to provide derivatives and second-order derivatives. Hence, if you want to successfully exploit existing toolboxes, you will need to know

how to hardcode derivatives and second-order derivatives involving large number of variables.

The emphasis of the lecture notes is on the conceptual understanding of the numerical methods, outlines their advantages and disadvantages, relates the approaches to each, and helps the reader to implement/hardcode them her/himself in order to truly assess the reader's understanding. Although some mathematical proofs are incorporated, they are not part of the principle learning outcomes; they are only included to provide additional information to the more motivated reader. Thus, a bird's eye view over the different methods, as well as their implementation are prioritised over the rigorousness of the mathematical notation. Furthermore, numerous pseudo-algorithms are provided, which serve as inspiration to answer the questions of each chapter. A separate chapter with solutions is included, with numerous detailed explanations.

These lecture notes discuss:

- four numerical methods for continuous optimisation,
- three approaches to incorporate constraints in numerical approaches, and
- provide a wide range of easy and difficult exercises to engage the reader with the practical use of the methods.

Books on continuous optimisation, e.g. 'Nocedal J, Wright SJ, *Numerical Optimization. Second edition.* <https://doi.org/10.1007/978-0-387-40065-5>' provide a mathematically more rigorous and complete overview of optimisation techniques than the lecture notes in front of you. On the other hand, the lecture notes focus on practical use and implementation. It is therefore better suited for readers novice in numerical methods for continuous optimisation.

1.2 What is optimisation?

Optimisation is the minimisation or maximisation of a function in terms of its variables. The function that is to be minimised or maximised is most generally referred to as the **objective function**. Depending on the application field, it may also be referred to as the **cost function**, **loss function** (both for minimisation), **fitness function**, **utility function** (both for maximisation), **potential energy**, **potential** or **energy function** (in the fields of physical sciences).

Let us consider for instance function m :

$$m(u_1, u_2) = 10u_1^2 + 5u_2^2, \tag{1.1}$$

where u_1 and u_2 are the two variables of the function. If we state that the function only accepts real numbers for its two variables and the function itself will also output a real number, we can note this mathematically as $m : \mathbb{R}^2 \rightarrow \mathbb{R}$. This means as much as 'function m eats two real numbers and as a results, m poops out one real number'.

As the number of variables can be quite large (instead of only two variables), we will store all the variables of the objective function in column $\underline{u}$:

$$\underline{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \dots \\ u_{n_{var}} \end{bmatrix}. \quad (1.2)$$

where n_{var} denotes the number of variables. If we want to turn column $\underline{u}$ in a row, we denote this by superscript T , which gives us the transpose of $\underline{u}$:

$$\underline{u}^T = [u_1 \quad u_2 \quad u_3 \quad \dots \quad u_{n_{var}}]. \quad (1.3)$$

This means that column $\underline{u}$ can also be expressed as:

$$\underline{u} = [u_1 \quad u_2 \quad u_3 \quad \dots \quad u_{n_{var}}]^T. \quad (1.4)$$

If we want to determine the values of the variables in $\underline{u}$ that minimise $m(\underline{u})$, we speak of a minimisation problem and express it as follows if the minimum of m is of interest:

$$m^* = \min_{\underline{u}} m(\underline{u}), \quad (1.5)$$

where m^* is the minimum of $m(\underline{u})$. If we want to find the values of the variables in $\underline{u}$ that minimise $m(\underline{u})$, but we are interested in the values of $\underline{u}$ at which m^* occurs, we write:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}), \quad (1.6)$$

where $\underline{u}^*$ is the minimiser of $m(\underline{u})$. Similarly if we are interested in a maximisation of m , we write:

$$m^* = \max_{\underline{u}} m(\underline{u}) \quad \text{or} \quad \underline{u}^* = \arg \max_{\underline{u}} m(\underline{u}). \quad (1.7)$$

In many cases, we cannot determine the minimum or maximum of $m(\underline{u})$ analytically (*read:* with pen and paper), but we need to determine it numerically (*read:* with the computer). These notes provide an introduction to numerical methods for continuous optimisation (*read:* minimisation or maximisation of an objective function using the computer). Important to realise is that the adjective **continuous** in continuous optimisation entails that the focus in these notes is on C^1 -continuous objective functions. This means that the objective function itself must be continuous, but also the objective function's gradient.

It is also important to realise that each maximisation problem can simply be expressed in terms of a minimisation problem (see Fig. 1.1):

$$\underline{u}^* = \arg \max_{\underline{u}} m(\underline{u}) = \arg \min_{\underline{u}} -m(\underline{u}), \quad (1.8)$$

and by inserting minimiser $\underline{u}^*$ in the original objective function, $m(\underline{u})$ (instead of $-m(\underline{u})$), the maximum can then be found:

$$m^* = m(\underline{u}^*). \quad (1.9)$$

From now on, we will therefore only focus on minimisation problems. These notes should therefore have been named ‘Introduction to numerical methods for continuous minimisation’ instead of ‘An introduction to numerical methods for continuous optimisation’.

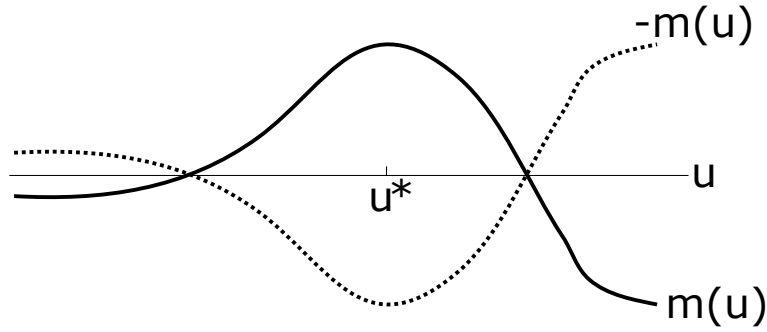


Figure 1.1: Sketch of function m in terms of its variable u , and function $-m$. The maximum of $m(u)$ occurs at u^* , which is also where $-m(u)$ is minimum. The absolute value of the maximum of $m(u)$ is furthermore the same as the absolute value of the minimum of $-m(u)$.

For reasons that become clear later in these notes, we can define two types of minima: local and global minima. A global minimum is the true minimum of an objective function. It can be defined as follows:

$$m(\underline{u}^*) \leq m(\underline{u}). \quad (1.10)$$

On the other hand, a local minimum is a minimum in a subdomain of the space of the variables. It is defined as follows:

$$m(\underline{u}^*) \leq m(\underline{u}), \quad (1.11)$$

where

$$\|\underline{u} - \underline{u}^*\| \leq \omega, \quad (1.12)$$

with $0 < \omega < \infty$ and $\|\underline{a}\|$ denoting the L^2 -norm, which is defined as:

$$\|\underline{a}\| = \sqrt{\sum_{i=1}^n (a_i)^2} = \sqrt{\underline{a}^T \underline{a}}. \quad (1.13)$$

For the left diagram in Fig. 1.16 for instance, the global minimum occurs at $u \approx -2$, but several local minima can be observed: at $u \approx -9.5$, $u \approx -4$, $u \approx -2$ and $u \approx 3$. The global minimum is thus also a local minimum.

Generally speaking, numerical approaches only find one minimum and one does not know if the found minimum is a local or global one - unless one knows the mathematical characteristics of the objective function. (Note that this assumes that the numerical approach is properly applied, because they may not find a minimum at all in case of improper use.)

1.3 Why is continuous optimisation important?

Numerical optimisation plays an important role in science and industry, because it is a general concept that appears in numerous application fields. In more detail, the answer to the question ‘Why is continuous optimisation important?’ has four components. For three out of the four components, the definition of *a model* is important. Although many definitions are available, we characterise *a model* as follows:

A model is a simplified description of reality that can be used to explain and/or predict the part of reality in which we are interested.

If we include more details in the model, we can expect the model to represent more aspects of reality. The disadvantage of incorporating more aspects is that the model will also be more complex to evaluate - i.e. the simulation/computational times increase. This implies that a model should be as simple as possible, but incorporate the mechanisms important to sufficiently accurately describe the part of reality in which we are interested.

Models are frequently defined in the language of mathematics, because complex models are often straightforward to describe in terms of mathematics. Mathematical models are furthermore able to quantitatively express a representation of reality, whereas non-mathematical models are only able to qualitatively express a representation of reality.

Now we have some idea of what a model is, let us go back to the question ‘Why is continuous optimisation important?’.

The first part of the answer to this question is *because the solutions of many models are the minimisers or maximisers of an objective function*. The reason for this in the physical and engineering sciences is that nature wants to minimise potential energies and maximise dissipation (i.e. entropy). This is a simple fact; it is just the way nature works. Similarly as the fact that we experience gravity; it is just a fact. Thus, if we can define a model in terms of a potential energy, we can minimise it in order to find the state in which the model prefers to be configured. We will investigate such models extensively in these notes, although the focus is mostly on the numerical methods - not on what the objective functions represent.

The second part of the answer to the question ‘Why is continuous optimisation important?’ is *because models need parameters and the most suited parameter values can be determined using numerical optimisation*. We will also consider these types of optimisation problems in these notes (Chapters 6 and 7).

The third part of the answer to the question ‘Why is numerical optimisation important?’ is *because at the moment that our model sufficiently accurately describes the real world, we want to exploit it to change the future to our advantage*.

As an example think about the following. You have bought stocks/shares of a company. You have a model of how the price of these stocks change over time. The question that you are interested in is ‘at what time do I need to sell my stocks to make the most profit?’. Another example, this time from engineering: you need to design a rope which should not break when a load of 1,000 kilograms hangs on it. ‘What should the diameter of the rope be to withstand a load of 1,000 kilograms?’ You don’t want the diameter to be too large, because the larger the volume of the rope, the more material is required (costs more), the manufacturing will be more difficult (costs more) and transporting the rope is more difficult (costs more). At the same time, you want the rope to be strong enough not to break when a load of 1,000 kilograms hangs on it.

The fourth part of the answer to the question ‘Why is continuous optimisation important?’ is *because it can be used to solve problems in linear algebra with many variables more efficiently than direct approaches*. Solving linear systems of equations is one of the most important issues in scientific computing and all over the world, entire research groups solely focus on solving linear systems. Solving linear systems of equations can be accelerated if the solution of the linear system of equations is expressed as the minimisers of an objective function (Chapter 8) - which is often but not always possible.

1.4 Outline

This book introduces:

- four numerical optimisation techniques (Chapters 3, 4, 5 and 12),
- three approaches to incorporate constraints in numerical approaches for minimisation problems (Chapters 6, 7 and 8),
- *inverse problems*: how minimisation approaches can be used to find the model parameters that are most suited to describe the real world (Chapter 9),
- a dedicated technique to increase the speed of inverse problems (Chapter 10), and
- how numerical approaches for minimisation approaches are used to solve systems of linear equations (Chapter 11).

In Chapter 2, several objective functions are introduced that will be used throughout these notes. The objective functions in Chapter 2 are the potential energies of several truss/spring networks in a 2D setting. Their minimisers indicate the configurations for which the networks are in force equilibrium (meaning that the networks abide Newton’s first law in their minimal potential energy configuration).

Chapters 3, 4 and 5 introduce three numerical approaches for minimisation problems - although hundreds exist. The reason that these three approaches are considered is two-fold. First, they are frequently used approaches. Second, each approach represents a separate class of minimisation approaches. We can distinguish (i) approaches that do not use any derivative information - the genetic algorithms of Chapter 3 are a popular example of this. Genetic algorithms have the advantage that they are able to pass through local minima if more than one local minimum is present in the search domain. This is partially thanks to the fact that they do not use a starting *point* (i.e. an initial guess), but a initial search *domain*. The most important disadvantage due to the lack of derivative information is that they are time-consuming (often too time-consuming). They are typically also not as accurate as the approaches of Chapters 4 and 5.

The second approach is presented in Chapter 4 and takes first-order derivative information into account. The conjugate gradient method starts in an initially selected point (the initial guess) and uses the first-order derivative to determine in which direction a step has to be made in order to decrease the objective function. Because derivative information is exploited, it is in most cases substantially faster than approaches that do not take derivative information into account. It is generally also more accurate.

The third type of approach, Newton's method, is present in Chapter 5 and incorporates not only first-order derivative information, but also second-order derivative information. Because both the slope of the objective function (i.e. the first-order derivatives), as well as the change of the slope of the objective function (i.e. the second-order derivatives), even more information about the behaviour of the objective function can be exploited to determine the minimum and the minimiser. Consequently, Newton's method is substantially faster and more precise than the aforementioned approaches. A disadvantage is that the Hessian must be invertible, which often translates in the practical requirement that the initial guess must be close to the minimiser. However, the Hessian of some objective functions is never invertible and hence, Newton's approach cannot be used at all. Although not discussed in these notes, different types of minimisation approaches may be used in sequence.

1.5 Derivative, gradient, second-order derivative and Hessian

As discussed in the previous subsection, first-order and second-order derivatives are crucial for many minimisation approaches as they provide the direction in which the objective function decreases. In this subsection, we will discuss two issues on a rather superficial level: (i) what is a derivative and gradient and (ii) what is a second-order derivative and Hessian.

1.5.1 Derivative and gradient

We start by discussing the derivative of a function. Derivatives are important when we want to minimise a function, because the derivative tells us how the function increases and decreases. Hence, if we have an estimate of the minimiser, following the objective function in the negative direction of the derivative can be expected to result in a better estimate of the minimiser.

For now, we focus on a function that depends on a single variable:

$$m(u) = -2e^{-(u+2)^2} - \cos(u)e^{-(u+3)^4} + \cos(u), \quad (1.14)$$

which is graphically presented in the left diagram of Fig. 1.2. The derivative of m with respect to u tells us the steepness of function m , at u . In other words, the derivative tells us how much m will change if we slightly change the variable, i.e. if we move from u to $u + \delta u$, where δu is very small (i.e. infinitesimally small). Mathematically, the derivative of m is expressed as:

$$\frac{dm}{du} = \lim_{\delta u \rightarrow 0} \frac{m(u + \delta u) - m(u)}{\delta u}, \quad (1.15)$$

which for Eq. (1.14) together with the differentiation rules you have hopefully learned before (or with a symbolic computation toolbox) gives us:

$$\frac{dm}{du} = 4(u+2)e^{-(u+2)^2} + \sin(u)e^{-(u+3)^4} + 4(u+3)^3 \cos(u)e^{-(u+3)^4} - \sin(u), \quad (1.16)$$

which is also graphically illustrated in Fig. 1.2.

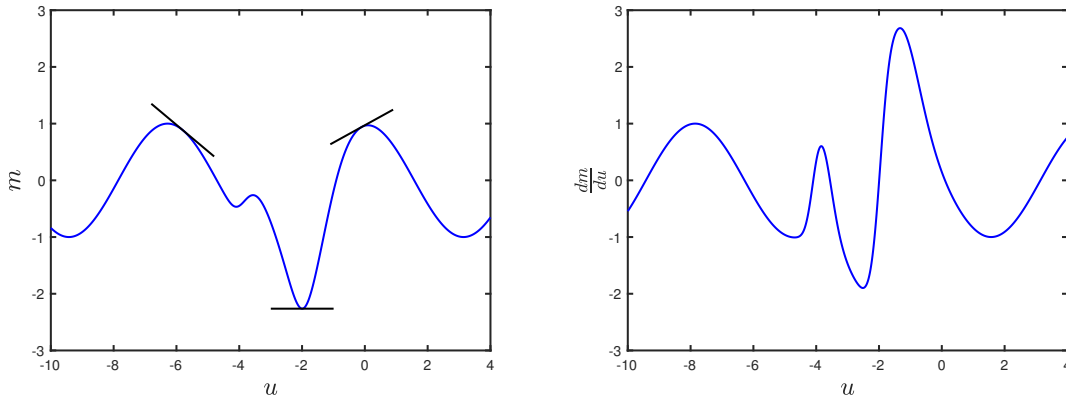


Figure 1.2: Left: $m(u)$ according to Eq. (1.14) with three slopes, right: the slope of m (i.e. $\frac{dm}{du}$ in Eq. (1.16)).

If we now consider a function dependent on two variables, e.g.:

$$m(\underline{u}) = u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20}(u_1^2 + u_2^2), \quad (1.17)$$

the slope/gradient is not simply a value, but a vector (see Fig. 1.3). After all, the slope of m can be expressed with respect to u_1 and with respect to u_2 . In the case that $n_{var} > 1$ such as here, we generally do not speak of the derivative, but of the gradient. The gradient of m with respect to $\underline{u}$ is therefore now expressed as a column of length 2:

$$\frac{\partial m}{\partial \underline{u}} = \begin{bmatrix} \frac{\partial m}{\partial u_1} \\ \frac{\partial m}{\partial u_2} \end{bmatrix} = \begin{bmatrix} \frac{u_1}{10} + e^{-u_1^2 - u_2^2} - 2u_1^2 e^{-u_1^2 - u_2^2} \\ \frac{u_2}{10} - 2u_1 u_2 e^{-u_1^2 - u_2^2} \end{bmatrix}. \quad (1.18)$$

Note that gradient $\frac{\partial m}{\partial \underline{u}}$ is mathematically often written as ∇m , where symbol ∇ indicates that the result is a column (although function m itself is a scalar function).

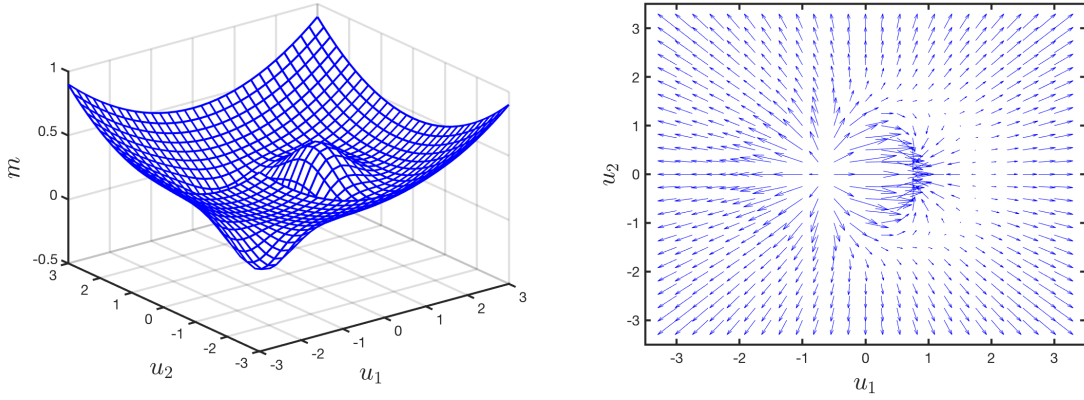


Figure 1.3: Left: $m(\underline{u})$ according to Eq. (1.17), right: vector $\frac{dm}{d\underline{u}}$ according to Eq. (1.18). The size of the vectors in the right diagram is a measure for the steepness of the slope.

1.5.2 Second-order derivative and Hessian

As we have just seen, the derivative tells us how much a function changes if we make an infinitesimal step. We could however also investigate how much the derivative itself changes - this gives us even more information about the behaviour of the objective function. Hence, we are now interested in the derivative of the derivative, which is often called the second-order derivative. For Eq. (1.14), we can write it as follows:

$$\frac{d^2m}{du^2} = \frac{d}{du} \frac{dm}{du} = (4 - 8(u+2)^2) e^{-(u+3)^4} + (1 + 12(u+3)^2 - 16(u+3)^6) \cos(u) e^{-(u+3)^4} - 8(u+3)^3 \sin(u) e^{-(u+3)^4} - \cos(u). \quad (1.19)$$

For a function that depends on a single variable, the second-order derivative is not very exciting. For a function that depends on more than one variable however, we end up with a matrix as we need to differentiate the gradient with respect to several variables. This matrix is normally referred to as the Hessian and can be computed as follows:

$$\frac{d^2m}{d\underline{u}^2} = \frac{d}{d\underline{u}} \frac{dm}{d\underline{u}} = \begin{bmatrix} \frac{\partial^2 m}{\partial u_1^2} & \frac{\partial^2 m}{\partial u_2 \partial u_1} & \frac{\partial^2 m}{\partial u_3 \partial u_1} & \cdots & \frac{\partial^2 m}{\partial u_{n_{var}} \partial u_1} \\ \frac{\partial^2 m}{\partial u_1 \partial u_2} & \frac{\partial^2 m}{\partial u_2^2} & \frac{\partial^2 m}{\partial u_3 \partial u_2} & \cdots & \frac{\partial^2 m}{\partial u_{n_{var}} \partial u_2} \\ \frac{\partial^2 m}{\partial u_1 \partial u_3} & \frac{\partial^2 m}{\partial u_2 \partial u_3} & \frac{\partial^2 m}{\partial u_3^2} & \cdots & \frac{\partial^2 m}{\partial u_{n_{var}} \partial u_3} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \frac{\partial^2 m}{\partial u_1 \partial u_{n_{var}}} & \frac{\partial^2 m}{\partial u_2 \partial u_{n_{var}}} & \frac{\partial^2 m}{\partial u_3 \partial u_n} & \cdots & \frac{\partial^2 m}{\partial u_{n_{var}}^2} \end{bmatrix}. \quad (1.20)$$

The Hessian, $\frac{d^2m}{d\underline{u}^2}$, is often abbreviated as $\nabla^2 m$. It is symmetric because of Schwarz's theorem, which states that the order of differentiation is irrelevant. Symmetry means that the value at

row i , column j is the same as the value at row j , column i , i.e. $\nabla^2 m = (\nabla^2 m)^T$. (If we for instance regard $i = 1$ and $j = 2$, we can say that $\frac{\partial^2 m}{\partial u_1 \partial u_2} = \frac{\partial^2 m}{\partial u_2 \partial u_1}$).

It is often important to minimise the computational time and storage to incorporate the symmetry of the Hessian in the implementation, because only half of components of the Hessian (and a bit more) need to be computed and the other half is then known as well. Another issue that can save even more computational time, is to recognise if the Hessian contains many zeros and if so, to establish it in a *sparse format* in your implementation. Sparsity implies that (i) only the non-zero terms are stored and (ii) computations will only handle the non-zero terms, saving both storage space and computational time.

Question 1.1 Compute the Hessian for Eq. (1.17).

Question 1.2 Assume objective function $m(\underline{u})$ and $\underline{u}$ contains 10^3 variables. How many independent components will be present in the Hessian of $m(\underline{u})$ if we assume that the Hessian is symmetric?

1.6 Linear algebra

A small amount of linear algebra is necessary to treat the content of this book. In this concise section, the necessary linear algebra operations are discussed. We have already seen that a column (of scalars) is written as $\underline{x}$ and if this column is written as a row, it must be transposed; $\underline{x}^T$. Matrices are also considered; we have already seen that the Hessian is a matrix. Whereas a column is denoted as a lowercase with a line underneath it, a matrix is denoted by a capital with two lines underneath it: $\underline{\underline{A}}$.

A column and row can only be multiplied with each other if they are of the same length. In this book, we are only interested in multiplying a column with a row such that the result is a scalar. If we consider the following two columns of equal lengths:

$$\underline{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \quad \underline{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}, \quad (1.21)$$

they can be multiplied with each other as follows:

$$\underline{a}^T \underline{b} = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3, \quad (1.22)$$

which is the same if they are multiplied as follows:

$$\underline{b}^T \underline{a} = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3. \quad (1.23)$$

It can be noted that they can also be multiplied such that the result is a matrix (i.e. $\underline{a}\underline{b}^T$ or $\underline{b}\underline{a}^T$, $\underline{a}\underline{b}^T \neq \underline{b}\underline{a}^T$), but this operation does not appear in this book.

A matrix can also be transposed. If we consider the following matrix for instance:

$$\underline{\underline{A}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad (1.24)$$

its transposed reads:

$$\underline{\underline{A}}^T = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}. \quad (1.25)$$

A symmetric matrix is a square matrix (i.e. its number of rows equals its number of columns), in which each scalar (at row i , column j) equals the scalar on the opposite side of the diagonal (at row j , column i) $A_{ij} = A_{ji}$. An example is thus:

$$\underline{\underline{A}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}. \quad (1.26)$$

The transposed of a symmetric matrix thus equals the matrix itself: $\underline{\underline{A}} = \underline{\underline{A}}^T$.

Matrices can be multiplied with columns and rows can be multiplied if their dimensions and lengths are of the appropriate size. We only consider the following three multiplications:

$$\underline{\underline{A}}\underline{b} \quad \underline{b}^T\underline{\underline{A}} \quad \underline{b}^T\underline{\underline{A}}\underline{c}. \quad (1.27)$$

The first multiplication is only possible if the number of columns of $\underline{\underline{A}}$ equals the length of $\underline{b}$. The result of this is a column consisting of the same number of components as the number of rows of $\underline{\underline{A}}$, e.g.:

$$\underline{\underline{A}}\underline{b} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} a_{11}b_1 + a_{12}b_2 \\ a_{21}b_1 + a_{22}b_2 \\ a_{31}b_1 + a_{32}b_2 \end{bmatrix}. \quad (1.28)$$

The second multiplication in Eq. (1.27) is only possible if the number of rows of $\underline{\underline{A}}$ equals the length of $\underline{b}$. The result of this is a row consisting of the same number of components as the number of columns of $\underline{\underline{A}}$, e.g.:

$$\underline{b}^T\underline{\underline{A}} = [b_1 \quad b_2] \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} = [a_{11}b_1 + a_{21}b_2 \quad a_{12}b_1 + a_{22}b_2 \quad a_{13}b_1 + a_{23}b_2]. \quad (1.29)$$

The third multiplication in Eq. (1.27) is a straightforward combination of the first two multiplications in Eq. (1.27). It is only possible if the length of $\underline{b}$ equals the number of rows of $\underline{\underline{A}}$ and the length of $\underline{c}$ equals the number of columns of $\underline{\underline{A}}$. The result of this double multiplication is a scalar, e.g.:

$$\begin{aligned} \underline{b}^T \underline{\underline{A}} \underline{c} &= \begin{bmatrix} b_1 & b_2 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} a_{11}b_1 + a_{21}b_2 & a_{12}b_1 + a_{22}b_2 & a_{13}b_1 + a_{23}b_2 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} \\ &= c_1(a_{11}b_1 + a_{21}b_2) + c_2(a_{12}b_1 + a_{22}b_2) + c_3(a_{13}b_1 + a_{23}b_2). \end{aligned} \quad (1.30)$$

Furthermore, it is useful to know the identity matrix, $\underline{\underline{I}}$, which is always square. The diagonal scalars of $\underline{\underline{I}}$ all equal 1, whereas all off-diagonal scalars equal 0, e.g.:

$$\underline{\underline{I}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (1.31)$$

The identity matrix comes into play in the definition of the inverse of a matrix, denoted by superscript -1. The inverse of a matrix $\underline{\underline{A}}$ is defined as follows:

$$\underline{\underline{A}}^{-1} \underline{\underline{A}} = \underline{\underline{A}} \underline{\underline{A}}^{-1} = \underline{\underline{I}}, \quad (1.32)$$

which entails that the inverse of a matrix only exists if its a square matrix. However, even if it a matrix is square, its inverse often does not exist. If a matrix is not invertible, it is called singular.

In this book, we frequently consider the following system of linear equations:

$$\underline{\underline{A}} \underline{x} = \underline{b}, \quad (1.33)$$

where all the components of $\underline{\underline{A}}$ and $\underline{b}$ are known and the scalars in $\underline{x}$ are unknown and must be found. This is only possible if $\underline{\underline{A}}$ is a invertible matrix (square, amongst others) and the lengths of $\underline{x}$ and $\underline{b}$ are the same and equal the (square) size of $\underline{\underline{A}}$. If $\underline{\underline{A}}$ is indeed invertible, $\underline{x}$ can be determined by premultiplying both sides of the equation in Eq. (1.33) by the inverse of $\underline{\underline{A}}$, yielding:

$$\underline{\underline{A}}^{-1} \underline{\underline{A}} \underline{x} = \underline{\underline{A}}^{-1} \underline{b}, \quad (1.34)$$

which, with the help of Eq. (1.32), can be written as:

$$\underline{\underline{I}} \underline{x} = \underline{x} = \underline{\underline{A}}^{-1} \underline{b}. \quad (1.35)$$

Hence, if the inverse of $\underline{\underline{A}}$ is known, $\underline{x}$ can be calculated by a straightforward matrix-column multiplication as we have discussed above. The problem is that calculating the inverse of a matrix takes a lot of time. For this reason, computing the inverse of a matrix is (generally speaking) to be avoided at all costs. If one needs to solve a system of linear equations as presented in Eq. (1.33), one is instead strongly urged to use a *linear solver* (in this book a *direct solver* is sufficiently fast); a combination of functions predefined in many programming languages that avoids the computation the matrix inverse, in order to more quickly calculate unknown column $\underline{x}$.

1.7 Essential programming tips

To answer most questions in these notes, you will be able to get away with rather poor programming, because the number of variables (n_{var}) is small. However, your implementation must be substantially fast to answer Question 6.3 and the questions in Chapter 7. Here are some tips to obtain fast codes in Matlab and Python:

- Never **append** a column or matrix in a for-loop (i.e. change the length of a column of matrix in for-loop), because this is time consuming. **Always** determine in advance the length of a column and the size of a matrix, initialise it with zeros or in a sparse format and then adjust the components in the for-loop.
- Print and plot as few temporary results as possible, because this is time consuming.
- **Never** use substitution in your code to replace symbols with numbers. This is time consuming.
- In case you need to solve a linear system as $\underline{\underline{A}}\underline{x} = \underline{b}$, where $\underline{\underline{A}}$ denotes a known, invertible $n \times n$ matrix, $\underline{b}$ a known column of length n and $\underline{x}$ a column with n unknowns, one may compute the inverse of the Hessian and determine the solution as $\underline{x} = \underline{\underline{A}}^{-1}\underline{b}$. If possible however, avoid computing the inverse because this is time consuming. Instead, use a direct solver - like the back slash in Matlab.
- The Hessian is always symmetric. This means that only half of the components of the Hessian need to be computed in your code. Therefore, do not compute all contributions, but only the unique ones and add each to both the upper triangular and lower triangular part of the Hessian.
- In order to exploit the fact that many components of the Hessian (*and* the gradient) are the same for the objective functions of Chapter 2, it is **absolutely not recommended** to determine the gradient and the Hessian of the *entire* objective function *at once*. Instead, determine them for each contribution of the objective function *independently*. Let us consider for instance the gradient of the objective function of Model 4a in Chapter 2. First, you focus on the first contribution, which reads $-\underline{g}^T \underline{u}$. You first determine the gradient for this contribution **only**. Analytically, it is straightforward that $\frac{\partial m}{\partial \underline{u}} = -\underline{g}$ for this first contribution and you will implement it like this. Now, we focus on the second contribution, which is actually a sum over ten indices. Instead of working out this sum in detail, you adjust your gradient as follows (where $\underline{f}$ is used for the gradient):

Hence, when you incorporate the gradient of the third component, you simply add another for-loop below this for-loop above. And you continue to add as many for-loop as there are sums in the objective function of Model 4a.

```

f = -g
for i = 1 : 10
    ai = A(i)
    u2i_1 = u(2i - 1)
    u2i = u(2i)
    dmdu2i_1 = ai * (2 * u2i_1 * (((u2i + 1)2 + (u2i_1)2)0.5 - 1)) / ((u2i + 1)2 + (u2i_1)2)0.5
    dmdu2i = ai * (((u2i + 1)2 + (u2i_1)2)0.5 - 1) * (2 * u2i + 2) / ((u2i + 1)2 + u2i_12)0.5
    f([2j - 1; 2j]) = f([2j - 1; 2j]) + [dmdu2i_1; dmdu2i]
end

```

- In case of large systems (i.e. with large numbers of variables), many components of the Hessian are zero. The ratio of zero components in the Hessian generally increases as the number of variables increases. If the Hessian has many zero components, it is generally faster to not compute the Hessian as a standard (**full**) matrix, but as a **sparse** matrix. The difference between full and sparse matrices in scientific computing software is that the software for sparse matrices knows where the zero components of the matrix are located and the software will ignore these components as they do not contribute. However, if you define a matrix in the standard (**full**) fashion, the software assumes that each component is non-zero and will therefore incorporate all of them in the computations. Now, the difference in speed between full and sparse matrices is probably negligible for the systems considered in this book (the maximum number of variables is only 80), but for larger systems of thousands, hundreds of thousands or millions of variables the difference in speed between full and sparse matrices is substantial (although the accuracy of the computation will not change). Hence, sparse matrices must be preferred for those cases.

Objective functions

In this short section, most of the objective functions of the lecture notes are introduced. The first one depends on a single variable and the last ones on 80 variables. The variables are denoted by $\underline{u} = [u_1 \ u_2 \ \dots \ u_{n_{var}}]^T$.

The minimisers of the objective functions ($\underline{u}^*$) correspond to the configurations of the spring systems in Fig. 2.1, for which force equilibrium holds. The reason is that the objective functions describe the potential energy of the spring systems. The potential energy itself is not of true interest here; it is only used to determine the equilibrium configuration (i.e. the minimiser).

The variables, $\underline{u}$, represent the displacements of nodes. Nodes are locations where two or more springs meet. External forces, stored in $\underline{g} = [g_1 \ g_2 \ \dots \ g_{n_{var}}]^T$, can only be applied at these nodes. If no external forces are applied (i.e. $\underline{g} = \underline{0}$), the spring systems as presented in Fig. 2.1 are already in force equilibrium (i.e. if $\underline{g} = \underline{0}$, then $\underline{u}^* = \underline{0}$ and $m^* = 0$).

Parameters $\underline{a} = [a_1 \ a_2 \ \dots \ a_{n_{par}}]^T$ are a measure for the effective stiffness of the springs. If a spring is made of titanium for instance, its effective stiffness is substantially larger than if the spring is made of rubber. In most exercises, the parameters are set as $\underline{a} = \underline{1}$ and the externally applied forces ($\underline{g}$) are also given. Hence, most often only the variables, $\underline{u}$, are of interest.

Model 1

The first objective function depends on two variables, u_1 and u_2 :

$$m(\underline{u}) = a_1 \left(\sqrt{(1+u_1)^2 + (1+u_2)^2} - \sqrt{2} \right)^2 + a_2 \left(\sqrt{(1-u_1)^2 + (1+u_2)^2} - \sqrt{2} \right)^2 - g_1 u_1 - g_2 u_2, \quad (2.1)$$

where for instance $\sqrt{(1+u_1)^2 + (1+u_2)^2}$ can be recognised as the length of the left spring (top-left in Fig. 2.1), $\sqrt{(1+u_1)^2 + (1+u_2)^2} - \sqrt{2}$ as the change of length of the same spring (as its original length is $\sqrt{2}$), and $a_1 \left(\sqrt{(1+u_1)^2 + (1+u_2)^2} - \sqrt{2} \right)^2$ as the energy stored in a spring (the power of two is necessary to ensure that both positive and negative length changes come with a positive stored energy). Scalar a_1 furthermore denotes the stiffnesses. g_1 and g_2 denote forces that are applied. Hence, a_1 , a_2 , g_1 and g_2 are parameters and not variables.

Model 2a

The second objective function depends on four variables and represents the potential energy of the second system of the top row in Fig. 2.1. It is expressed as:

$$\begin{aligned}
m(\underline{u}) = & a_1 \left(\sqrt{(u_1)^2 + (1 + u_2)^2} - 1 \right)^2 + a_2 \left(\sqrt{(u_3)^2 + (1 + u_4)^2} - 1 \right)^2 + \\
& a_3 \left(\sqrt{(1 + u_3 - u_1)^2 + (u_4 - u_2)^2} - 1 \right)^2 + a_4 \left(\sqrt{(1 + u_3)^2 + (1 + u_4)^2} - \sqrt{2} \right)^2 + \\
& a_5 \left(\sqrt{(1 - u_1)^2 + (1 + u_2)^2} - \sqrt{2} \right)^2 - \underline{g}^T \underline{u}. \quad (2.2)
\end{aligned}$$

Model 2b

The third objective function is exactly the same as the third one, except that two contributions are omitted:

$$\begin{aligned}
m(\underline{u}) = & a_1 \left(\sqrt{(u_1)^2 + (1 + u_2)^2} - 1 \right)^2 + a_2 \left(\sqrt{(u_3)^2 + (1 + u_4)^2} - 1 \right)^2 + \\
& a_3 \left(\sqrt{(1 + u_3 - u_1)^2 + (u_4 - u_2)^2} - 1 \right)^2 - \underline{g}^T \underline{u}. \quad (2.3)
\end{aligned}$$

Hence, it represents the third system of the top row in Fig. 2.1.

Model 3a

The fourth objective function depends on 20 variables and is defined as follows:

$$\begin{aligned}
m(\underline{u}) = & -\underline{g}^T \underline{u} + \sum_{i=1}^{10} a_i \left(\sqrt{(u_{2i-1})^2 + (1 + u_{2i})^2} - 1 \right)^2 + \\
& \sum_{i=1}^9 a_{i+10} \left(\sqrt{(1 + u_{2i+1} - u_{2i-1})^2 + (u_{2i+2} - u_{2i})^2} - 1 \right)^2 + \\
& \sum_{i=1}^9 a_{i+19} \left(\sqrt{(1 + u_{2i+1})^2 + (1 + u_{2i+2})^2} - \sqrt{2} \right)^2 + \\
& \sum_{i=1}^9 a_{i+28} \left(\sqrt{(1 - u_{2i-1})^2 + (1 + u_{2i})^2} - \sqrt{2} \right)^2. \quad (2.4)
\end{aligned}$$

It expresses the potential energy of the system on the second row of Fig. 2.1, which consists of 37 springs.

Model 3b

The fifth objective function describes the potential energy as the same system of Model 3a, except that the diagonal springs are not present (third row of Fig. 2.1):

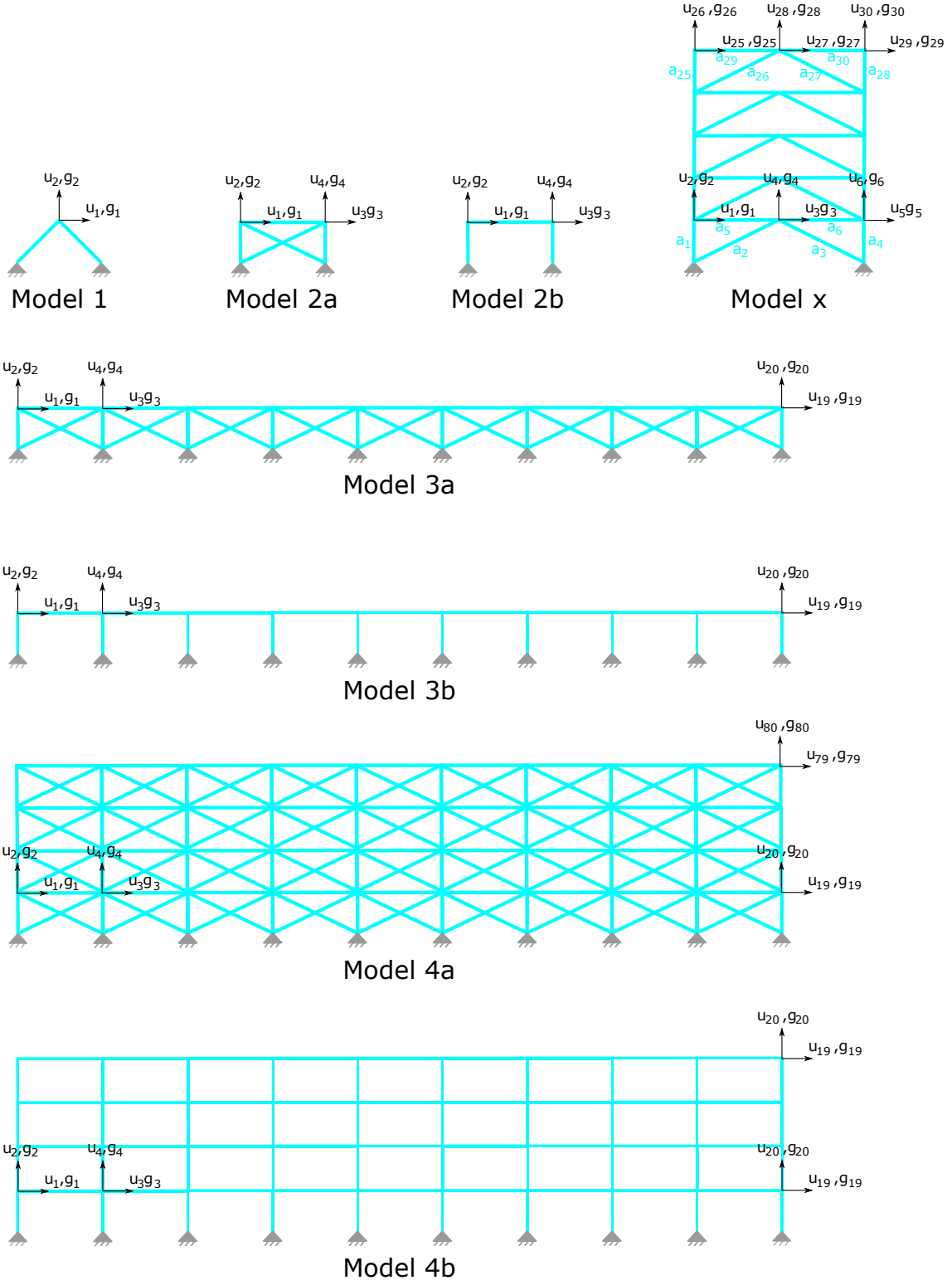


Figure 2.1: The spring systems that physically correspond to the introduced objective functions. Springs are shown in blue, fixed foundations in grey and the variables and the forces that could be applied at the variables as black arrows.

$$m(\underline{u}) = -\underline{g}^T \underline{u} + \sum_{i=1}^{10} a_i \left(\sqrt{(u_{2i-1})^2 + (1 + u_{2i})^2} - 1 \right)^2 + \sum_{i=1}^9 a_{i+10} \left(\sqrt{(1 + u_{2i+1} - u_{2i-1})^2 + (u_{2i+2} - u_{2i})^2} - 1 \right)^2. \quad (2.5)$$

Model 4a

The sixth objective function describes the potential energy of the system on the fourth row of Fig. 2.1. It includes 80 variables and is defined as:

$$\begin{aligned} m(\underline{u}) = & -\underline{g}^T \underline{u} + \sum_{i=1}^{10} a_i \left(\sqrt{(u_{2i-1})^2 + (1 + u_{2i})^2} - 1 \right)^2 + \\ & \sum_{i=1}^9 a_{i+10} \left(\sqrt{(1 + u_{2i+1})^2 + (1 + u_{2i+2})^2} - \sqrt{2} \right)^2 + \\ & \sum_{i=1}^9 a_{i+19} \left(\sqrt{(1 - u_{2i-1})^2 + (1 + u_{2i})^2} - \sqrt{2} \right)^2 + \\ & \sum_{i=1}^{30} a_{i+28} \left(\sqrt{(1 + u_{2i+20} - u_{2i})^2 + (u_{2i+19} - u_{2i-1})^2} - 1 \right)^2 + \\ & \sum_{i=1}^{36} a_{i+58} \left(\sqrt{(1 + u_{2i+1+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i-1+2\lfloor \frac{i-1}{9} \rfloor})^2 + (u_{2i+2+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i+2\lfloor \frac{i-1}{9} \rfloor})^2} - 1 \right)^2 + \\ & \sum_{i=1}^{27} a_{i+94} \left(\sqrt{(1 + u_{2i+21+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i-1+2\lfloor \frac{i-1}{9} \rfloor})^2 + (1 + u_{2i+22+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i+2\lfloor \frac{i-1}{9} \rfloor})^2} - \sqrt{2} \right)^2 + \\ & \sum_{i=1}^{27} a_{i+121} \left(\sqrt{(1 - u_{2i+19+2\lfloor \frac{i-1}{9} \rfloor} + u_{2i+1+2\lfloor \frac{i-1}{9} \rfloor})^2 + (1 + u_{2i+20+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i+2+2\lfloor \frac{i-1}{9} \rfloor})^2} - \sqrt{2} \right)^2, \end{aligned} \quad (2.6)$$

where

$$\lfloor x \rfloor = \text{floor}(x) = b \quad \text{if } b \leq x < b + 1, \quad (2.7)$$

with $x \in \mathbb{R}$ and $b \in \mathbb{Z}$ (where $\mathbb{Z}$ denotes the set of all integers).

Model 4b

The seventh objective function describes the potential energy of the system on the last row of Fig. 2.1. It is thus the same as that of Model 4a, except that the contributions of the diagonal springs are omitted:

$$\begin{aligned}
m(\underline{u}) = & -\underline{g}^T \underline{u} + \sum_{i=1}^{10} a_i \left(\sqrt{(u_{2i-1})^2 + (1 + u_{2i})^2} - 1 \right)^2 + \\
& \sum_{i=1}^{30} a_{i+10} \left(\sqrt{(1 + u_{2i+20} - u_{2i})^2 + (u_{2i+19} - u_{2i-1})^2} - 1 \right)^2 + \\
& \sum_{i=1}^{36} a_{i+40} \left(\sqrt{(1 + u_{2i+1+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i-1+2\lfloor \frac{i-1}{9} \rfloor})^2 + (u_{2i+2+2\lfloor \frac{i-1}{9} \rfloor} - u_{2i+2\lfloor \frac{i-1}{9} \rfloor})^2} - 1 \right)^2. \quad (2.8)
\end{aligned}$$

Model X

The final objective function describes the potential energy of the top-right system Fig. 2.1. The potential energy of this system can be expressed as:

$$\begin{aligned}
m(\underline{u}) = & -\underline{g}^T \underline{u} + a_1 \left(\sqrt{(1 + u_2)^2 + u_1^2} - 1 \right)^2 + a_2 \left(\sqrt{(1 + u_3)^2 + (1 + u_4)^2} - \sqrt{2} \right)^2 + \\
& + a_3 \left(\sqrt{(1 - u_3)^2 + (1 + u_4)^2} - \sqrt{2} \right)^2 + a_4 \left(\sqrt{(1 + u_6)^2 + u_5^2} - 1 \right)^2 + \\
& \sum_{i=1}^{10} a_{4+i+4\lfloor \frac{i-1}{2} \rfloor} \left(\sqrt{(u_{2i+2+2\lfloor \frac{i-1}{2} \rfloor} - u_{2i+2\lfloor \frac{i-1}{2} \rfloor})^2 + (1 + u_{2i+1+2\lfloor \frac{i-1}{2} \rfloor} - u_{2i-1+2\lfloor \frac{i-1}{2} \rfloor})^2} - 1 \right)^2 + \\
& \sum_{i=1}^8 a_{3i+4} \left(\sqrt{(1 + u_{2i+6+2\lfloor \frac{i}{2} \rfloor} - u_{2i+2\lfloor \frac{i}{2} \rfloor})^2 + (u_{2i+5+2\lfloor \frac{i}{2} \rfloor} - u_{2i-1+2\lfloor \frac{i}{2} \rfloor})^2} - 1 \right)^2 + \\
& \sum_{i=1}^4 a_{6i+2} \left(\sqrt{(1 + u_{6i+4} - u_{6i-4})^2 + (1 + u_{6i+3} - u_{6i-5})^2} - \sqrt{2} \right)^2 + \\
& \sum_{i=1}^4 a_{6i+3} \left(\sqrt{(1 + u_{6i+4} - u_{6i})^2 + (1 - u_{6i+3} + u_{6i-1})^2} - \sqrt{2} \right)^2. \quad (2.9)
\end{aligned}$$

Genetic algorithms

In this section, we discuss our first type of optimisation approach: genetic algorithms (GAs). GAs only evaluate the objective function itself - not the gradient or the Hessian. This entails that they do not incorporate knowledge of the slope of the objective function, nor the change of the slope of the objective function. This makes them slow (time consuming) if the number of variables is large.

However, an important advantage of GAs is that they do not require an initial guess of the minimiser. Instead, they require an initial search domain. This is an advantage if it is difficult to provide a good initial guess, which can be the case for parameter identification (which is often defined as an optimisation problem, as discussed later on). Another advantage of GAs is that if multiple local minima are present in the initial search domain, GAs may be able to find the lowest of the local minima.

3.1 Elitism & recombination

A basic description of how GAs find an objective function's minimiser is as follows. First, many possible minimisers are randomly placed in an initial search domain (meaning that GAs are so-called 'stochastic optimisation methods'). For each of the possible minimisers, the objective function is evaluated. Based on the values of the objective functions, the possible minimisers are ranked; the worst ones are discarded (those with large values for the objective function) and the best ones (those with low values for the objective function) are used to produce a new generation of possible minimisers. This aspect is referred to 'elitism'.

Each of the possible minimisers that remains after the ranking process will become a 'parent' of new possible minimisers that do not exist yet, the so-called 'children'. Each parent randomly chooses another parent and together they produce not one, but two children. This means that if n_{prnts} remain after the ranking, the next generation will consist of $2n_{prnts}$ children.

When two parents meet, they produce two children. The first child randomly inherits values for the possible minimisers from its parents. The second child inherits the other values. This process is easiest to graphically explain and is therefore visualized in Fig. 3.1. As the children randomly inherit 'DNA' from their parents, this process is referred to as 'recombination' or 'crossover'.

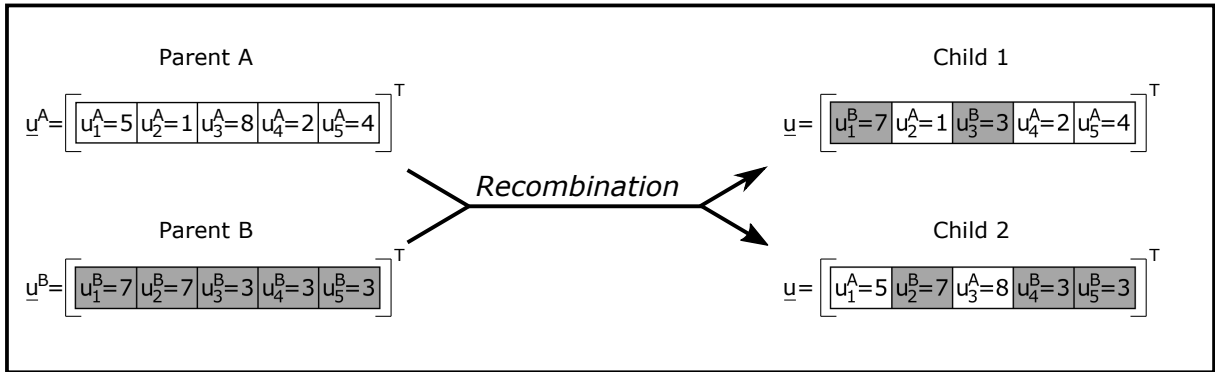


Figure 3.1: 'Recombination': Illustration of how two children inherit values for the different variables from their parents ($n_{var} = 5$).

After the $2n_{prnts}$ children of a new generation are created, the objective function of each child is evaluated. Then, they are ranked together with their parents. Again, the worst ones are

discarded and the best ones will produce a new generation of children. Hence, after the children are ranked together with the parents, the classification between ‘children’ and ‘parents’ is lost, since the best ones will produce the new generation - independent of the fact whether they were parents or children in the previous generation.

The entire process is repeated for n_{maxgen} generations, or until all possible minimisers fall below a user-defined tolerance. In case the process stops because the maximum number of generations is reached (n_{maxgen}), all remaining parents and children are ranked one more time and the one with the lowest value for its objective function is considered the true minimiser. A basic pseudo-code in Table 3.1 summarises the algorithm.

Table 3.1: A basic pseudo-algorithm for GAs, not expressed in a particular syntax.

	<i>%INITIATION</i>
1	Set the maximum number of generations (n_{maxgen}) and the tolerance (tol).
2	Set the number of parents that will produce the next generation: n_{prnts} .
2	Set the number of the initial parents of the 0 th generation: n_{prnts0} .
4	Set the upper and lower bounds of the initial search domain.
5	Randomly place the first possible minimisers in the initial search domain.
6	Evaluate objective function for the n_{prnts0} initial parents of the 0 th generation.
	<i>%GA PROCESS</i>
7	while the possible minimisers $> tol$ & the generation number $< n_{maxgen}$
8	Rank the possible minimisers and keep the n_{prnts} best ones to become parents.
9	for each parent
10	Randomly select the 2 nd parent.
11	Randomly select values of both parents for the 1 st child.
11	Give the remaining values of both parents to the 2 nd child.
12	Evaluate the objective function for the children and store them.
13	end
14	Rank all possible minimisers based on the objective function evaluations.
15	end
	<i>%COMPLETION</i>
16	The minimiser with the lowest objective function evaluation is the ‘true’ minimiser.

So far, we can recognise the following numerical parameters that must be selected by the user:

- the lower and upper bounds for each variable in $\underline{u}$ that define the initial search domain (columns $\underline{u}_{minbounds}$ and $\underline{u}_{maxbounds}$ in the pseudo-algorithms of this chapter),
- the tolerance below which the process is considered to have converged (tol in the pseudo-algorithms of this chapter) ,
- the maximum number of generations (n_{maxgen} in the pseudo-algorithms of this chapter),
- the number of parents that will produce the children of the next generation (n_{prnts} in the pseudo-algorithms of this chapter), and
- the number of parents of the 0th generation (n_{prnts0} in the pseudo-algorithms of this chapter).

Implementation details

A more detailed pseudo-algorithm of Table 3.1 in MATLAB syntax is presented in Table 3.2 for the objective functions of Chapter 2. Two notable differences between the pseudo-algorithms of Tables 3.1 and 3.2 can be observed. These differences regard the use of MATLAB functions `randperm` and `randsample`, which are relatively time-consuming functions. The pseudo-algorithm of Table 3.2 is formulated such that the use of `randperm` and `randsample` is minimised. This makes the pseudo-algorithm of Table 3.2 not entirely stochastic, but the implementation is substantially faster.

In lines 10 and 11 of Table 3.1, each parent randomly selects a second parent and for each first child the recombination of the values of the two parents is randomly performed. However, in lines 31 & 32 of Table 3.2 no truly random selection of the second parent. Instead, the randomness of the parent selection is performed only once per generation: for all parents on lines 26 & 27). The random genome selection is also only performed once per generation (line 28).

Line 26 randomly permutes the list of parents. If ten parents are present, line 26 would for instance produce:

$$\underline{random_sequence} = [4 \ 7 \ 9 \ 5 \ 3 \ 10 \ 8 \ 2 \ 1 \ 6]. \quad (3.1)$$

Lines 31 and 32 take components i and $i+1$ of this list as the first and second parents, respectively. Hence, the decision with whom each parent will team up to produce two children is decided in line 26. Note that in line 27, the first component of list *random_sequence* is copied to the end of this list, so that *random_sequence* is ultimately of length $n_{prnts} + 1$ and hence, all parents are used twice. The fact that each parent is used exactly twice removes some stochasticity from the algorithm, but this is to price to pay in order to minimise the use of `randperm` and `randsample`.

The recombination is also randomised for each generation at once (line 28), so that function `randsample` does not have called n_{prnts} times in the `for`-loop of lines 30-39. To enable the random selection of the possible genomes in line 34, all possible genomes must first be produced. This is performed on lines 9-19. The number of possible genomes depends on the number of variables of the objective function (n_{var}) and in the case of $n_{var} = 3$, the possible genomes are:

$$\underline{\underline{possible_genomes1}} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \quad (3.2)$$

where each column represent a genome. However, because the first and the last genome are the same as the first and the second parent, they are removed on lines 17 and 18. This means for the case of $n_{var} = 3$ after line 18 that:

$$\underline{\underline{possible_genomes1}} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}. \quad (3.3)$$

Table 3.2: A pseudo-algorithm of Table 3.1 in MATLAB syntax for the objective functions of Chapter 2. $\underline{U}$: matrix in which realisations are stored. $\underline{M}$: column in which objective function evaluations are stored.

	<i>%INITIALISATION</i>
1	Set $\underline{a}$, $\underline{g}$, $n_{var} = \text{length}(\underline{g})$ <i>% parameters of the objective function</i>
2	Set n_{maxgen} , n_{prnts} , n_{prnts0} , tol , $\underline{u}_{minbounds}$, $\underline{u}_{maxbounds}$ <i>% parameters of the GA</i>
	<i>% Initialise parents of 0th generation and evaluate their objective functions</i>
3	$\underline{U} = \underline{u}_{minbounds} + \text{rand}(n_{var}, n_{prnts0}) \cdot (\underline{u}_{maxbounds} - \underline{u}_{minbounds})$
4	$\underline{M} = \text{zeros}(1, n_{prnts0})$
5	for $i = 1 : n_{prnts0}$
6	$\underline{u} = \underline{U}(:, i)$
7	Compute $m_i = m(\underline{u})$, store $\underline{M}(i) = m_i$
8	end
	<i>% Initialisation of the possible genomes and their counter part for later use</i>
9	$\underline{possible_genomes1} = \text{zeros}(n_{var}, 2^{n_{var}})$
10	for $i = 1 : n_{var}$
11	for $j = 1 : 2^{i-1}$
12	$len = \frac{2^{n_{var}}}{2^i}$
13	$nr = 1 + \frac{(j-1) \cdot 2^{n_{var}}}{2^{i-1}}$
14	$\underline{possible_genomes1}(i, nr : nr + len - 1) = \text{ones}(1, len)$
15	end
16	end
17	$\underline{rem} = [1 \ 2^{n_{var}}]$
18	$\underline{possible_genomes1}(:, \underline{rem}) = []$
19	$\underline{possible_genomes2} = \text{ones}(n_{var}, 2^{n_{var}} - 2) - \underline{possible_genomes1}$

	<i>%GA PROCESS</i>
20	$n_{gen} = 0$, $res = 10^8$, <i>% initialise values for while-loop</i>
21	while $res > tol$ && $n_{gen} < n_{maxgen}$
	<i>%Rank possible minimisers</i>
22	$[\underline{M}, b] = \text{sort}(\underline{M}, 'ascend')$ <i>% sort $\underline{M}$ from low to high</i>
23	$\underline{U} = \underline{U}(:, b)$ <i>% sort $\underline{U}$ accordingly</i>
	<i>%Store best ones and allocate space for children</i>
24	$\underline{U} = [\underline{U}(:, 1 : n_{prnts}) \ \text{zeros}(n_{var}, 2n_{prnts})]$
25	$\underline{M} = [\underline{M}(1 : n_{prnts}) \ \text{zeros}(1, 2n_{prnts})]$
26	$\underline{random_sequence} = \text{randperm}(n_{prnts})$
27	$\underline{random_sequence} = [\underline{random_sequence}, \underline{random_sequence}(1)]$
28	$\underline{random_genome_list} = \text{randsample}(2^{n_{var}} - 2, n_{prnts}, \text{true})$
	<i>%Produce children and evaluate their objective functions</i>
29	$cnt = n_{prnts} + 1$, $cnt_genome = 1$
30	for $i = 1 : n_{prnts}$
31	$\underline{u}_{p1} = \underline{U}(:, \underline{random_sequence}(i))$ <i>%values of the 1st parent</i>
32	$\underline{u}_{p2} = \underline{U}(:, \underline{random_sequence}(i + 1))$ <i>%values of 2nd parent</i>
33	$nr_genome = \underline{random_genome_list}(cnt_genome)$
34	$\underline{u}_{c1} = \underline{possible_genomes1}(:, nr_genome) \cdot \underline{u}_{p1} + \underline{possible_genomes2}(:, nr_genome) \cdot \underline{u}_{p2}$
35	$\underline{u}_{c2} = \underline{possible_genomes2}(:, nr_genome) \cdot \underline{u}_{p1} + \underline{possible_genomes1}(:, nr_genome) \cdot \underline{u}_{p2}$
36	Compute $m_{c1} = m(\underline{u}_{c1})$ & $m_{c2} = m(\underline{u}_{c2})$
37	$\underline{U}(:, cnt : cnt + 1) = [\underline{u}_{c1}, \underline{u}_{c2}]$, $\underline{M}(cnt : cnt + 1) = [m_{c1}; m_{c2}]$ <i>%store</i>
38	$cnt = cnt + 2$, $cnt_genome = cnt_genome + 1$
39	end
40	$n_{gen} = n_{gen} + 1$, $res = \sqrt{\left(\max(\underline{U}, [], 2) - \min(\underline{U}, [], 2)\right)^T * \left(\max(\underline{U}, [], 2) - \min(\underline{U}, [], 2)\right)}$
41	end
	<i>%COMPLETION</i>
42	$[\underline{M}, b] = \text{sort}(\underline{M}, 'ascend')$, $\underline{U} = \underline{U}(:, b)$
43	$m^* = \underline{M}(1)$, $\underline{u}^* = \underline{U}(:, 1)$

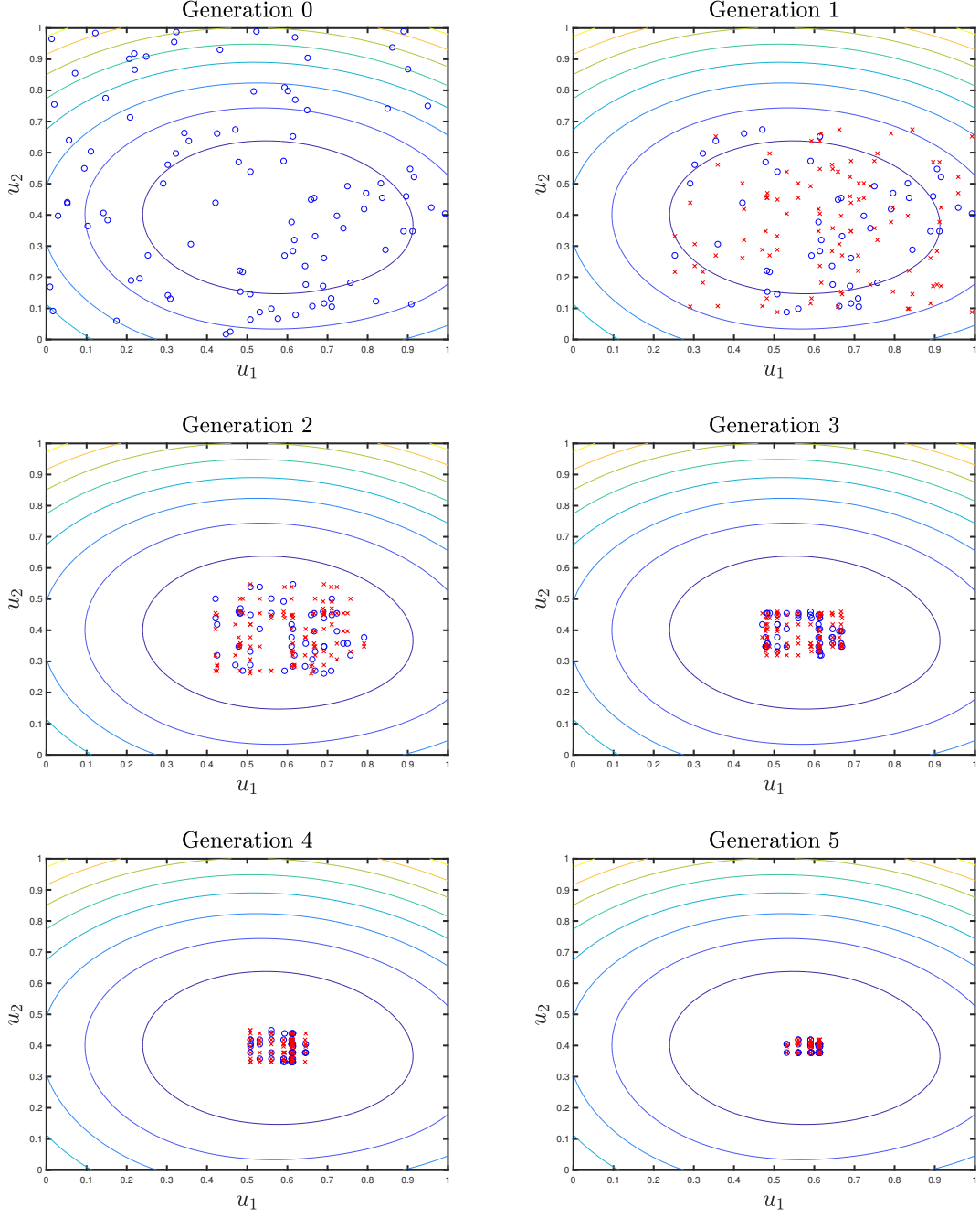


Figure 3.2: Parents (blue circles) and children (red crosses) of the GA of Tables 3.1 and 3.2 for the objective function of Model 1 ($a_1 = a_2 = 1$ and $g_1 = g_2 = 1$), together with the objective function's isolines. *An isoline is a line for which the objective function has a constant value.*

Because of the way the handing over of values of parents to the first and second child is implemented (lines 34 & 35), we also need a mirror image of possible_genomes1 (see line 19):

$$\underline{\underline{possible_genomes2}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}. \quad (3.4)$$

Results

In Fig. 3.2, parents and children are presented for the first generations of the objective function of Model 1 with the algorithm of Table 3.2. It can be observed that the parents and children seem to converge to the minimum, but a rather organised pattern appears. It is also clearly visible that most of the children fall on top of existing parents as soon as such a pattern appears (and it always appears with the current GA). Consequently, the space of the minimisers is not properly sampled and the minimiser found after convergence is in most cases rather inaccurate. This inaccuracy is illustrated in Fig. 3.3, where several minimisers are presented when the GA is run several times.

The problem of the GA in this subsection is that it includes insufficient randomisation. This insufficient randomisation is not caused by the removal of some stochasticity in the implementation of Table 3.2. Instead, it is intrinsic to the GA of this subsection.

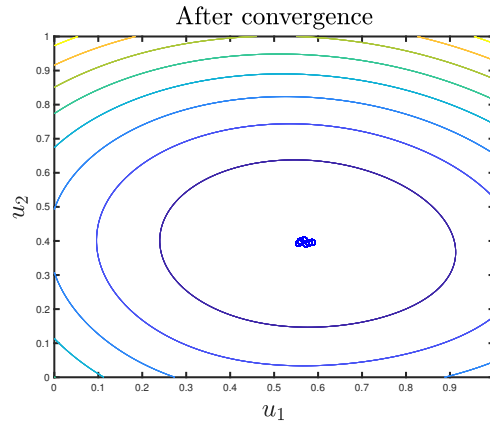


Figure 3.3: Several minimisers found after convergence with the GA of Tables 3.1 and 3.2 for the objective function of Model 1 ($a_1 = a_2 = 1$ and $g_1 = g_2 = 1$).

3.2 Mutations

In order to sample the space of the minimisers better, more randomisation normally added GAs. This is achieved by ‘mutating’ each child.

The process of adding a mutation to a child is illustrated in Fig. 3.4. First, a child is produced in the same manner as in the previous subsection (thin red cross, parents as blue circles). Subsequently, its location is randomly shifted in a circle (bold red cross) that is centred around its ‘unmutated’ location. The radius of the circle is $R_{rand}d$, where R_{rand} denotes a parameter to be defined by the user and d denotes the distance between the parents (i.e. L^2 -norm). As distance d generally speaking decreases (on average) for each generation, the radius also decreases per generation (on average) - which is desired.

As d can be regarded as the distance between the two parents, $R_{rand}d$ can be regarded as the radius of a circle in case of $n_{var} = 2$ and in the case that $n_{var} = 3$, $R_{rand}d$ can be considered as the radius of a sphere. In case that $n_{var} > 3$, the meaning of $R_{rand}d$ is less trivial to imagine,

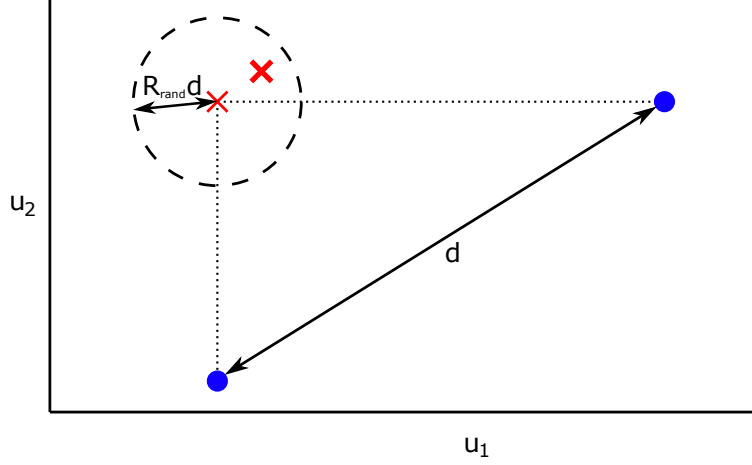


Figure 3.4: Illustration of ‘mutation’ in case $n_{var} = 2$: First, two parents (blue circles) produce a child without mutation (thin red cross). The child is then ‘mutated’ (bold red cross), meaning that it is randomly located in a circle around its unmutated location. The radius of the circle is $R_{rand}d$, where d denotes the distance between the parents and R_{rand} denotes a parameter to be defined by the user.

as d is the L^2 -norm of the difference between the two parents:

$$d = \|\underline{u}_{p1} - \underline{u}_{p2}\| = \sqrt{\sum_{i=1}^{n_{var}} (u_{p1/i} - u_{p2/i})^2} = \sqrt{(\underline{u}_{p1} - \underline{u}_{p2})^T (\underline{u}_{p1} - \underline{u}_{p2})}. \quad (3.5)$$

Implementation details

Adding mutations to the previous GA only requires a minimum of implementation efforts. In Table 3.3, the additional three lines relative to the previous pseudo-algorithm in MATLAB syntax are presented in red, together with the lines of the previous implementation that surround them.

Table 3.3: The additional lines of MATLAB code (red) that need to be included in the pseudo-code of Table 3.2 to incorporate mutations (together with their surrounding lines of the original pseudo-code in black).

28	<code>random_genome_list = randsample(2^{n_{var}} - 2, n_{prnts}, true)</code>
28b	<code><u>mutations</u> = -R_{rand} + 2R_{rand} rand(n_{var} + 1, 2n_{prnts})</code>
29	<code>cnt = 1, cnt_genome = 1</code>
34	<code><u>u</u>_{c1} = possible_genomes1(:, nr_genome) . * <u>u</u>_{p1} + possible_genomes2(:, nr_genome) . * <u>u</u>_{p2}</code>
34b	<code><u>u</u>_{c1} = <u>u</u>_{c1} + <u>mutations</u>(1, cnt - n_{prnts}) * norm(<u>u</u>_{p1} - <u>u</u>_{p2}) * $\frac{\text{mutations}(2:n_{var}+1, cnt-n_{prnts})}{\text{norm}(\text{mutations}(2:n_{var}+1, cnt-n_{prnts}))}$</code>
35	<code><u>u</u>_{c2} = possible_genomes2(:, nr_genome) . * <u>u</u>_{p1} + possible_genomes1(:, nr_genome) . * <u>u</u>_{p2}</code>
35b	<code><u>u</u>_{c2} = <u>u</u>_{c2} + <u>mutations</u>(1, cnt - n_{prnts} + 1) * norm(<u>u</u>_{p1} - <u>u</u>_{p2}) * $\frac{\text{mutations}(2:n_{var}+1, cnt-n_{prnts}+1)}{\text{norm}(\text{mutations}(2:n_{var}+1, cnt-n_{prnts}+1))}$</code>
36	<code>Compute $m_{c1} = m(\underline{u}_{c1})$ & $m_{c2} = m(\underline{u}_{c2})$</code>

In Table 3.3 matrix mutations introduces the randomness of the mutations. For $n_{var} = 3$ and $n_{prnts} = 4$, mutations is of size 4×8 and with $R_{rand} = 0.5$, it may look as follows:

$$\underline{mutations} = \begin{bmatrix} 0.3147 & 0.1324 & 0.4575 & 0.4572 & -0.0782 & 0.1557 & 0.1787 & 0.1555 \\ 0.4058 & -0.4025 & 0.4649 & -0.0146 & 0.4157 & -0.4643 & 0.2577 & -0.3288 \\ -0.3730 & -0.2215 & -0.3424 & 0.3003 & 0.2922 & 0.3491 & 0.2431 & 0.2060 \\ 0.4134 & 0.0469 & 0.4706 & -0.3581 & 0.4595 & 0.4340 & -0.1078 & -0.4682 \end{bmatrix}. \quad (3.6)$$

Each column consists of the $n_{var} + 1$ random numbers between $-R_{rand}$ and R_{rand} that will be used for each child. Of each column, the first component is used as the random variable that is to be multiplied by d to define the distance between the mutated and unmutated child. The remaining n_{var} random numbers define the direction between the mutated and unmutated child. Because a direction is normal vector in the space of the minimisers (i.e. it has n_{var} components), it can be recognised in lines 34b and 35b as:

$$\frac{\underline{mutations}(2 : n_{var} + 1, i)}{\text{norm}(\underline{mutations}(2 : n_{var} + 1, i))}, \quad (3.7)$$

where i is actually expressed in lines 34b and 35b as $cnt - n_{prnts}$ or $cnt - n_{prnts} + 1$, depending on if the first or second child is considered. Note also that the syntax for distance d is $\text{norm}(\underline{u}_{p1} - \underline{u}_{p2})$ in lines 34b and 35b.

Results

In Fig. 3.5 the results for the GA with mutations ($R_{rand} = 0.5$) are presented for the first few generations of the same objective function as for Fig. 3.2. Comparing both figures, it is clear that much more randomness is present thanks to the mutations. Consequently, the GA is substantially more accurate, meaning that the estimated minimisers indeed converge towards the true minimiser.

3.3 Variations

GAs come in many different types and the one presented here is definitely not the best one. Each type has its own advantages and disadvantages. However, ‘recombination’, ‘elitism’ and ‘mutations’ are shared by many of these variants.

One new variant can for instance be created by not removing the genomes that match the parents themselves from matrix possible_genomes1 (i.e. the column with all ones and the column with all zeros). On their own these two genomes do not add new information as they are duplicates of the parents. In practise however, they sample the space near the existing parents because a mutation is later added to them.

Another variant can be created by forcing all children to remain within the initial search domain, which is not incorporated in the described GA.

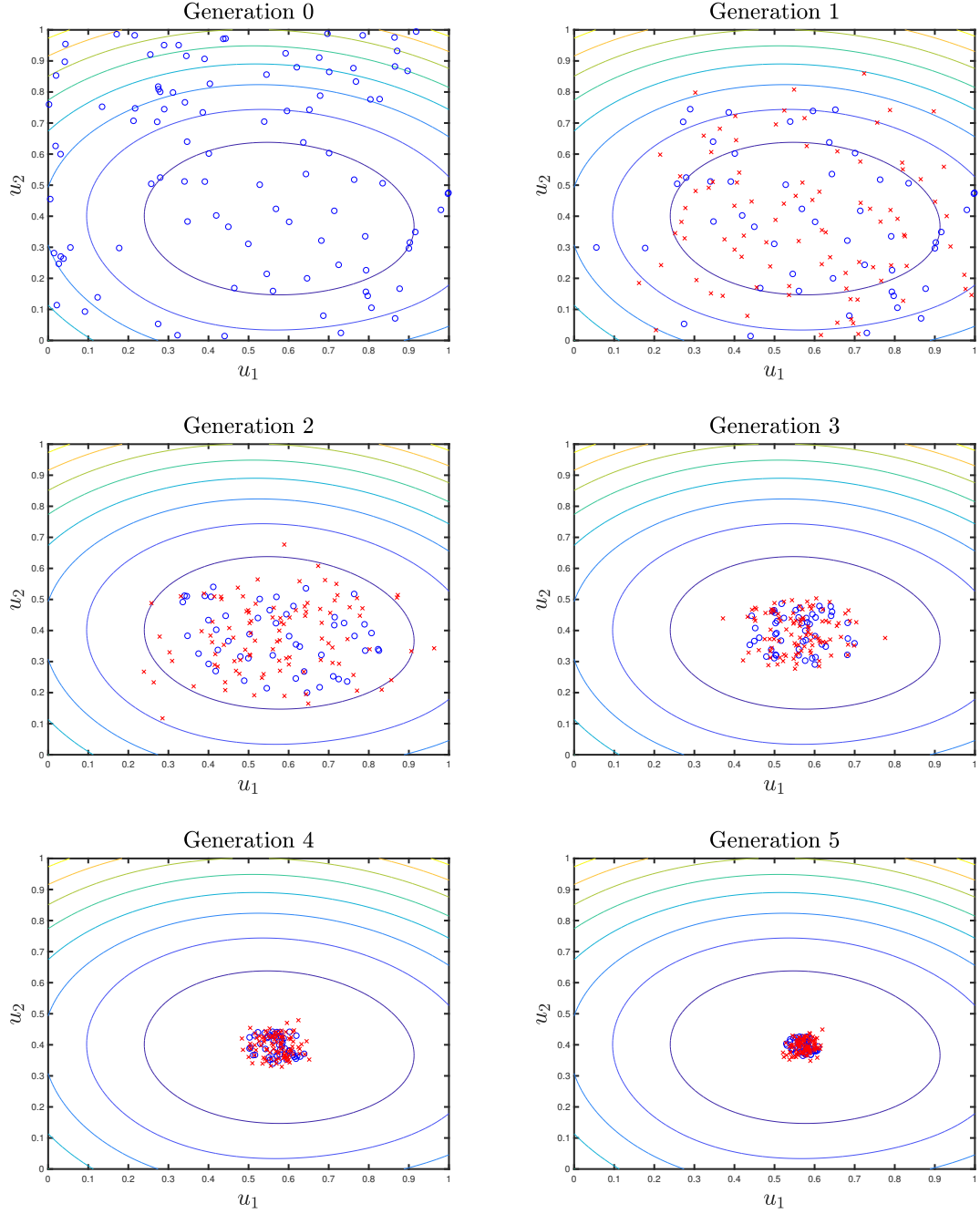


Figure 3.5: Parents (blue circles) and children (red crosses) of the GA with mutations for the objective function of Model 1 ($a_1 = a_2 = 1$ and $g_1 = g_2 = 1$), together with the objective function's isolines. Clearly, substantially more randomness is presented than in Fig. 3.2.

Other variants can for instance be created by changing the method of recombination (i.e. cross over). Above, we have applied so-called *uniform* recombination. This means that the method independently considers for each variable how they will be inherited by the children (see again Fig. 3.1). Another recombination method is single-point recombination. This means that all the variables in front a single cut-off point (randomly selected for each combination of parents) are inherited by one child and the others by the other child (see the two top rows in Fig. 3.6).

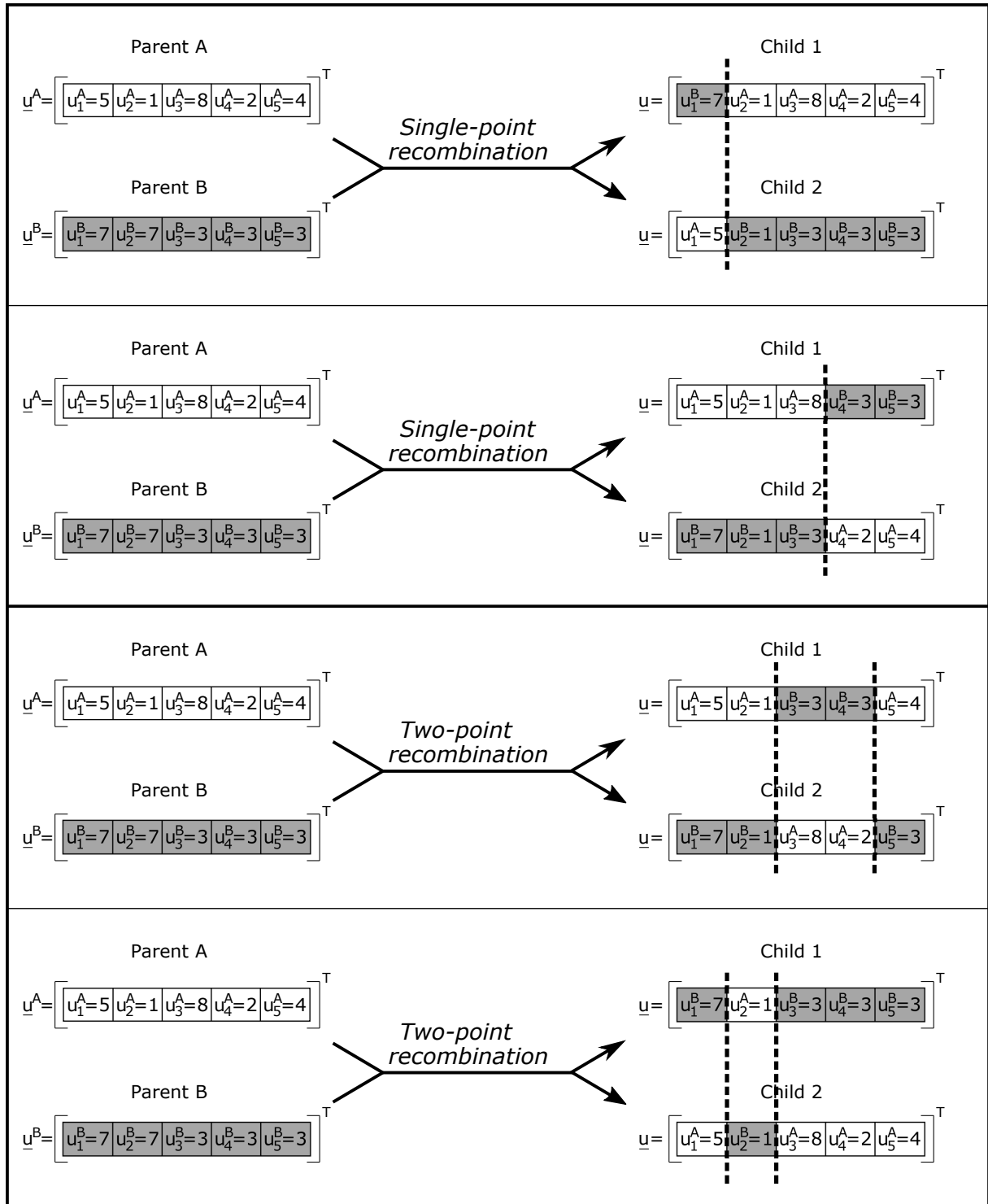


Figure 3.6: Single-point recombination (the two top rows) and two-point recombination (the two bottom rows). The dashed-bold vertical lines on top of the children depict the cut-off points.

This method can be generalised to more than one cut-off point (see the two bottom rows in Fig. 3.6 for two-point recombination). Hence, uniform recombination can be considered as the ultimate generalisation where a cut-off point is placed between all variables (i.e. for uniform recombination, the number of cut-off points equals $n_{var} - 1$).

3.4 Questions

Before answering any of these questions, first read the programming tips at the end of Chapter 1.

Question 3.1 Implement a GA based on the pseudo-algorithms of this chapter and use it to minimise the objective function of Model 1 with $\underline{a} = \underline{1}$ and $\underline{g} = \underline{1}$.

(A) Use the following numerical parameters: $n_{prnts0} = 100$, $n_{prnts} = 50$, $n_{maxgen} = 10^2$, $R_{rand} = 10^{-15}$, $\underline{u}_{minbounds} = \underline{0}$, $\underline{u}_{maxbounds} = \underline{1}$ and $tol = 10^{-7}$. A value of (almost) zero for R_{rand} means that no mutation takes place. Run the code several times and observe that each time the minimisers are different (as in Fig. 3.3).

(B) Keep all parameters the same, except set $R_{rand} = 0.5$. Run the code several times and observe that each time the minimisers are (practically) the same. (The minimisers are reported in the chapter with solutions at the end of the notes.) What is the another substantial difference that occurs compared to the case $R_{rand} = 10^{-15}$?

Question 3.2 Use your implemented GA to minimise the objective function of Model 2a with $\underline{a} = \underline{1}$ and $\underline{g} = [0 \ 1 \ 1 \ 0]^T$. Use the following numerical parameters: $n_{prnts0} = 10^3$, $n_{prnts} = 500$, $n_{maxgen} = 200$, $R_{rand} = 0.5$, $\underline{u}_{minbounds} = \underline{0}$, $\underline{u}_{maxbounds} = \underline{1}$ and $tol = 10^{-7}$.

Question 3.3 Use your implemented GA to minimise the objective function of Model 2b with $\underline{a} = \underline{1}$ and $\underline{g} = [0 \ 1 \ 1 \ 0]^T$. Use the same numerical parameters as in **Question 3.2**. What particularity do you observe about the minimiser that is possible due to the fact that mutations are incorporated?

Question 3.4 Use your implemented GA to minimise the objective function of Model 3a with $\underline{a} = \underline{1}$ and $\underline{g} = \underline{0}$, except $g_2 = g_{19} = 1$. Use the same numerical parameters as in the two previous questions.

Question 3.5 Use your implemented GA to minimise the objective function of Model 3b with $\underline{a} = \underline{1}$ and $\underline{g} = \underline{0}$, except $g_2 = g_{19} = 1$. Use the same numerical parameters as in the previous questions, except set $n_{maxgen} = 5 \cdot 10^3$. Many more generations are necessary than for Model 3a because the characteristic of the objective function has changed. This is not caused due to the fact that the minimiser is outside of the initial search domain, because if you change the bounds of the initial search domain, you will observe that roughly the same number of generations are required to reach convergence.

The next questions do not focus on Model 4a, 4b and X of the previous chapter, because the GA will require even more time to reach convergence because of the increase of the number of variables. Instead, the next questions focus on the ability of the implemented GA to find the lowest minimum of several minima inside the initial search domain.

Question 3.6 Use your implemented GA to minimise the following objective function with two variables:

$$m(\underline{u}) = -e^{-50\sqrt{(u_1-0.25)^2+(u_2-0.25)^2}} - 1.2e^{-50\sqrt{(u_1-0.75)^2+(u_2-0.75)^2}} + \sqrt{(u_1 - 0.35)^2 + (u_2 - 0.75)^2} - 0.5. \quad (3.8)$$

The objective function has three minima ($m^* \approx -0.5$, $\underline{u}^* = [0.35 \ 0.75]^T$ and $m^* \approx -0.99$, $\underline{u}^* = [0.25 \ 0.25]^T$), of which the one given by $m^* = -1.3$, $\underline{u}^* = [0.75 \ 0.75]^T$ is the global minima (see Fig. 3.7). If you run the GA several times with the same numerical parameters as for Question 3.5 (but with $n_{genmax} = 100$), you observe it often finds the global minimum, but not always.

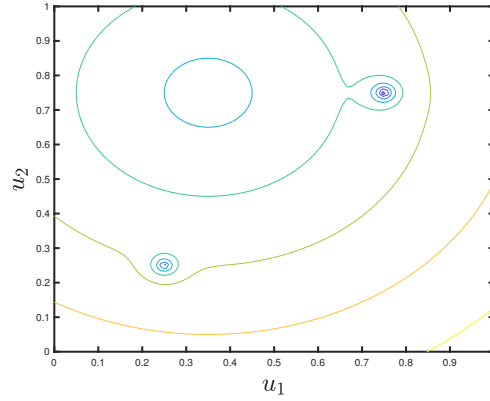


Figure 3.7: The isolines of the objective function in Eq. (3.8).

Question 3.7 Use your implemented GA to minimise the following objective function:

$$m(\underline{u}) = -e^{-50\sqrt{(u_1-0.25)^2+(u_2-0.25)^2+(u_3-0.25)^2}} - 1.2e^{-50\sqrt{(u_1-0.75)^2+(u_2-0.75)^2+(u_3-0.75)^2}} + \sqrt{(u_1 - 0.35)^2 + (u_2 - 0.75)^2 + (u_3 - 0.35)^2} - 0.5, \quad (3.9)$$

which is very similar to the one of Eq. (3.8), except that it depends on three variables. Run the GA with the same parameters as for the previous question and observe that most of the times the local minimum at $\underline{u}^* = [0.35 \ 0.75 \ 0.35]^T$ is found, instead of the global minimum at $\underline{u}^* = [0.75 \ 0.75 \ 0.75]^T$. Determine how to change the numerical parameters such that the chance of converging to the global minimum increases.

Gradient descent approaches

Genetic algorithms do not incorporate knowledge about the gradient of the objective function, since only the objective function is evaluated for each parent/child. In this chapter, we will discuss gradient descent approaches which do include knowledge of the gradient (but not of the Hessian). Gradient descent approaches are generally faster than approaches that do not take gradient information into account (of which GAs are just one example). The reason for this increase of speed is that the gradient provides information in which direction the estimated minimiser must be updated in order to reduce the objective function. In GAs this information is not known and therefore, they propose many new realisations (i.e. children) that increase the objective function, instead of decrease.

Gradient descent approaches do not start with an initial search domain, but with a single realisation of the estimated minimiser - often called the *initial guess*. The gradient at this realisation tells us in which direction the estimated minimiser can be updated to decrease the objective function. After the estimated minimiser is updated, the objective function and its gradient are again evaluated (at the updated estimate of the minimiser), and a new direction in which to update the estimated minimiser becomes available. Numerous of such updates (i.e. iterations) are performed, one after another, until the objective function does not reduce anymore. When an update does not yield a (sufficient) reduction of the objective function, it is assumed that the minimum and the minimiser are found.

4.1 Steepest descent direction

Gradient descent algorithms start with a single realisation of the estimated minimiser, i.e. the initial guess. The estimated minimiser is continuously updated (i.e. corrected) according to:

$$\underline{u}^{i+1} = \underline{u}^i + \alpha \underline{h}^i, \quad (4.1)$$

where $\underline{u}^i$ denotes the current estimated minimiser (at the i^{th} iteration), $\underline{h}^i$ denotes a descent direction at the i^{th} iteration, α denotes the stepsize (i.e. ‘how far the estimated minimiser will be moved’ in descent direction $\underline{h}^i$) and $\underline{u}^{i+1}$ denotes the new estimated minimiser.

Steepest descent algorithms make use of the knowledge that the gradient of the objective function at $\underline{u}^i$:

$$\underline{f}^i = \left. \frac{\partial m}{\partial \underline{u}} \right|_{\underline{u}^i}, \quad (4.2)$$

points in the direction of the maximum increase of the objective function. Steepest descent algorithms therefore propose a step in the opposite direction of the gradient in order to achieve a reduction of the objective function. In other words, they use $\underline{h}^i = -\underline{f}^i$. Note that notation $\left|_{\underline{u}^i}$ means that gradient $\frac{\partial m}{\partial \underline{u}}$ is evaluated at $\underline{u}^i$ (i.e. at the estimated minimiser of the i^{th} iteration).

In Table 4.1, a pseudo-algorithm is presented for the steepest descent approach with a fixed stepsize (i.e. α is a constant value). In Fig. 4.1 you can see an illustration of the steepest descent algorithm applied to the Rosenbrock function:

$$m(\underline{u}) = 10(u_2 - (u_1)^2)^2 + (u_1 - 1)^2. \quad (4.3)$$

Clearly, only a few updates/iterations are required if the stepsize is large and hence, the algorithm is fast (left diagram in Fig. 4.1). For large stepsizes however, the estimate of the minimum and minimiser are relatively inaccurate. If a small stepsize is used, the accuracy of the minimum and minimiser is much better, but as many more updates/iterations are required, the algorithm is substantially slower (right diagram in Fig. 4.1).

Table 4.1: A pseudo-algorithm for a steepest descent algorithm with a fixed stepsize.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}$ <i>% set parameters of the objective function</i>
2	Set $\alpha, \underline{u}$ <i>% set stepsize and initial guess</i>

	<i>%STEEPEST DESCENT ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$
4	$m^{old} = 10^{100}$ <i>% dummy value</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}$
7	Compute $\underline{f} = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
8	$\underline{h} = -\underline{f}$
9	$\underline{u} = \underline{u} + \alpha \underline{h}$
10	Compute $m^{new} = m(\underline{u})$
11	end

	<i>%COMPLETION</i>
12	$m^* = m^{old}, \underline{u}^* = \underline{u} - \alpha \underline{h}$

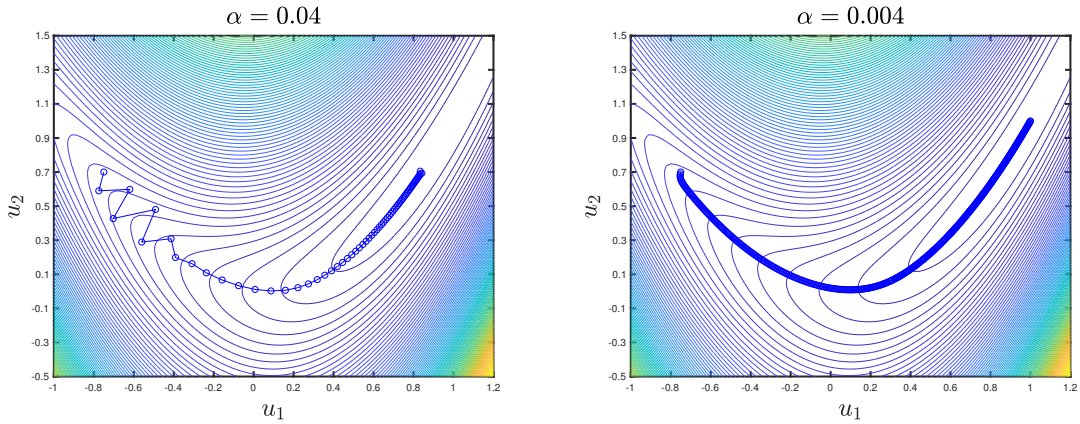


Figure 4.1: Illustration of the updates of the estimated minimiser (blue circles, starting from $\underline{u} = [-0.75 \ 0.7]^T$) according to the steepest descent algorithm with two fixed stepsizes (α) for the Rosenbrock function.

4.2 Backtracking line search

Because the evaluation of the gradient requires more time than the evaluation of the objective function, gradient descent algorithms are faster if they investigate the reduction of the objective function for several stepsizes in the search direction than if they use a fixed stepsize. This is called a *line search*, as search direction $\underline{h}_i$ is fixed for each iteration, but the stepsize not. Some line search algorithms take this to the extreme; they consider that the best stepsize is the

one that minimises the objective function. Hence, these line search algorithm aim to solve the following (1D) minimisation problem:

$$\alpha^* = \min_{\alpha} m(\underline{u}^i + \alpha \underline{h}^i), \quad (4.4)$$

which needs to be solved for each iteration. The issue here is that quite a large number of stepsizes must be sampled (i.e. the objective function must be evaluated for many different values of α), before minimiser α^* is found. This may in fact reduce the speed of the gradient descent algorithm.

The *backtracking line search* is a well-known algorithm that does not search the stepsize that minimises the objective function. Instead, the backtracking line search accepts many possible stepsizes. The backtracking line search initially proposes a large stepsize for which it evaluates the objective function. If the objective function meets some criterion/criteria, it accepts the stepsize and updates the estimated minimiser accordingly. If the objective function at the proposed stepsize does not meet the criterion/criteria, the backtracking line search decreases the stepsize until a stepsize is found for which the objective function meets the criterion/criteria.

The most important criterion to accept a stepsize in the backtracking line search is that the objective function at the stepsize is smaller than the objective function at the current estimated minimiser (for which $\alpha = 0$). However, if any stepsize that reduces the objective function would be accepted, the overall convergence behaviour of the gradient descent algorithm is poor (i.e. the algorithm is slow). The criterion is therefore adjusted by stating that if a large stepsize is proposed, it will only be accepted if a large reduction of the objective function is made. If on the other hand a small stepsize is proposed, a smaller reduction of the objective function suffices.

This criterion was proposed by Armijo, resulting in the Armijo rule, which states that objective functions of acceptable stepsizes (α) abide:

$$m(\underline{u}^i + \alpha \underline{h}^i) \leq m(\underline{u}^i) + c \alpha \underline{h}^{iT} \underline{f}^i, \quad (4.5)$$

where $0 < c < 1$ denotes a numerical parameter that controls how much decrease we accept for a certain stepsize. This is illustrated in Fig. 4.2.

A pseudo-algorithm for the steepest descent approach including the backtracking line search is presented in Table 4.2. A result of the steepest descent algorithm with the backtracking line search for the Rosenbrock function is presented in the right diagram of Fig. 4.3. The number of iterations has clearly reduced compared to the algorithm with a fixed stepsize (left diagram of Fig. 4.3).

4.3 Conjugate gradient direction

Consecutive search directions of the steepest descent approach make quite a particular pattern as can be seen in the bottom diagram of Fig. 4.3 in which a zoom of consecutive estimates near

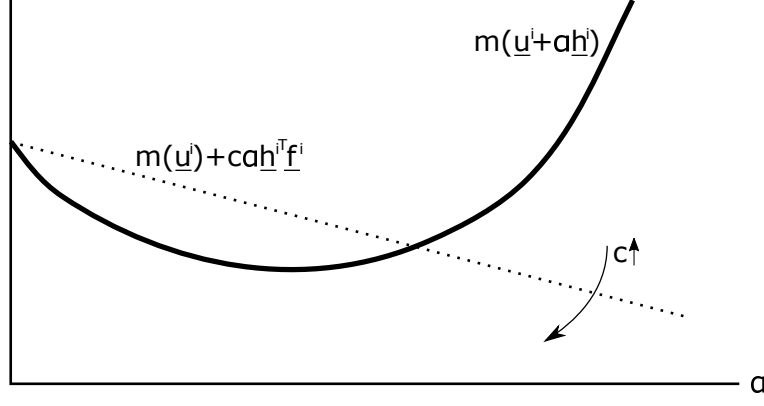


Figure 4.2: Illustration of the objective function (in the search direction) as a function of α (solid). If a stepsize is proposed for which the Armijo rule holds (i.e. for which the objective function is below the dashed line), the stepsize is accepted. If the objective function at the proposed stepsize is larger than the dashed line, the stepsize is reduced by a factor of r (r is not shown in the diagram) and the objective function is evaluated again and investigated if it is below the dashed line. This process is repeated until a suitable stepsize is found. Note that if the value of c is increased, the slope of the dashed line increases.

Table 4.2: A pseudo-algorithm for a steepest descent algorithm with a backtracking line search algorithm based on the Armijo rule. Parameter α_{init} denotes the initial stepsize of each iteration. Parameter $0 < r < 1$ denotes how much the stepsize is decreased if the current stepsize does not abide the Armijo rule.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}$ <i>% set parameters of the objective function</i>
2	Set α_{init} , $0 < c < 1$, $0 < r < 1$, $\underline{u}$ <i>% set parameters of the algorithm</i>

	<i>%STEEPEST DESCENT ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$
4	$m^{old} = 10^{100}$ <i>% dummy value</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}$
7	Compute $\underline{f} = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
8	$\underline{h} = -\underline{f}$
	<i>%Backtracking line search</i>
9	$\alpha = \frac{1}{r}\alpha_{init}$
10	$m^x = 10^{100}$ <i>% dummy value</i>
11	while $m^x > m^{new} + c\alpha\underline{h}^T\underline{f}$
12	$\alpha = r\alpha$ <i>% decrease stepsize by factor r</i>
13	$\underline{u}^x = \underline{u} + \alpha\underline{h}$
14	Compute $m^x = m(\underline{u}^x)$
15	end
16	$m^{new} = m^x$, $\underline{u} = \underline{u}^x$
17	end

	<i>%COMPLETION</i>
18	$m^* = m^{old}$, $\underline{u}^* = \underline{u} - \alpha\underline{h}$

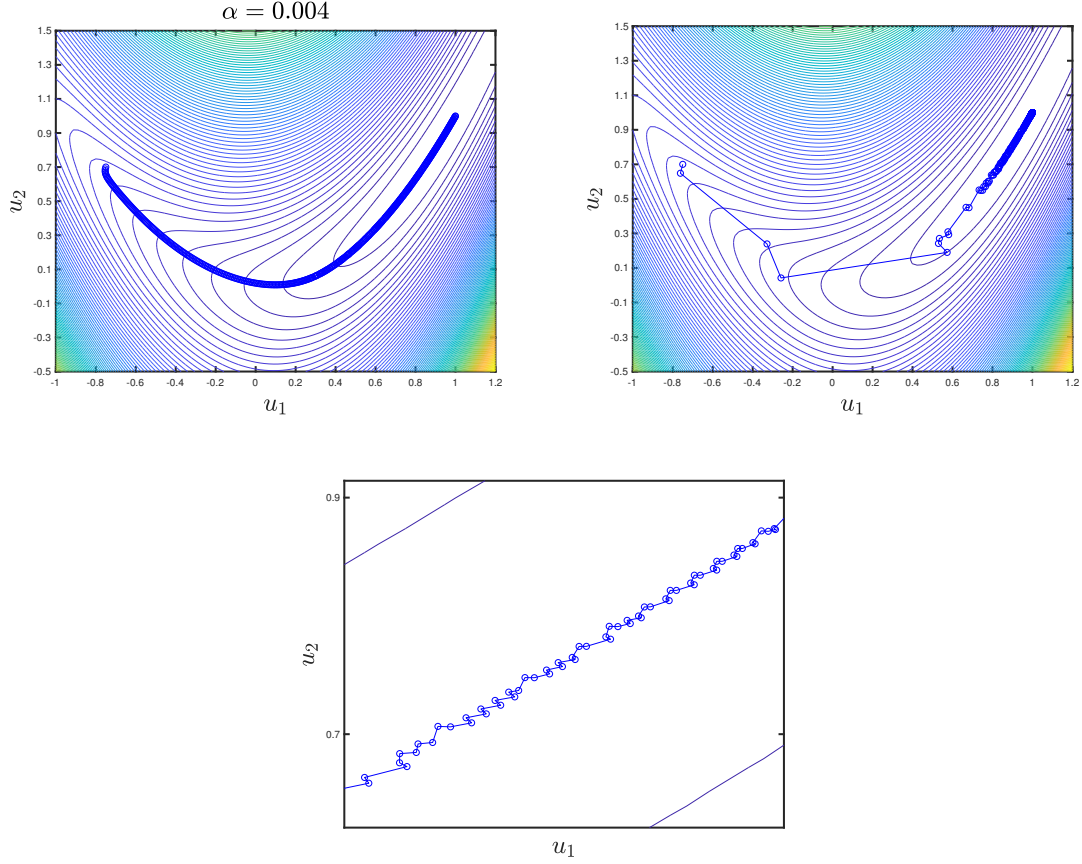


Figure 4.3: Illustration of the updates of the estimated minimiser (blue circles, starting from $u = [-0.75 \ 0.7]^T$) according to the steepest descent algorithm with a backtracking line search for the Rosenbrock function (right), compared to the steepest descent algorithm with a fixed stepsize (left). Bottom: zoom of the right diagram.

the minimiser are presented. We can see that many search directions are almost parallel to each other, together forming some zigzag pattern. If an algorithm would follow the direction of the valley more accurately, substantially less iterations would be required and the speed of the algorithm would be increased.

Conjugate gradient approaches aim to achieve this: they generally do not make a step in the direction of the steepest descent. Instead, they make a step in a direction that combines the current steepest descent direction together with the steepest descent direction and/or search direction of the previous estimate (i.e. previous iteration). Different ratios to weigh the current steepest descent direction with the previous steepest descent direction and/or previous search direction were formulated in the past:

$$\beta = \frac{\underline{f}^{newT} \underline{f}^{new}}{\underline{f}^{oldT} \underline{f}^{old}}, \quad (4.6)$$

$$\beta = \frac{\underline{f}^{newT} (\underline{f}^{new} - \underline{f}^{old})}{\underline{f}^{oldT} \underline{f}^{old}}, \quad (4.7)$$

$$\beta = \frac{\underline{f}^{newT} (\underline{f}^{new} - \underline{f}^{old})}{\underline{h}^{oldT} (\underline{f}^{new} - \underline{f}^{old})}, \quad (4.8)$$

$$\beta = \frac{\underline{f}^{newT} \underline{f}^{new}}{\underline{h}^{oldT} (\underline{f}^{new} - \underline{f}^{old})}. \quad (4.9)$$

Often, it is impossible to know in advance which ratio maximises the speed of the conjugate gradient algorithm (i.e. minimises the number of iterations). In that case, one can for instance perform a few iterations with each ratio (starting from the same initial guess), observe which ratio reduces the objective function the most and assume that the same ratio also behaves the best for the entire minimisation process.

A pseudo-algorithm for a conjugate gradient framework with a two-way backtracking line search is presented in Table 4.3. Since generally none of the ratios behave perfectly for the entire minimisation process, the ratio is occasionally reset so that the current steepest descent direction is taken once in a while. In the pseudo-algorithm of Table 4.3, n_{reset} determines how often every n_{reset} iterations the current steepest descent direction is used. It may be clear from Table 4.3 that in case $\beta < 0$, the current steepest descent direction is also employed.

In the right diagram of Fig. 4.4 it is clearly visible that taking conjugate directions instead of steepest descent directions (left diagram of Fig. 4.4) substantially reduces the number of iterations, whilst the accuracy of the determined minimiser remains relatively unaffected. Hence, conjugate gradient approaches are faster than steepest descent approaches. This becomes substantially more noticeable for objective functions with a large number of variables.

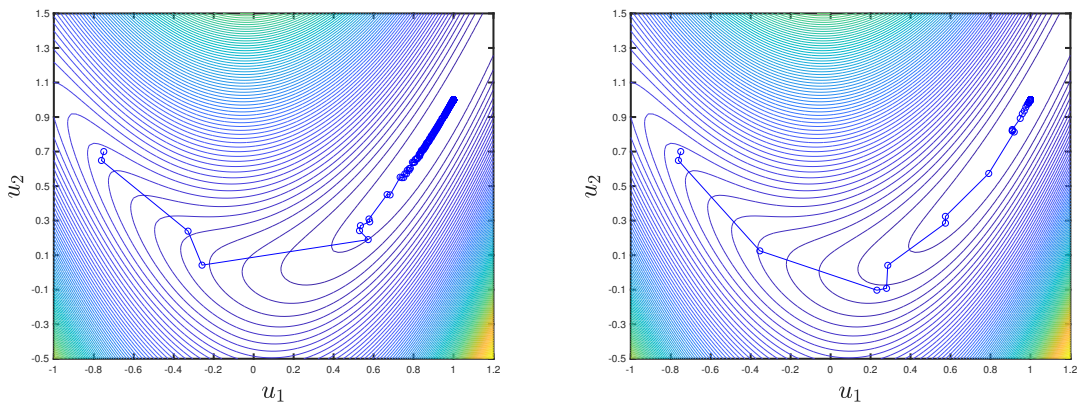


Figure 4.4: Right: Illustration of the updates of the estimated minimiser (blue circles, starting from $\underline{u} = [-0.75 \ 0.7]^T$) according to the conjugate gradient algorithm with a backtracking line search for the Rosenbrock function. Left: the results for the steepest descent algorithm with a backtracking line search.

Table 4.3: A pseudo-algorithm for a conjugate gradient algorithm with a backtracking line search.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}$ <i>% set parameters of the objective function</i>
2	Set $\alpha_{init}, 0 < c < 1, 0 < r < 1, n^{reset}, \underline{u}$ <i>% set parameters of the algorithm</i>
	<i>%CONJUGATE GRADIENT ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$
4	$\alpha = \alpha_{init}, m^{old} = 10^{100}, \underline{f}^{new} = 0, \underline{h}^{new} = 0, cnt = 0$ <i>% dummy values and counter</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}, \underline{f}^{old} = \underline{f}^{new}, \underline{h}^{old} = \underline{h}^{new}$
7	Compute $\underline{f}^{new} = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
	<i>%Determine search direction</i>
8	if $\text{mod}(cnt, n^{reset}) = 0$
9	$\underline{h}^{new} = -\underline{f}^{new}$
10	else
11	β according to Eq. (4.6), (4.7), (4.8) or (4.9)
12	$\underline{h}^{new} = -\underline{f}^{new} + \max([0, \beta]) \underline{h}^{old}$
13	end
	<i>%Backtracking line search</i>
14	$\alpha = \frac{1}{r} \alpha_{init}$
15	$m^x = 10^{100}$ <i>% dummy value</i>
16	while $m^x > m^{new} + c \alpha \underline{h}^{newT} \underline{f}^{new}$
17	$\alpha = r \alpha$ <i>% decrease stepsize by factor r</i>
18	$\underline{u}^x = \underline{u} + \alpha \underline{h}^{new}$
19	Compute $m^x = m(\underline{u}^x)$
20	end
21	$m^{new} = m^x, \underline{u} = \underline{u}^x, cnt = cnt + 1$
22	end
	<i>%COMPLETION</i>
23	$m^* = m^{old}, \underline{u}^* = \underline{u} - \alpha \underline{h}^{new}$

4.4 Variations

Like all optimisation approaches, conjugate gradient approaches come in plenty of variants. We have for instance already seen that the choice for β yields different variants. Another variant can be created by changing the stopping criterion (i.e. vice versa, the continuation criterion). With the stopping criterion of all the pseudo-algorithms presented in this chapter (i.e. ‘stop if $m^{new} \geq m^{old}$ ’), the algorithm may continue for many iterations whilst the reduction of the objective function obtained from one iteration to the next is substantially small. In that case, it may be useful to stop the algorithm if the obtained reduction falls below a user-defined threshold. Continuation criterion ‘**while** $m^{new} < m^{old}$ ’ may then be replaced by ‘**while** $m^{old} - m^{new} > tol$ ’, where tol denotes the user-defined threshold.

Another variable issue is when to accept a step in the backtracking line search. In the above discussion, we have only mentioned the Armijo rule as an upper bound for an acceptable stepsize. However, often a lower bound is also incorporated that effectively ensures that very small stepsizes terminate the line search. The so-called curvature condition is often used to define the lower bound. Depending on the employed version of the curvature condition, the Armijo rule and the curvature condition together are often referred to as the Wolfe conditions or the strong Wolfe conditions.

The presented conjugate gradient direction stops if no (sufficient) reduction of the objective function is found in the search direction. One could in that case resort to another search direction in order to investigate if a change of search direction would yield a (sufficient) reduction of the objective function. The steepest descent direction could then be used as the direction to fall back on. This would yield another variation of a gradient descent approach.

The line search algorithm itself is another possibility to vary. Above, we have discussed the backtracking line search, but there are many other algorithms. In case the evaluation of the gradient requires substantially more time than the evaluation of the objective function, it may increase the approach's speed if the line search determines the stepsize that truly minimises the objective function in the search direction. The golden section search is a line search algorithm that achieves this (i.e. it actually solves the minimisation problem of Eq. (4.4)) by only sampling the stepsize. (Alternatively, one could also use a one-dimensional GA, but this would be slower.) Below, the golden section search is discussed in more detail.

Golden section search

The golden section search uses smartly placed stepsizes to find a stepsize that minimises the objective function in the search direction. (From this perspective, it is similar to the bisection method, which searches for a root instead.) The golden section search starts with the seeding of four stepsizes: two on the boundaries of the search domain in which the minimum is to be found, and two inside the search domain. In the top diagram of Fig. 4.5, these stepsizes are presented as α^1 , α^2 , α^3 and α^4 . Since we will take $\alpha^1 = 0$ for which the objective function is already known, the objective function must initially only be evaluated at α^2 , α^3 and α^4 . In case $\alpha^1 > \alpha^2$, $\alpha^2 > \alpha^3$ and $\alpha^3 > \alpha^4$, it cannot be guaranteed that a minimum is located in the initial search domain and hence, the initial search domain must be enlarged. If the initial search domain is sufficiently large, the golden section search starts to decrease the search domain until it is so small that the two inner stepsizes (α^2 and α^3) both correspond to a single optimum stepsize (or within a user-defined tolerance).

The top diagram in Fig. 4.5 illustrates the two situations that can occur if the initial search domain is sufficiently large. The first situation is characterised by $m^3 > m^2$ (i.e. m^3 corresponds to the star in the top diagram of Fig. 4.5). If $m^3 > m^2$, a minimum must be present in the domain bounded by α^1 and α^3 . Of course, a minimum may also be present for $\alpha > \alpha^3$, but this cannot be guaranteed. Consequently, the search domain is shrunk to bounds α^1 and α^3 (central row, left diagram in Fig. 4.5).

The second situation is characterised by $m^3 < m^2$ (i.e. m^3 corresponds to the square in the top diagram of Fig. 4.5). In this case, a minimum must be present in the domain bounded by α^2 and α^4 . Of course, a minimum may also be present between α^1 and α^2 , but as it is certain that a minimum is present between α^2 and α^4 , the search domain is shrunk to bounds α^2 and α^4 (central row, right diagram in Fig. 4.5).

In case the first situation occurs (i.e. $m^3 > m^2$) and hence, the new search domain is bounded by α^1 and α^3 (central row, left diagram in Fig. 4.5), stepsize α^2 is transformed to stepsize α^3 (bottom row, left diagram in Fig. 4.5) and a new α^2 is seeded for which the objective function must be evaluated (bottom row, left diagram in Fig. 4.5). Subsequently, the configuration in which four stepsizes are present for which the objective function is evaluated is the same as in the top diagram of Fig. 4.5 and hence, sufficient information is now known how to decrease the search domain further.

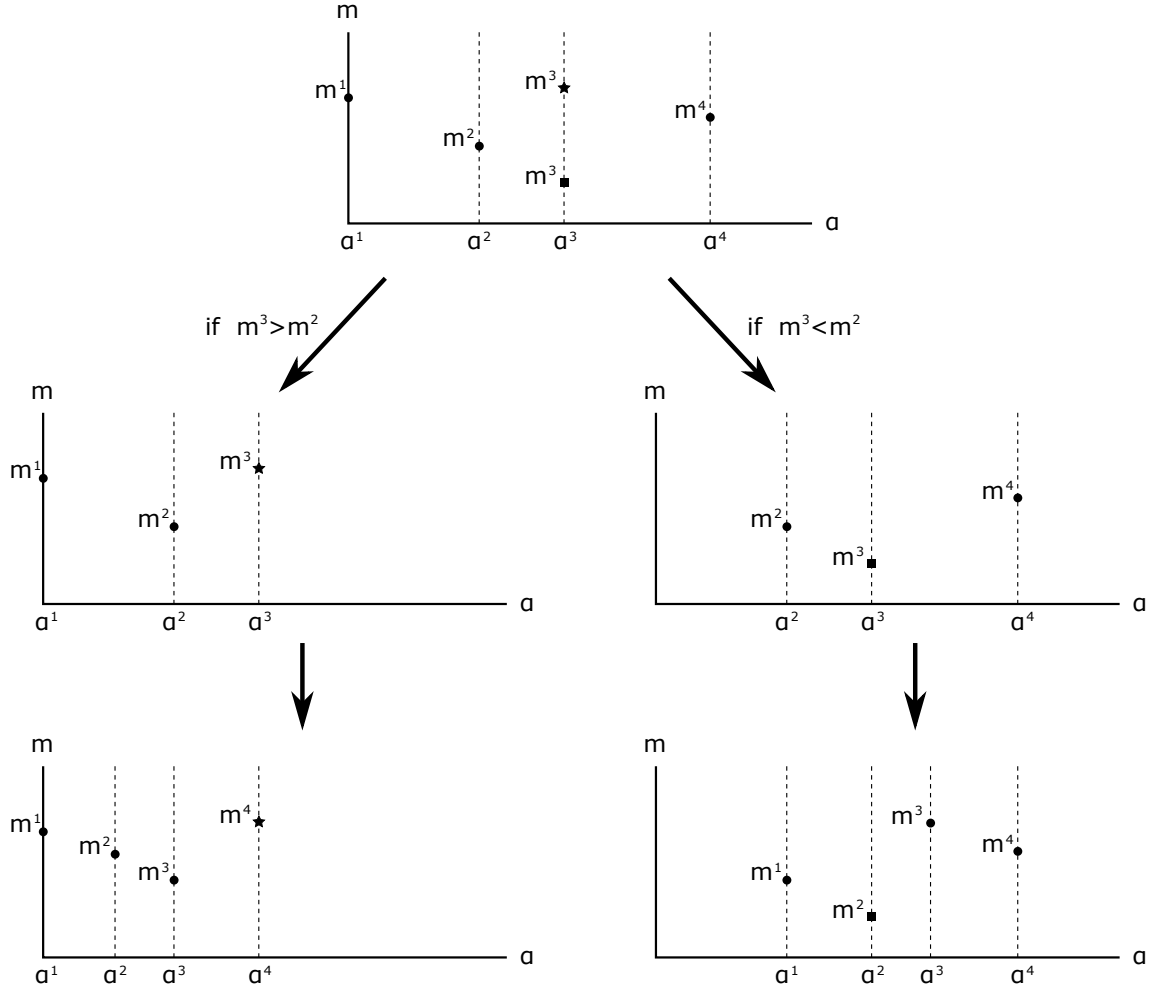


Figure 4.5: Golden section search: updating the search domain and the numbering.

If the second situation occurs (i.e. $m^3 < m^2$) for which the new search domain is bounded by a^2 and a^4 (central row, right diagram in Fig. 4.5), stepsize a^3 is transformed in a^2 and a new stepsize for a^3 must be seeded for which the objective function is evaluated (bottom row, right diagram in Fig. 4.5). At that moment, the situation is the same as it was originally (i.e. four seeded stepsizes for which the objective function is evaluated - cf. the bottom right diagram with the top diagram in Fig. 4.5) and hence, sufficient information is now known how to decrease the search domain further.

The golden section search keeps refining the search domain until the values of a^2 and a^3 are the same, or until the difference between the values of a^2 and a^3 falls below a user-defined threshold. The former stopping criterion is incorporated in the pseudo-algorithm of Table 4.4, in which the golden section search is employed as the line search algorithm for a conjugate gradient approach.

Golden ratio

The smart aspect of the golden section search is that no matter where the optimum stepsize is located in the initial search domain, it always requires the same number of reduction steps of the initial search domain to converge to the optimum stepsize. So how should we place the stepsizes such that the number of reduction steps remains the same irrespective of where the optimum stepsize is located? It turns out that the ratio between distances $a^2 - a^1$, $a^3 - a^2$ and $a^4 - a^3$ holds the secret to this.

Table 4.4: A pseudo-algorithm for a conjugate gradient algorithm with a golden section line search.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}$ <i>% set parameters of the objective function</i>
2	Set $\alpha_{init}, \phi = \frac{1+\sqrt{5}}{2}, n^{reset}, \underline{u}$ <i>% set parameters of the algorithm</i>
	<i>%CONJUGATE GRADIENT ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$
4	$\alpha = \alpha_{init}, m^{old} = 10^{100}, \underline{f}^{new} = \underline{0}, \underline{h}^{new} = \underline{0}, cnt = 0$ <i>% dummy values and counter</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}, \underline{f}^{old} = \underline{f}^{new}, \underline{h}^{old} = \underline{h}^{new}$
7	Compute $\underline{f}^{new} = \frac{\partial m}{\partial \underline{u}} \Big _{\underline{u}}$
	<i>%Determine search direction</i>
8	if $\text{mod}(cnt, n^{reset}) = 0$
9	$\underline{h}^{new} = -\underline{f}^{new}$
10	else
11	β according to Eq. (4.6), (4.7), (4.8) or (4.9)
12	$\underline{h}^{new} = -\underline{f}^{new} + \max([0, \beta]) \underline{h}^{old}$
13	end
14	$\alpha^1 = 0, m^1 = m^{new}$
15	$\alpha^2 = \frac{\alpha_{init}\phi}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}^{new}, \text{ compute } m^2 = m(\underline{u}^x)$
16	$\alpha^3 = \frac{\alpha_{init}(\phi+1)}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}^{new}, \text{ compute } m^3 = m(\underline{u}^x)$
17	$\alpha^4 = \alpha_{init}, \underline{u}^x = \underline{u} + \alpha^4 \underline{h}^{new}, \text{ compute } m^4 = m(\underline{u}^x)$
	<i>%Enlarge initial search domain if necessary</i>
18	while $m^2 < m^1 \ \& \ m^3 < m^2 \ \& \ m^4 < m^3$
19	$\alpha_{init} = 2\alpha_{init}$
20	$\alpha^2 = \frac{\alpha_{init}\phi}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}^{new}, \text{ compute } m^2 = m(\underline{u}^x)$
21	$\alpha^3 = \frac{\alpha_{init}(\phi+1)}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}^{new}, \text{ compute } m^3 = m(\underline{u}^x)$
22	$\alpha^4 = \alpha_{init}, \underline{u}^x = \underline{u} + \alpha^4 \underline{h}^{new}, \text{ compute } m^4 = m(\underline{u}^x)$
23	end
	<i>%Golden section search</i>
24	while $\alpha^3 > \alpha^2$
25	if $m^3 > m^2$
26	$\alpha^4 = \alpha^3, m^4 = m^3, \alpha^3 = \alpha^2, m^3 = m^2$
27	$\alpha^2 = \alpha^1 + \frac{(\alpha^4 - \alpha^1)\phi}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}^{new}, \text{ compute } m^2 = m(\underline{u}^x)$
28	else
29	$\alpha^1 = \alpha^2, m^1 = m^2, \alpha^2 = \alpha^3, m^2 = m^3$
30	$\alpha^3 = \alpha^1 + \frac{(\alpha^4 - \alpha^1)(\phi+1)}{2\phi+1}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}^{new}, \text{ compute } m^3 = m(\underline{u}^x)$
31	end
32	end
33	$[m^{new}, b] = \min([m^2, m^3]), \underline{\alpha}^x = [\alpha^2, \alpha^3], \underline{u} = \underline{u} + \underline{\alpha}^x(b) \underline{h}^{new}, cnt = cnt + 1$
33	end
	<i>%COMPLETION</i>
34	$m^* = m^{old}, \underline{u}^* = \underline{u} - \underline{\alpha}^x(b) \underline{h}^{new}$

To explain this, we first set $\alpha^2 - \alpha^1 = a_1$ and $\alpha^3 - \alpha^2 = b_1$ for the initial search domain (see Fig. 4.6). Because the first situation must decrease the search domain in the same way as the second situation, distance $\alpha^4 - \alpha^3$ must equal distance $\alpha^2 - \alpha^1$, entailing $\alpha^4 - \alpha^3 = a_1$ (see Fig. 4.6). After the first reduction of the search domain (irrespective if this leads to the first or the second situation), a_2 and b_2 are used to denote the associated distances in the shrunken search domain (see Fig. 4.6).

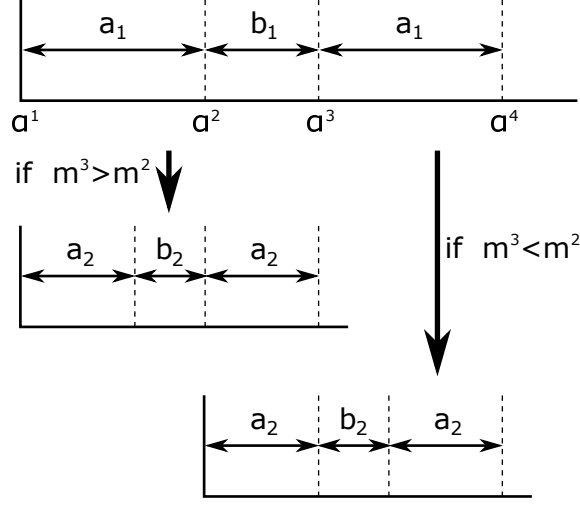


Figure 4.6: Golden section search: after reduction of the search domain (top image), whether resulting in the first situation (central image) or in the second situation (bottom image), the ratio between distances a_1 and b_1 is the same as the ratio between distances a_2 and b_2 .

Based on Fig. 4.6, we can write the following two equations:

$$a_2 = b_1 \quad \text{and} \quad a_1 = a_2 + b_2, \quad (4.10)$$

and crucial is that the ratio a and b in the old search domain must be the same as as in the new (shrunk) search domain:

$$\frac{a_1}{b_1} = \frac{a_2}{b_2}. \quad (4.11)$$

If we now incorporate the two equations of Eq. (4.10) in Eq. (4.11), we obtain:

$$\frac{a_2 + b_2}{a_2} = \frac{a_2}{b_2}, \quad (4.12)$$

which, with the help of $\phi = \frac{a_2}{b_2} \left(= \frac{a_1}{b_1} \right)$, can be rewritten as follows:

$$1 + \frac{1}{\phi} = \phi. \quad (4.13)$$

Multiplying both sides of the equation by ϕ , and a little bit of rearrangement, yields:

$$\phi^2 - \phi - 1 = 0, \quad (4.14)$$

resulting in:

$$\phi = \frac{1 + \sqrt{5}}{2}, \quad (4.15)$$

which is the so-called ‘golden ratio’ and therefore explains the name ‘golden section search’. If we consider that the initial search domain is of length α_{init} (as $\alpha^1 = 0$ and $\alpha^4 = \alpha_{init}$), we can write:

$$\alpha_{init} = 2a_1 + b_1, \quad (4.16)$$

yielding with help of $\phi = \frac{a_1}{b_1}$:

$$a_1 = \frac{\alpha_{init}\phi}{2\phi + 1} \quad \text{and} \quad b_1 = \frac{\alpha_{init}}{2\phi + 1}, \quad (4.17)$$

which gives us sufficient information to determine the location of α^2 and α^3 in the initial search domain and in all possible shrunken search domains (see the pseudo-algorithm of Table 4.4).

4.5 Final comments

Conjugate gradient approaches start with an initial guess, whereas GAs start with an initial search domain. This means that GAs have the ability to find the lowest of several minima (not guaranteed as we have seen in the previous chapter), whereas the minimum found by conjugate gradient approaches relies heavily on the initial guess.

A solution may therefore be to combine GAs with conjugate gradient approaches; first a few generation of a GA are considered and then, a conjugate gradient approach is started from the minimum found by the GA. Alternatively, a conjugate gradient approach can be applied numerous of times; each time with a different starting point.

4.6 Questions

Before answering any of these questions, first read the programming tips at the end of Chapter 1.

Question 4.1 Implement the conjugate gradient framework with the backtracking line search (including only the Armijo rule) and apply it to the objective functions mentioned below. Use the following numerical parameters: $\underline{u}^0 = \underline{0}$, $r = c = 0.5$, $n_{reset} = 25$. Be smart and first implement the steepest descent algorithm with a fixed stepsize, then include the backtracking line search, and finally, change the steepest descent direction to a conjugate gradient direction.

Model 1: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{1}$.

Model 2a: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_2 = g_3 = 1$.

Model 2b: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_2 = g_3 = 1$.

Model 3a: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_2 = g_{19} = 1$.

Model 3b: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_2 = g_{19} = 1$.

Model 4a: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_{62} = g_{79} = 1$.

Model 4b: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_{62} = g_{79} = 1$.

Model 4b: with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_{79} = 1$.

Question 4.2 Use the conjugate gradient framework that you have implemented for Question 4.1 to minimise the following objective function with two variables:

$$m(\underline{u}) = -e^{-50\sqrt{(u_1-0.25)^2+(u_2-0.25)^2}} - 1.2e^{-50\sqrt{(u_1-0.75)^2+(u_2-0.75)^2}} + \sqrt{(u_1-0.35)^2+(u_2-0.75)^2} - 0.5. \quad (4.18)$$

The objective function has three minima ($m^* \approx -0.5$, $\underline{u}^* = [0.35 \ 0.75]^T$ and $m^* \approx -0.99$, $\underline{u}^* = [0.25 \ 0.25]^T$), of which the one given by $m^* = -1.3$, $\underline{u}^* = [0.75 \ 0.75]^T$ is the global minima (see Fig. 3.7). Use the same numerical parameters as in Question 4.1, but set the initial guess as a random realisation in domain $(0, 1) \times (0, 1)$. Observe that the conjugate gradient method hardly ever finds the global minimum and that it sometimes does not find a minimum at all.

Question 4.3 Change the backtracking line search of your conjugate gradient implementation to the golden section search and use it to minimise the objective function of Model 4a with $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ except $g_{62} = g_{79} = 1$.

Newton's method

Previously, we have seen GAs as minimisation frameworks that only require the evaluation of the objective function and gradient descent frameworks that require the evaluation of the objective function as well as the evaluation of the gradient. Gradient descent frameworks are substantially faster than GAs because they ‘know’ in which direction to update the estimated minimiser thanks to the gradient. For this reason, in the exercises we have minimised the objective functions of Chapter 2 depending on 80 variables with gradient descent approaches, but not with GAs - it would just last too long.

In this chapter, an approach is discussed that is based on both the evaluation of the gradient and the evaluation of the Hessian (but not the evaluation of the objective function). It is even faster than gradient descent approaches, because it incorporates more knowledge about the landscape of the objective function; the gradient tells us the slope of the objective function and the Hessian tells us how the slope of the objective function changes. For small minimisation problems as in these notes (where the maximum number of variables is 80), the increase of speed is hardly noticeable.

The advantage of the method discussed in this chapter is its speed, but it also has disadvantages and limitations. An important limitation is that the Hessian (i.e. a matrix of size $n_{var} \times n_{var}$, see Chapter 1) must be invertible. If this is not the case, the method cannot be applied. The condition that the Hessian must be invertible means in practise often that the initial guess must be sufficiently close to the true minimiser.

The method discussed in this chapter is called Newton’s method. The method is considered from two points of view: 1) Newton’s method as a root finding technique and 2) Newton’s method as a minimisation technique. *However, both considerations are exactly the same* and the reason that both considerations are discussed is that the first consideration is often easier to understand, whereas the second consideration is useful for later on.

5.1 Newton’s method as a root finding technique

When discussing Newton’s method as a root finding approach, we start with the *interior extremum theorem* (also known as Fermat’s theorem on stationary points). This theorem states that the stationary points of an objective function are defined as points where the gradient equals zero:

$$\underline{f}^{**} = \left. \frac{\partial m}{\partial \underline{u}} \right|_{\underline{u}^{**}} = \underline{0}, \quad (5.1)$$

where $\underline{u}^{**}$ denotes a stationary point.

Minimizers are stationary points, but maximizers and saddle points are also stationary points (see Fig. 5.1 for an objective function depending on one variable). Depending on the initial guess and on the character of the objective function, one may therefore find any of these three types of points. This has advantages and disadvantages. The disadvantage is that if nothing is known about the objective function, one needs to analyse which of the three types of points is found after a stationary point is found. An advantage for constrained minimisation is that the method also converges to saddle points. This will become clear in a later chapter.

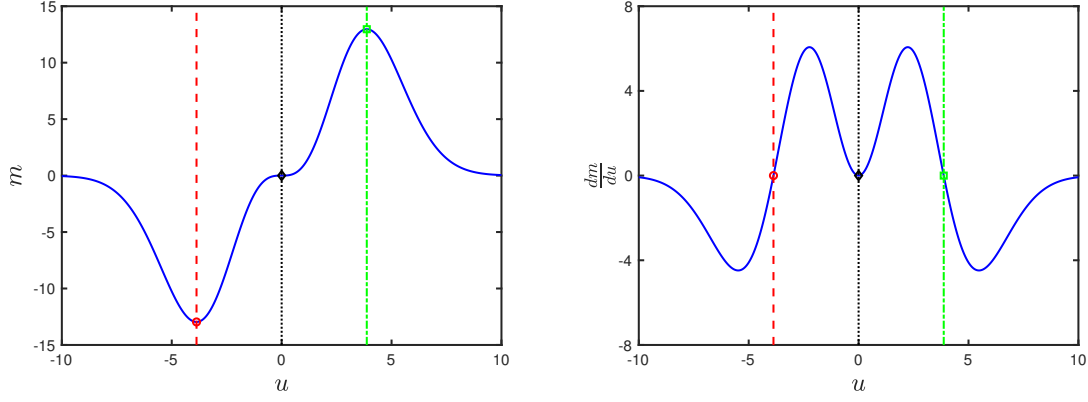


Figure 5.1: Illustration of stationary points for objective function $m(u) = u^3 e^{-0.1u^2}$. Left: the objective function (blue, solid) with a minimum (red, dashed), a saddle point (black, dotted) and a maximum (green, dash-dotted). Right: the objective function's derivative (blue, solid) and the same stationary points (i.e. for which the derivative equals zero).

Similar to gradient approaches, Newton's method starts with an initial guess (defined by the user) for which the gradient is calculated. As the initial guess is generally not a stationary point, the gradient evaluated at the initial guess is not zero, i.e. Eq. (5.1) does not hold. In that case, a first-order Taylor expansion of the interior extremum theorem is made around the initial guess:

$$\tilde{f}|_{\underline{u}^i + \underline{h}} \approx \underline{f}^i + \underline{\underline{K}}^i \underline{h} = \underline{0}, \quad (5.2)$$

with:

$$\underline{f}^i = \left. \frac{\partial m}{\partial \underline{u}} \right|_{\underline{u}^i}, \quad (5.3)$$

$$\underline{\underline{K}}^i = \left. \frac{\partial^2 m}{\partial (\underline{u})^2} \right|_{\underline{u}^i}, \quad (5.4)$$

where $\underline{f}^i$ and $\underline{\underline{K}}^i$ denote the gradient and the Hessian, respectively, evaluated at $\underline{u}^i$. Here, $\underline{u}^i$ denotes the estimated stationary point at the i^{th} iteration ($i = 0$ for the initial guess).

Matrix $\underline{\underline{K}}$ of size $n_{var} \times n_{var}$ is called the Hessian in optimisation. More details about the expression of the Hessian were discussed at the end of Chapter 1; for instance that the Hessian is always symmetric. For objective functions depending on one variable ($n_{var} = 1$), the Hessian is just a scalar that is nothing more than the derivative of $\underline{f}^i$. This is graphically illustrated in Fig. 5.2 for several points by the dashed red lines.

$\underline{h}$ in Eq. (5.2) denotes the correction to the current estimate ($\underline{u}^i$). $\underline{h}$ is unknown and needs to

be computed. This is done by rewriting Eq. (5.2) to:

$$\underline{\underline{K}}^i \underline{h} = -\underline{f}^i, \quad (5.5)$$

which is a system of linear equations as we have also seen in Chapter 1 (although in the discussion of Chapter 1, matrix $\underline{\underline{K}}^i$ was replaced by $\underline{A}$, unknown column $\underline{h}$ by $\underline{x}$ and right-hand side $-\underline{f}^i$ by $\underline{b}$). In the programming tips at the end of Chapter 1, it was mentioned that such systems are advised not to be solved by computing the inverse of the matrix, but to use some linear solver (such as the back slash in Matlab). In this chapter, such linear solvers are not discussed and in the exercises, you are advised to employ a direct linear solver that is available in the programming language that you use.

After $\underline{h}$ is computed by solving the system of linear equations, the estimated stationary point is updated according to:

$$\underline{u}^{i+1} = \underline{u}^i + \underline{h}, \quad (5.6)$$

where $\underline{u}^{i+1}$ denotes the updated stationary point, of the next iteration ($i + 1$). Subsequently, the gradient at the new estimate is computed and if the gradient is not sufficiently close to zero, the process is repeated. If the gradient is sufficiently close to zero, the process is stopped.

A pseudo-algorithm of Newton's method is presented in Table 5.1. It is clearly straightforward, but be not mistaken; the Hessian is not necessarily straightforward to implement as you will see in the exercises. On top of that, the way in which the Hessian is implemented (e.g. full format versus sparse format) has a substantial influence on the computational time of the algorithm - although again, this is more noticeable if the number of variables of the objective function is large.

Table 5.1: A pseudo-algorithm for Newton's method.

<i>%INITIALISATION</i>	
1	Set $\underline{a}, \underline{g}$ <i>% parameters of the objective function</i>
2	Set $\underline{u}, tol$ <i>% initial guess and tolerance</i>
<i>%NEWTON'S METHOD</i>	
3	Compute $\underline{f} = \frac{\partial m}{\partial \underline{u}} \big _{\underline{u}}$
4	while $\sqrt{\underline{f}^T \underline{f}} > tol$
5	Compute $\underline{\underline{K}} = \frac{\partial^2 m}{\partial (\underline{u})^2} \big _{\underline{u}}$
6	Solve $\underline{\underline{K}} \underline{h} = -\underline{f}$
7	$\underline{u} = \underline{u} + \underline{h}$
8	Compute $\underline{f} = \frac{\partial m}{\partial \underline{u}} \big _{\underline{u}}$
9	end
<i>%COMPLETION</i>	
10	$\underline{u}^* = \underline{u}$
11	Compute $m^* = m(\underline{u}^*)$

In Fig. 5.2 the behaviour of Newton's method is presented for an objective function depending on one variable. At the initial guess, the derivative (i.e. gradient if $n_{var} > 1$) and second-order derivative (i.e. Hessian if $n_{var} > 1$) are computed and by stating that the derivative must

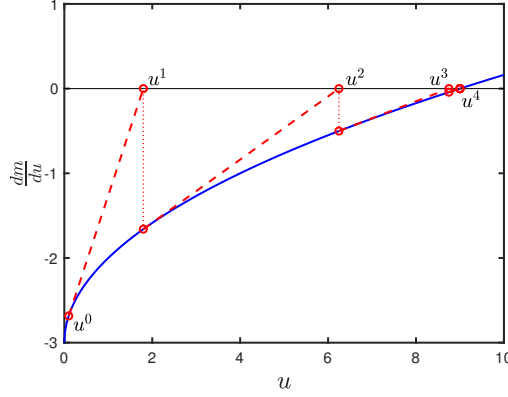


Figure 5.2: Illustration of the updates of Newton’s method in one dimension. The objective function reads $m(u) = \frac{2}{3}u^{1.5} - 3u$ and hence, its derivative (blue line) reads $\frac{dm}{du} = \sqrt{u} - 3$. The Hessian ($\frac{d^2m}{du^2} = \frac{1}{2\sqrt{u}}$, simply a scalar for an objective function depending on one variable as is the case here) is presented by the dashed red lines for each update of the minimizer.

equal zero, a correction (i.e. $\underline{h}$) is computed. At the updated stationary point the derivative is again evaluated and if it is not sufficiently close to zero, a new iteration is made for which the second-order derivative needs to be computed as well. This process is repeated until the derivative/gradient is sufficiently close to zero, i.e. until the L^2 -norm of the gradient (recognisable as $\sqrt{\underline{f}^T \underline{f}}$ in Table 5.1) falls below a user-defined threshold (recognisable as tol in Table 5.1).

5.2 Newton’s method as a minimisation technique

The basis of the consideration above is to first express the interior extremum theorem for the objective function and then to formulate a first-order Taylor for the interior extremum theorem. In the approach discussed in this section however, we turn the sequence around (as mentioned before, the result is exactly the same): we first formulate a (second-order) Taylor expansion for the objective function and subsequently, we express the interior extremum theorem for the Taylor expansion.

The second-order Taylor expansion of the objective function around $\underline{u}^i$ can be expressed as follows:

$$\tilde{m}(\underline{u}^i + \underline{h}) \approx m(\underline{u}^i) + \underline{h}^T \underline{f}^i + \frac{1}{2} \underline{h}^T \underline{K}^i \underline{h}. \quad (5.7)$$

Two of such second-order Taylor expansions are presented in Fig. 5.3 for an objective function that depends on one variable.

Then we write the interior extremum theorem for the Taylor expression, where it is important to recognise that the variables of the Taylor expansion are $\underline{h}$ (and $\underline{u}^i$ is constant in the Taylor

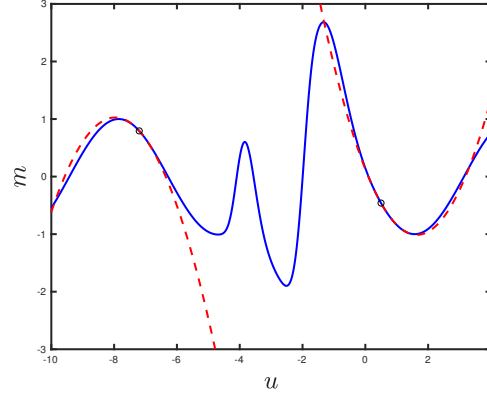


Figure 5.3: Two second-order Taylor expansions (red dashed) around the black circles. The objective function (solid blue) reads: $m(u) = -2e^{-(u+2)^2} - \cos(u)e^{-(u+3)^4} + \cos(u)$.

expansion):

$$\frac{\partial \tilde{m}}{\partial \underline{h}} = \frac{\partial}{\partial \underline{h}} \left(m(\underline{u}^i) + \underline{h}^T \underline{f}^i + \frac{1}{2} \underline{h}^T \underline{\underline{K}}^i \underline{h} \right) = \underline{0}, \quad (5.8)$$

which yields the same final expression as in the previous subsection:

$$\underline{f}^i + \underline{\underline{K}}^i \underline{h} = \underline{0}. \quad (5.9)$$

5.3 Practicalities

In Fig. 5.4, it can be seen that (even though the first few Newton iterations make the objective function increase!), Newton's method requires much less iterations than the conjugate gradient method. However, it is also important to note (noticeable for objective functions with large numbers of variables) that a Newton iteration requires more time than a conjugate gradient iteration, because the Hessian must be constructed and a system of linear equations must be solved in each Newton iteration. Solving a system of linear equations can take a substantial amount of time for large number of variables, because as discussed in Chapter xx, the number of operations involved in solving a system of linear equations scales $\sim n_{var}^3$. Nevertheless, Newton's method is generally significantly faster than the conjugate gradient method for large number of variables.

The L^2 -norm of the gradient ($\sqrt{\underline{f}^T \underline{f}}$) converges in a quadratic way to zero when the minimum is approached. Practically, this can be observed by the number of zeros behind the dot, which is multiplied by a factor of at least two for each iteration. This is presented in Table 5.2 for the Rosenbrock function in Fig. 5.4. It can be observed that the quadratic convergence only appears when the estimate of the minimiser is rather close to the true minimiser. It can also be observed that the convergence may even be better than quadratic. The take home message is that if any

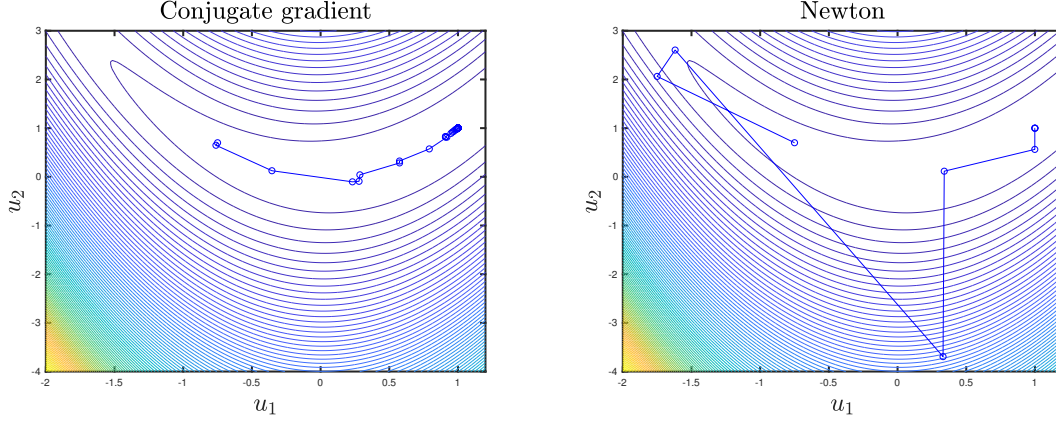


Figure 5.4: Right: Newton’s method for the Rosenbrock function at initial guess $\underline{u} = [-0.75 \ 0.7]^T$. Left: The conjugate gradient method for the Rosenbrock function at the same initial guess.

convergence rate less than the quadratic one is observed (e.g. linear convergence; when only one additional zero appears behind the dot for a new iteration), there is probably a mistake in the implementation of the Hessian.

Table 5.2: Quadratic convergence of the L^2 -norm of the gradient of Newton’s method (applied to the Rosenbrock function at initial guess $\underline{u} = [-0.75 \ 0.7]^T$).

iteration	$\sqrt{\underline{f}^T \underline{f}}$
1	78.1041
2	6.3579
3	90.5023
4	1.3193
5	19.4154
6	0.0018
7	0.00004
8	0.0000000000000004

As mentioned before, Newton’s method not only finds minima, but also saddle-points and maxima. If it is not clear what type of stationary point is found, the Hessian can be evaluated at the found stationary point:

$$\underline{\underline{K}}^{**} = \frac{\partial^2 m}{\partial (\underline{u})^2} \bigg|_{\underline{u}^{**}}, \quad (5.10)$$

and an eigenvalue analysis can then be applied to the Hessian. Subsequently, one may state that:

- a minimum is found if all eigenvalues of $\underline{\underline{K}}^{**}$ are positive,

- a maximum is found if all eigenvalues of $\underline{\underline{K}}^{**}$ are negative,
- a saddle-point is found in all other cases.

The solution of the system of linear equations of Eq. (5.5), i.e. $\underline{h}$, can be interpreted as a search direction. Considering $\underline{h}$ as a search direction, the update of the estimate of the minimisers can also be expressed as:

$$\underline{u}^{i+1} = \underline{u}^i + \alpha \underline{h}, \quad (5.11)$$

where, similarly as in the previous chapter, α denotes the stepsize. Writing the update in this way, it is clear that a line search method can be incorporated in order to determine a good (or even optimal) stepsize for each iteration (as was done in the previous chapter). However, this is not discussed in these notes.

Note furthermore that if $0 < \alpha < 1$, the method is often referred to as the *damped Newton's method* or *relaxed Newton's method*. Note furthermore that the search direction resulting from Newton's method is not necessarily a descent direction, as we have for instance seen for the first two iterations for the Rosenbrock function (see right diagram in Fig. 5.4). In that case, only $\alpha < 0$ would actually make the objective direction reduce.

5.4 Why is the Hessian symmetric?

In most cases that one encounters in practise, the Hessian is symmetric. We will demonstrate for a 2D objective function, $m(u_1, u_2)$, that:

$$\frac{\partial}{\partial u_2} \frac{\partial m}{\partial u_1} = \frac{\partial}{\partial u_1} \frac{\partial m}{\partial u_2}, \quad (5.12)$$

and hence, we demonstrate that the associated Hessian is symmetric. To do so, we will perturb u_1 with δ_1 and u_2 with δ_2 . Fig. 5.5 graphically illustrates these perturbations and the associated objective function evaluations.

With this notation the first-order derivatives, evaluated at $[u_1, u_2]^T$ can be written as:

$$\left. \frac{\partial m}{\partial u_1} \right|_{\substack{u_1 \\ u_2}} = \lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2) - m(u_1, u_2)}{\delta_1}, \quad (5.13)$$

$$\left. \frac{\partial m}{\partial u_2} \right|_{\substack{u_1 \\ u_2}} = \lim_{\delta_2 \rightarrow 0} \frac{m(u_1, u_2 + \delta_2) - m(u_1, u_2)}{\delta_2}. \quad (5.14)$$

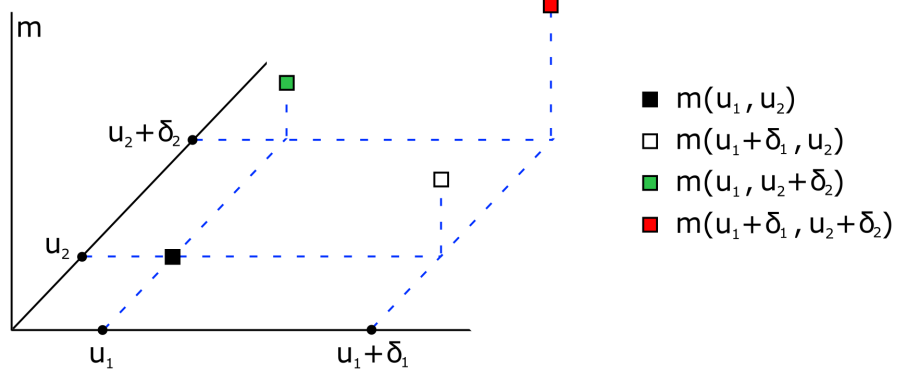


Figure 5.5: Objective function m with perturbation δ_1 of u_1 and perturbation δ_2 of u_2 .

Alternatively, the first one may be evaluated at $[u_1, u_2 + \delta_2]^T$ and the second one at $[u_1 + \delta_1, u_2]^T$. This yields:

$$\left. \frac{\partial m}{\partial u_1} \right|_{\substack{u_1 \\ u_2 + \delta_2}} = \lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1, u_2 + \delta_2)}{\delta_1}, \quad (5.15)$$

$$\left. \frac{\partial m}{\partial u_2} \right|_{\substack{u_1 + \delta_1 \\ u_2}} = \lim_{\delta_2 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2)}{\delta_2}. \quad (5.16)$$

The first second-order derivatives can then be written as:

$$\frac{\partial}{\partial u_2} \frac{\partial m}{\partial u_1} = \lim_{\delta_2 \rightarrow 0} \frac{\left. \frac{\partial m}{\partial u_1} \right|_{u_2 + \delta_2} - \left. \frac{\partial m}{\partial u_1} \right|_{u_2}}{\delta_2} \quad (5.17)$$

$$= \lim_{\delta_2 \rightarrow 0} \frac{\lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1, u_2 + \delta_2)}{\delta_1} - \lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2) - m(u_1, u_2)}{\delta_1}}{\delta_2}, \quad (5.18)$$

which can be written as follows, if m is continuous in u_1 :

$$\frac{\partial}{\partial u_2} \frac{\partial m}{\partial u_1} = \lim_{\delta_2 \rightarrow 0} \frac{\lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1, u_2 + \delta_2)}{\delta_1} - \frac{m(u_1 + \delta_1, u_2) - m(u_1, u_2)}{\delta_1}}{\delta_2}, \quad (5.19)$$

and also as follows:

$$\frac{\partial}{\partial u_2} \frac{\partial m}{\partial u_1} = \lim_{\delta_2 \rightarrow 0} \frac{\lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2) + m(u_1, u_2)}{\delta_1}}{\delta_2}. \quad (5.20)$$

Because of the following equality for the limit:

$$\frac{1}{b} \lim_{x \rightarrow a} g(x) = \lim_{x \rightarrow a} \frac{g(x)}{b}, \quad (5.21)$$

where g denotes a function of variable x and a and b denote scalars, we can now also write:

$$\frac{\partial}{\partial u_2} \frac{\partial m}{\partial u_1} = \lim_{\delta_2 \rightarrow 0} \lim_{\delta_1 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2) + m(u_1, u_2)}{\delta_1 \delta_2}. \quad (5.22)$$

Next, we continue with the second second-order derivative, that we can express as follows:

$$\frac{\partial}{\partial u_1} \frac{\partial m}{\partial u_2} = \lim_{\delta_1 \rightarrow 0} \frac{\left. \frac{\partial m}{\partial u_2} \right|_{u_1 + \delta_1} - \left. \frac{\partial m}{\partial u_2} \right|_{u_1}}{\delta_1} \quad (5.23)$$

$$= \lim_{\delta_1 \rightarrow 0} \frac{\lim_{\delta_2 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2)}{\delta_2} - \lim_{\delta_2 \rightarrow 0} \frac{m(u_1, u_2 + \delta_2) - m(u_1, u_2)}{\delta_2}}{\delta_1}. \quad (5.24)$$

If m is continuous in u_2 , this expression can also be written as:

$$\frac{\partial}{\partial u_1} \frac{\partial m}{\partial u_2} = \lim_{\delta_1 \rightarrow 0} \frac{\lim_{\delta_2 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2)}{\delta_2} - \frac{m(u_1, u_2 + \delta_2) - m(u_1, u_2)}{\delta_2}}{\delta_1}, \quad (5.25)$$

which can be simplified to:

$$\frac{\partial}{\partial u_1} \frac{\partial m}{\partial u_2} = \lim_{\delta_1 \rightarrow 0} \frac{\lim_{\delta_2 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2) - m(u_1, u_2 + \delta_2) + m(u_1, u_2)}{\delta_2}}{\delta_1}. \quad (5.26)$$

If we again make use of Eq. (5.21), we can now write:

$$\frac{\partial}{\partial u_1} \frac{\partial m}{\partial u_2} = \lim_{\delta_1 \rightarrow 0} \lim_{\delta_2 \rightarrow 0} \frac{m(u_1 + \delta_1, u_2 + \delta_2) - m(u_1 + \delta_1, u_2) - m(u_1, u_2 + \delta_2) + m(u_1, u_2)}{\delta_1 \delta_2}. \quad (5.27)$$

Now if objective function m is ‘sufficiently continuous’, the sequence of the limits in Eqs. (5.22) and (5.27) can be reversed and hence, Eqs. (5.22) and (5.27) are exactly the same. This demonstrates that the Hessian is symmetric in case objective function m is ‘sufficiently continuous’.

5.5 The quadratic convergence explained in detail

We will now explain the quadratic convergence of the L^2 -norm of the gradient (*near the minimizer*) in detail.

One variable case

First, we will consider it for a function depending on a single variable, because it is simpler. In one dimension, the L^2 -norm of the gradient reduces to the absolute value of the only first-order derivative of the objective function, $m(u)$. We will consider an iteration i and the subsequent iteration; iteration $i + 1$. We aim to show that:

$$|f^{i+1}| \leq c|f^i|^2, \quad (5.28)$$

where f denotes the first-order derivative (i.e. gradient), c denotes some constant (over which we do not worry too much about) and the important issue is that the absolute value of f^{i+1} depends quadratically on that of f^i .

We will consider that for iteration i we have computed update h by solving the following linear equation:

$$f^i + (f')^i h = 0, \quad (5.29)$$

where f' denotes the derivative of f with respect to u . We now express this equation as follows to make it better readable:

$$f(u^i) + f'(u^i)h = 0. \quad (5.30)$$

The estimate of minimizer at iteration $i + 1$, thus reads:

$$u^{i+1} = u^i + h, \quad (5.31)$$

With Eq. (5.30), we can expand the absolute value of the first-order derivative at the next iteration (iteration $i + 1$) as follows:

$$|f(u^{i+1})| = |f(u^{i+1}) - f(u^i) - f'(u^i)h|, \quad (5.32)$$

$$= |f(u^i + h) - f(u^i) - f'(u^i)h|. \quad (5.33)$$

In the above equation, we now insert the following relation:

$$\int_0^1 f'(u^i + \tau h)h \, d\tau = [f(u^i + \tau h)]_0^1 = f(u^i + h) - f(u^i), \quad (5.34)$$

which yields:

$$|f(u^{i+1})| = \left| \int_0^1 f'(u^i + \tau h) h \, d\tau - f'(u^i) h \right|, \quad (5.35)$$

$$= \left| \int_0^1 f'(u^i + \tau h) h - f'(u^i) h \, d\tau \right|. \quad (5.36)$$

$$= \left| \int_0^1 (f'(u^i + \tau h) - f'(u^i)) h \, d\tau \right|. \quad (5.37)$$

Now, we make use of the following inequality:

$$\left| \int y(x) dx \right| \leq \int |y(x)| \, dx, \quad (5.38)$$

which yields for Eq. (5.37):

$$|f(u^{i+1})| \leq \int_0^1 |(f'(u^i + \tau h) - f'(u^i)) h| \, d\tau. \quad (5.39)$$

Next, we make use of the following equality for the absolute values of two scalars, a and b :

$$|ab| = |a| |b|, \quad (5.40)$$

which enables us to write Eq. (5.39) as follows:

$$|f(u^{i+1})| \leq \int_0^1 |f'(u^i + \tau h) - f'(u^i)| |h| \, d\tau, \quad (5.41)$$

$$\leq |h| \int_0^1 |f'(u^i + \tau h) - f'(u^i)| \, d\tau, \quad (5.42)$$

We now assume that the second-order derivative, i.e. f'' , is Lipschitz continuous (or at least locally, near the minimizer). Lipschitz continuity of a function $y(x)$ holds if:

$$\frac{|y(x^1) - y(x^2)|}{|x^1 - x^2|} \leq \tilde{c}, \quad (5.43)$$

where $0 < \tilde{c} < \infty$ is the Lipschitz constant (and $x^1 \neq x^2$). Applying this inequality to $f'(u)$ in our case, yields:

$$|f'(u^i + \tau h) - f'(u^i)| \leq \tilde{c}|\tau h|, \quad (5.44)$$

which we insert in Eq. (5.42) and obtain:

$$|f(u^{i+1})| \leq |h| \int_0^1 \tilde{c}|\tau h| d\tau, \quad (5.45)$$

$$\leq \tilde{c}|h|^2 \int_0^1 \tau d\tau, \quad (5.46)$$

$$\leq \frac{\tilde{c}}{2}|h|^2. \quad (5.47)$$

By inserting the relation between h and $f(u^i)$ and $f'(u^i)$ according to Eq. (5.30), we obtain the quadratic relation between the two norms of consecutive gradients:

$$|f(u^{i+1})| \leq \frac{\tilde{c}}{2} \frac{|f(u^i)|^2}{|f'(u^i)|^2}, \quad (5.48)$$

$$\leq \frac{\tilde{c}}{2|f'(u^i)|^2} |f(u^i)|^2, \quad (5.49)$$

$$\leq c |f(u^i)|^2. \quad (5.50)$$

Before we continue with the multivariable situation, we return to the integral in Eq. (5.42). Above, we have used the definition of Lipschitz continuity to ultimately obtain the following relation:

$$\int_0^1 |f'(u^i + \tau h) - f'(u^i)| d\tau = \frac{\tilde{c}}{2}|h|. \quad (5.51)$$

Although mathematically not so rigorous, we can obtain this relation also more intuitively. To this purpose, the first-order derivatives, $f'(u)$, for which Newton's method succeeds are presented on the left in Fig. 5.6, and their associated second-order derivatives, $f''(u)$, on the right. The graphs presented are all near the minimizer, for which they are sufficiently smooth (such that the second-order derivatives are Lipschitz continuous).

The triangles in the diagrams on the right approximate the integral above rather well; only some approximation is still involved because $f'(u)$ is not perfectly linear. If the triangles would be

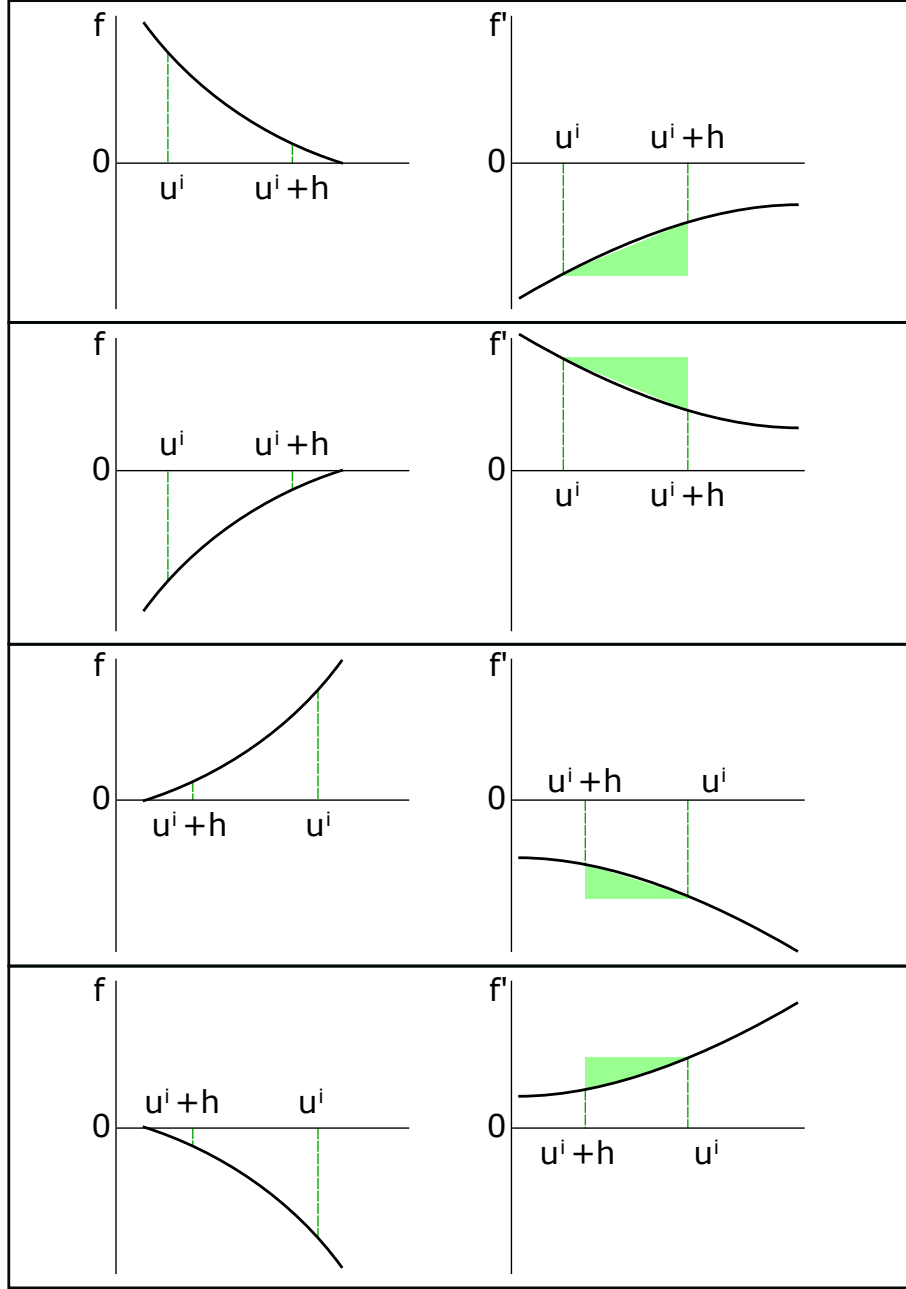


Figure 5.6: Left: First-order derivatives $f(u)$ (near the minimizer) for which Newton's method succeeds. Right: Second-order derivatives $f'(u)$ (near the minimizer) associated with the first-order derivatives on the left.

the perfect solution of the integral above, it would read:

$$\int_0^1 |f'(u^i + \tau h) - f'(u^i)| d\tau = \frac{1}{2} |f'(u^i + h) - f'(u^i)| |h|. \quad (5.52)$$

However, because some approximation is involved, we replace the height (i.e. $|f'(u^i + h) - f'(u^i)|$) of the triangle with $\tilde{c}$. Thus, Eq. (5.51) would again be correct.

Multivariate case

In case objective function m depends on more than one variable, the norm of the gradients of two successive iteration (iterations i and $i + 1$) also converges quadratically. Thus, we will now show that the following inequality holds:

$$\|\underline{f}^{i+1}\| \leq c \|\underline{f}^i\|^2, \quad (5.53)$$

and to make the equations below more easily readable, we will use the following notation (for iteration i as example):

$$\underline{f}^i = \underline{f}|_{\underline{u}^i} = \underline{f}(\underline{u}^i), \quad \underline{\underline{K}}^i = \underline{\underline{K}}|_{\underline{u}^i} = \underline{\underline{f}}'(\underline{u}^i). \quad (5.54)$$

We again start with the relation for update $\underline{h}$ for iteration i :

$$\underline{f}(\underline{u}^i) + \underline{\underline{f}}'(\underline{u}^i)\underline{h} = \underline{0}, \quad (5.55)$$

and expand the L^2 -norm of the gradient for the next iteration ($i + 1$) as follows:

$$\|\underline{f}(\underline{u}^{i+1})\| = \|\underline{f}(\underline{u}^{i+1}) - \underline{f}(\underline{u}^i) - \underline{\underline{f}}'(\underline{u}^i)\underline{h}\|, \quad (5.56)$$

$$= \|\underline{f}(\underline{u} + \underline{h}) - \underline{f}(\underline{u}^i) - \underline{\underline{f}}'(\underline{u}^i)\underline{h}\|. \quad (5.57)$$

$$(5.58)$$

In the above equation, we insert the following relation:

$$\int_0^1 \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h})\underline{h} \, d\tau = [\underline{f}(\underline{u}^i + \tau \underline{h})]_0^1 = \underline{f}(\underline{u}^i + \underline{h}) - \underline{f}(\underline{u}^i), \quad (5.59)$$

resulting in:

$$\|\underline{f}(\underline{u}^{i+1})\| = \left\| \int_0^1 \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h})\underline{h} \, d\tau - \underline{\underline{f}}'(\underline{u}^i)\underline{h} \right\|, \quad (5.60)$$

$$= \left\| \int_0^1 \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h})\underline{h} - \underline{\underline{f}}'(\underline{u}^i)\underline{h} \, d\tau \right\| \quad (5.61)$$

$$= \left\| \int_0^1 (\underline{\underline{f}}'(\underline{u}^i + \tau \underline{h}) - \underline{\underline{f}}'(\underline{u}^i))\underline{h} \, d\tau \right\|, \quad (5.62)$$

We then make use of the following inequality:

$$\left\| \int \underline{y}(\tau) \, \mathrm{d}\tau \right\| \leq \int \|\underline{y}(\tau)\| \, \mathrm{d}\tau \quad (5.63)$$

yielding:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq \int_0^1 \left\| \left(\underline{\underline{f}}'(\underline{u}^i + \tau \underline{h}) - \underline{\underline{f}}'(\underline{u}^i) \right) \underline{h} \right\| \, \mathrm{d}\tau. \quad (5.64)$$

Next, we make use of the following inequality for the norm of a matrix multiplied with a column:

$$\|\underline{\underline{A}} \underline{b}\| \leq \|\underline{\underline{A}}\| \|\underline{b}\|, \quad (5.65)$$

resulting in:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq \int_0^1 \left\| \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h}) - \underline{\underline{f}}'(\underline{u}^i) \right\| \|\underline{h}\| \, \mathrm{d}\tau, \quad (5.66)$$

$$\leq \|\underline{h}\| \int_0^1 \left\| \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h}) - \underline{\underline{f}}'(\underline{u}^i) \right\| \, \mathrm{d}\tau. \quad (5.67)$$

$$(5.68)$$

If we now again assume that the Hessian is Lipschitz continuous, we can use the following inequality:

$$\frac{\left\| \underline{\underline{f}}'(\underline{u}^i + \tau \underline{h}) - \underline{\underline{f}}'(\underline{u}^i) \right\|}{\|\tau \underline{h}\|} \leq \tilde{c}, \quad (5.69)$$

with $0 < \tilde{c} < \infty$, which yields:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq \|\underline{h}\| \int_0^1 \tilde{c} \|\tau \underline{h}\| \, \mathrm{d}\tau, \quad (5.70)$$

$$\leq \tilde{c} \|\underline{h}\|^2 \int_0^1 \tau \, \mathrm{d}\tau, \quad (5.71)$$

$$\leq \frac{\tilde{c}}{2} \|\underline{h}\|^2. \quad (5.72)$$

$$(5.73)$$

If we finally slightly rewrite Eq. (5.55) and insert the resulting expression for $\underline{h}$ on the right hand side in the equation above, we obtain:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq \frac{\tilde{c}}{2} \left\| \left(\underline{\underline{f'}}(\underline{u}^i) \right)^{-1} \underline{f}(\underline{u}^i) \right\|^2. \quad (5.74)$$

Subsequently, we again make use of Eq. (5.65) to write:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq \frac{\tilde{c}}{2} \left\| \left(\underline{\underline{f'}}(\underline{u}^i) \right)^{-1} \right\|^2 \|\underline{f}(\underline{u}^i)\|^2, \quad (5.75)$$

so that we finally obtain:

$$\|\underline{f}(\underline{u}^{i+1})\| \leq c \|\underline{f}(\underline{u}^i)\|^2, \quad (5.76)$$

with

$$c = \frac{\tilde{c}}{2} \left\| \left(\underline{\underline{f'}}(\underline{u}^i) \right)^{-1} \right\|^2, \quad (5.77)$$

showing that indeed the norm of two consecutive gradients behaves quadratically.

5.6 Questions

Before answering any of these questions, first read all the programming tips in Chapter 1.

Question 5.1 Implement Newton's method to determine the minima of (using $\underline{u}^0 = \underline{0}$ and $tol = 10^{-12}$):

Model 1: All a 's are 1, all g 's are 1.

Model 2a: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 2b: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 3a: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 3b: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 4a: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{79} = 1$.

Question 5.2 Determine the L^2 -norm of the gradient at the minimiser for Model 4a that you have found with the conjugate gradient approach and compare this with the L^2 -norm of the gradient at the minimiser for Model 4a that you have found with Newton's method. What can you conclude?

Question 5.3 Use your implementation of Newton's method that you have implemented for Question 5.1 to minimise the following objective function with two variables:

$$m(\underline{u}) = -e^{-50\sqrt{(u_1-0.25)^2+(u_2-0.25)^2}} - 1.2e^{-50\sqrt{(u_1-0.75)^2+(u_2-0.75)^2}} + \sqrt{(u_1-0.35)^2+(u_2-0.75)^2} - 0.5. \quad (5.78)$$

Set the initial guess as a random realisation in domain $(0, 1) \times (0, 1)$ and $tol = 10^{-12}$. Equip your implementation with the ability to distinguish different types of stationary points found.

As you will notice, the Hessian is never invertible, making Newton's method inapplicable in this case. On the other hand, in the previous chapter we have seen that the conjugate gradient method practically always finds a minimum for this objective function (albeit not often the global minimum). In Chapter 2 furthermore, we have seen that GAs also practically always find a minimum for this objective function and more often than the conjugate gradient method, the global minimum (albeit it depends on how many initial parents are seeded).

Quasi-Newton methods

In the previous chapter, we have seen that Newton's method requires only a few iterations compared to conjugate gradient methods. This makes them faster, but they have some disadvantages as we have seen. First, Newton's method fails if the Hessian is singular because then the system of linear equations that needs to be solved in each iteration cannot be solved. Second, the solving of the system of linear equations is relatively time consuming as the number of operations scales with n_{var}^3 . Another disadvantage often mentioned is that the construction of the Hessian may take a considerable amount of time.

Quasi-Newton methods are similar to Newton's method, except that an approximation of the Hessian is made (based on the gradients of previous iterations) and that a line search must be performed to determine a suitable step size. They alleviate the three aforementioned disadvantages of Newton's method by the following ways:

- The approximation of the Hessian is always positive definite, meaning that the system of linear equations (necessary to determine the search direction) can always be solved.
- Instead of approximating the Hessian, quasi-Newton methods actually approximate the Hessian's inverse. Consequently, the system of linear equations does not need to be solved (for which the number of operations scales with n_{var}^3) in order to determine the search direction. Instead, determining the search direction only requires one matrix-column multiplication in quasi-Newton methods (for which the number of operations scales only with n_{var}^2).
- The Hessian matrix is not required to be accurately constructed as in Newton's method. Instead, it is updated based on the two previous estimates of the minimiser and the gradient at both of these estimates.

Of course, quasi-Newton methods also have some disadvantages. They generally require more iterations than Newton's method. Furthermore, their line search does not only require the evaluation of the objective function for possible step sizes (as in the backtracking line search as discussed for conjugate gradient approaches), but also the evaluation of the gradient.

Another disadvantage is that the approximate Hessian (and its inverse) is a full matrix, instead of a sparse matrix. This means that all $n_{var}^2/2 + n_{var}/2$ independent scalars of the approximate Hessian matrix must be stored. For small problems, e.g. $n_{var} < 1000$, this is not a problem, but this becomes a serious problem of storage if n_{var} is large. The problem of storage can be avoided as we will discuss towards the end of this chapter.

6.1 The best known quasi-Newton method

Quasi-Newton approaches are very similar to conjugate gradient methods and Newton's method since each iteration consists of the following two steps:

- First, a search direction is calculated.
- Second, the step size in the search direction is calculated (which we did not consider for Newton's method, but we could have done that).

Hence, again we can write the update of the estimated minimiser as:

$$\underline{u}^{i+1} = \underline{u}^i + \alpha^i \underline{h}^i, \quad (6.1)$$

where $\underline{u}^i$, α^i and $\underline{h}^i$ denote the estimated minimiser, the step size and the search direction of the i^{th} iteration, respectively.

Search direction

In quasi-Newton's methods, the search direction for the i^{th} iteration is computed as follows:

$$\underline{h}^i = -\underline{\tilde{L}}^i \underline{f}^i, \quad (6.2)$$

where $\underline{f}^i$ denotes the gradient at $\underline{u}^i$ and $\underline{\tilde{L}}^i$ denotes the inverse of the approximated Hessian at $\underline{u}^i$. This is the same search direction as in Newton's method (cf. Eq. (5.5)), except that in Newton's method the true Hessian is used (and computing the inverse must be avoided), whereas in quasi-Newton methods the inverse of an approximated Hessian is used. Note that the tilde on top of $\underline{\tilde{L}}$ is used to distinguish its approximation from the real one, i.e. $\underline{\tilde{L}} = \underline{K}^{-1}$ and $\underline{\tilde{L}} = \underline{\tilde{K}}^{-1}$, where $\underline{\tilde{K}}$ denotes the approximated Hessian.

In most quasi-Newton's methods, the procedure to approximate the Hessian and its inverse starts with an initially assumed Hessian (its inverse to be precise). Subsequently, the Hessian's inverse is updated every iteration using the estimated minimizers of the previous two iterations, and the gradient evaluated at the same two estimated minimizers. Probably the best known approach to update the approximated Hessian's inverse was suggested by Broyden, Fletcher, Goldfarb and Shanno (BFGS). The BFGS update reads:

$$\underline{\tilde{L}}^i = \underline{\tilde{L}}^{i-1} + \frac{(\Delta \underline{u}^T \Delta \underline{f} + \Delta \underline{f}^T \underline{\tilde{L}}^{i-1} \Delta \underline{f}) (\Delta \underline{u} \Delta \underline{u}^T)}{(\Delta \underline{u}^T \Delta \underline{f})^2} - \frac{\underline{\tilde{L}}^{i-1} \Delta \underline{f} \Delta \underline{u}^T + \Delta \underline{u} \Delta \underline{f}^T \underline{\tilde{L}}^{i-1}}{\Delta \underline{u}^T \Delta \underline{f}}, \quad (6.3)$$

where $\underline{\tilde{L}}^{i-1}$ denotes the approximated Hessian's inverse of the previous iteration and:

$$\Delta \underline{f} = \underline{f}^i - \underline{f}^{i-1} \quad \text{and} \quad \Delta \underline{u} = \underline{u}^i - \underline{u}^{i-1}. \quad (6.4)$$

The above expressions only state how to update the approximated Hessian's inverse, but of course one has to formulate an initial one, i.e. $\underline{\tilde{L}}^0$. It is straightforward to use the identity matrix as the initial Hessian's inverse (i.e. $\underline{\tilde{L}}^0 = \underline{I}$). This is an acceptable choice, because the identity matrix is positive definite and it makes the first search direction equal to the steepest descent direction (as used in conjugate gradient methods):

$$\underline{h}^0 = -\underline{\tilde{L}}^0 \underline{f}^0 = -\underline{I} \underline{f}^0 = -\underline{f}^0, \quad (6.5)$$

Line search

To guarantee that the approximated Hessian of the next iteration is invertible, i.e. to make sure that its inverse exists, step size α^i must not only abide the Armijo rule (as we have seen for conjugate gradient methods), but it must also satisfy the so-called curvature condition. The curvature condition ensures that $\Delta \underline{u}^T \Delta \underline{f} > 0$ in the expression for the update of the approximated Hessian's inverse (see Eq. (6.3)).

The Armijo rule (also known as the sufficient decrease condition) and the curvature condition are together referred to as the Wolfe conditions. Step size α^i satisfies the Wolfe conditions if:

$$\begin{aligned} m(\underline{u}^i + \alpha^i \underline{h}^i) &\leq m(\underline{u}^i) + c^1 \alpha^i \underline{h}^{iT} \underline{f}^i, & (\text{Armijo}) \\ \underline{h}^{iT} \underline{f}^{i+\alpha^i \underline{h}^i} &\geq c^2 \underline{h}^{iT} \underline{f}^i, & (\text{curvature}) \end{aligned}$$

where

$$\underline{f}^{i+\alpha^i \underline{h}^i} = \left. \frac{\partial m}{\partial \underline{u}} \right|_{\underline{u}^i + \alpha^i \underline{h}^i}. \quad (6.6)$$

Parameters c^1 and c^2 are to be chosen according $0 < c^1 < c^2 < 1$ and are typically set as $c^1 = 10^{-4}$ and $c^2 = 0.9$. It is clear from Eqs. (Armijo) and (curvature) that the Armijo condition requires the objective function to be evaluated at a proposed step size, whereas the curvature condition requires the gradient to be evaluated at the proposed step size.

It also worth noting that the following variant of the Wolfe conditions exists:

$$\begin{aligned} m(\underline{u}^i + \alpha^i \underline{h}^i) &\leq m(\underline{u}^i) + c^1 \alpha^i \underline{h}^{iT} \underline{f}^i, & (\text{Armijo}) \\ |\underline{h}^{iT} \underline{f}^{i+\alpha^i \underline{h}^i}| &\leq c^2 |\underline{h}^{iT} \underline{f}^i|, & (\text{curvature}) \end{aligned}$$

which are called the strong Wolfe conditions. In Fig. 6.1, acceptable domains for possible step sizes are graphically presented for the Armijo rule only, the Wolfe conditions and the strong Wolfe conditions. It is clear that the Wolfe conditions push acceptable step sizes closer to a minimum than the Armijo rule alone. In turn, the strong Wolfe conditions push acceptable step sizes closer to a minimum than the Wolfe conditions. Yet, all three types of line searches are known as *inexact* line searches. The golden section search of Chapter 4 on the other hand is an example of an *exact* line search, because it is substantially more accurate.

Now the Wolfe conditions or strong Wolfe conditions must be enforced to ensure that the inverse of the approximated Hessian of the next iteration exists, the backtracking line search (as used for the Armijo rule only) does not suffice. Instead, one may use the bisection method. The bisection method uses three step sizes in a domain (two on the edges of the domain and one in the center) to shrink the domain until one of the step sizes is proposed in an acceptable domain.

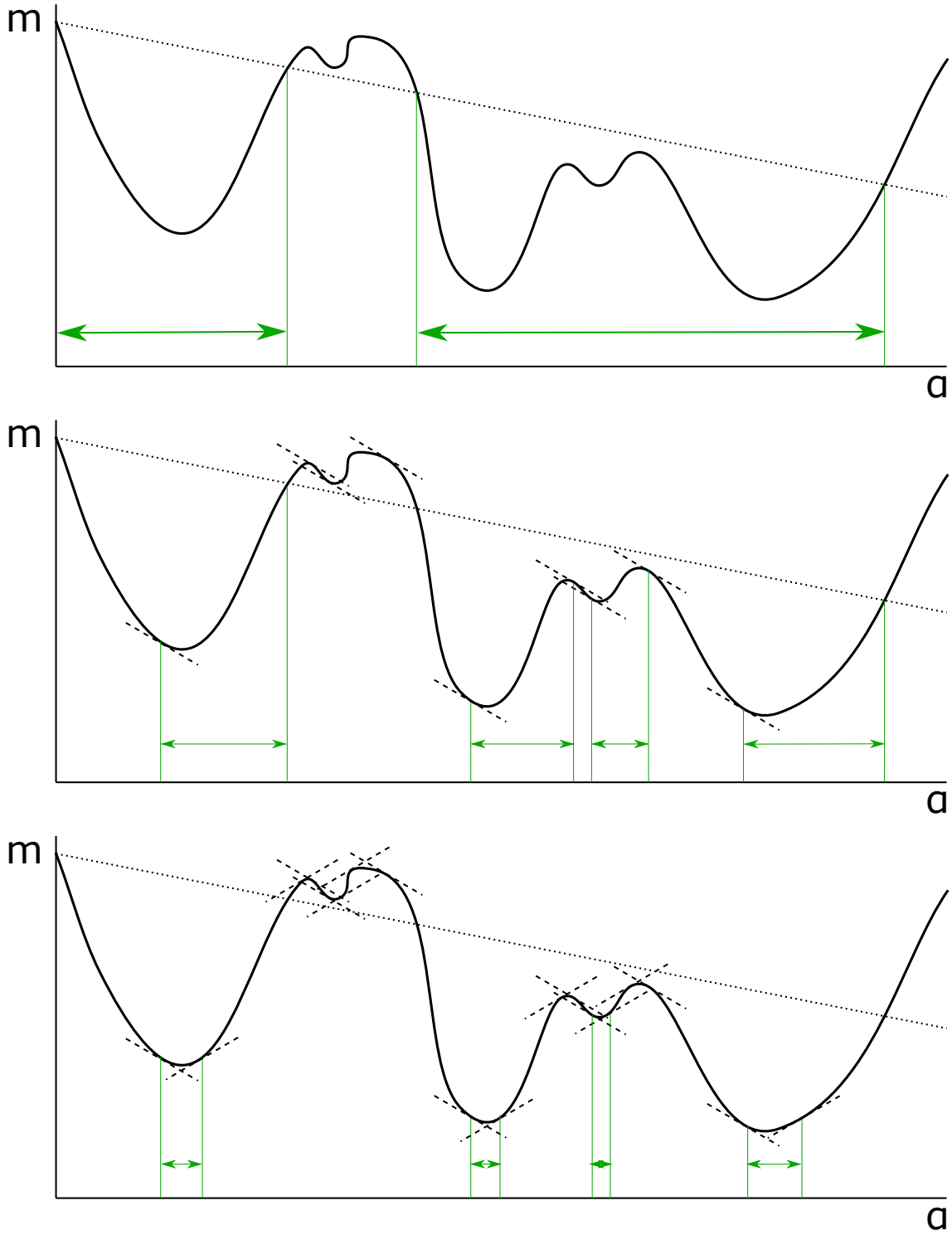


Figure 6.1: Domains (indicated by green arrows) in which a step size is accepted according to the Armijo rule (top), the Wolfe conditions (center) and the strong Wolfe conditions (bottom). The objective function is presented by the bold line, the Armijo rule by the dotted line and $c^2 \underline{h}^{iT} \underline{f}^i$ (center) and $c^2 |\underline{h}^{iT} \underline{f}^i|$ (bottom) by the dashed lines.

Table 6.1: A pseudo-algorithm for the BFGS method with a line search satisfying the Wolfe conditions.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}, n_{var}$ <i>% set parameters of the objective function</i>
2	Set $\alpha_{init}, 0 < c^1 < c^2 < 1, 0 < r < 1, tol, \underline{u}, \underline{\underline{L}}^{new} = \text{eye}(n_{var})$ <i>% set parameters of the algorithm</i>
	<i>%QUASI-NEWTON ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
4	$m^{old} = 10^{100}, \underline{f}^{new} = \underline{0}, cnt = 0$ <i>% dummy values and counter</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}, \underline{f}^{old} = \underline{f}^{new}, \underline{\underline{L}}^{old} = \underline{\underline{L}}^{new}, \underline{f}^{new} = \underline{f}^2, \Delta \underline{f} = \underline{f}^{new} - \underline{f}^{old}$
	<i>%Determine search direction</i>
7	if $cnt = 0$
8	$\underline{h} = -\underline{\underline{L}}^{old} \underline{f}^{new}$
9	else
10	$\underline{\underline{L}}^{new} = \underline{\underline{L}}^{old} + \frac{(\Delta \underline{u}^T \Delta \underline{f} + \Delta \underline{f}^T \underline{\underline{L}}^{old} \Delta \underline{f})(\Delta \underline{u} \Delta \underline{u}^T)}{(\Delta \underline{u}^T \Delta \underline{f})^2} - \frac{\underline{\underline{L}}^{old} \Delta \underline{f} \Delta \underline{u}^T + \Delta \underline{u} \Delta \underline{f}^T \underline{\underline{L}}^{old}}{\Delta \underline{u}^T \Delta \underline{f}}$
11	$\underline{h} = -\underline{\underline{L}}^{new} \underline{f}^{new}$
12	end
	<i>%Line search</i>
	<i>%Determine initial search domain, but stop if acceptable stepsize is found</i>
13	$signal1 = 0, \alpha^3 = \alpha^{init}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}$
14	Compute $m^3 = m(\underline{u}^x)$ and $\underline{f}^3 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
15	if $m^3 \leq m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $\underline{h}^T \underline{f}^3 \geq c^2 \underline{h}^T \underline{f}^{new}$
16	$signal1 = 1$
17	end
18	while $m^3 < m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $signal1 = 0$
19	$\alpha^3 = \frac{\alpha^3}{r}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}$
20	Compute $m^3 = m(\underline{u}^x)$ and $\underline{f}^3 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
21	if $m^3 \leq m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $\underline{h}^T \underline{f}^3 \geq c^2 \underline{h}^T \underline{f}^{new}$
22	$signal1 = 1$
23	end
24	end
	<i>%Apply bisection method if no acceptable stepsize is found yet</i>
25	if $signal1 = 0$
26	$signal2 = 0, \alpha^1 = 0, \alpha^2 = \frac{\alpha^3}{2}$
27	$\underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
28	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
29	while $signal2 = 0$
30	if $\alpha^3 - \alpha^1 < tol$
31	$signal2 = 1, m^2 = m^{new}, \underline{f}^2 = \underline{f}^{new}$
32	elseif $m^2 > m^{new} + c^1 \alpha^2 \underline{h}^T \underline{f}^{new}$
33	$\alpha^3 = \alpha^2, m^3 = m^2, \underline{f}^3 = \underline{f}^2, \alpha^2 = \frac{\alpha^1 + \alpha^2}{2}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
34	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
35	elseif $\underline{h}^T \underline{f}^2 < c^2 \underline{h}^T \underline{f}^{new}$
36	$\alpha^1 = \alpha^2, \alpha^2 = \frac{\alpha^2 + \alpha^3}{2}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
37	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
38	else
39	$signal2 = 1$
40	end
41	end
42	end
	<i>%Complete iteration</i>
43	$\Delta \underline{u} = \underline{u}^x - \underline{u}, \underline{u} = \underline{u}^x, cnt = cnt + 1$
44	if $signal1 = 1$
45	$m^{new} = m^3, \underline{f}^2 = \underline{f}^3$
46	else
47	$m^{new} = m^2$
48	end
49	end
	<i>%COMPLETION</i>
50	$m^* = m^{old}, \underline{u}^* = \underline{u} - \Delta \underline{u}$

Table 6.2: A pseudo-algorithm for the BFGS method for the strong Wolfe conditions.

	<i>%INITIALISATION</i>
1	Set $\underline{a}, g, n_{var}$ <i>% set parameters of the objective function</i>
2	Set $\alpha_{init}, 0 < c^1 < c^2 < 1, 0 < r < 1, tol, \underline{u}, \underline{\tilde{L}}^{new} = \text{eye}(n_{var})$ <i>% set parameters of the algorithm</i>
	<i>%QUASI-NEWTON ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
4	$m^{old} = 10^{100}, \underline{f}^{new} = \underline{0}, cnt = 0$ <i>% dummy values and counter</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}, \underline{f}^{old} = \underline{f}^{new}, \underline{\tilde{L}}^{old} = \underline{\tilde{L}}^{new}, \underline{f}^{new} = \underline{f}^2, \Delta \underline{f} = \underline{f}^{new} - \underline{f}^{old}$
	<i>%Determine search direction</i>
7	if $cnt = 0$
8	$\underline{h} = -\underline{\tilde{L}}^{old} \underline{f}^{new}$
9	else
10	$\underline{\tilde{L}}^{new} = \underline{\tilde{L}}^{old} + \frac{(\Delta \underline{u}^T \Delta \underline{f} + \Delta \underline{f}^T \underline{\tilde{L}}^{old} \Delta \underline{f})(\Delta \underline{u} \Delta \underline{u}^T)}{(\Delta \underline{u}^T \Delta \underline{f})^2} - \frac{\underline{\tilde{L}}^{old} \Delta \underline{f} \Delta \underline{u}^T + \Delta \underline{u} \Delta \underline{f}^T \underline{\tilde{L}}^{old}}{\Delta \underline{u}^T \Delta \underline{f}}$
11	$\underline{h} = -\underline{\tilde{L}}^{new} \underline{f}^{new}$
12	end
	<i>%Line search</i>
	<i>%Determine initial search domain, but stop if acceptable stepsize is found</i>
13	$signal1 = 0, \alpha^3 = \alpha^{init}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}$
14	Compute $m^3 = m(\underline{u}^x)$ and $\underline{f}^3 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
15	if $m^3 \leq m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $ \underline{h}^T \underline{f}^3 \leq c^2 \underline{h}^T \underline{f}^{new} $
16	$signal1 = 1$
17	end
18	while $m^3 < m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $\underline{h}^T \underline{f}^3 < -c^2 \underline{h}^T \underline{f}^{new}$ & $signal1 = 0$
19	$\alpha^3 = \frac{\alpha^3}{r}, \underline{u}^x = \underline{u} + \alpha^3 \underline{h}$
20	Compute $m^3 = m(\underline{u}^x)$ and $\underline{f}^3 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
21	if $m^3 \leq m^{new} + c^1 \alpha^3 \underline{h}^T \underline{f}^{new}$ & $ \underline{h}^T \underline{f}^3 \leq c^2 \underline{h}^T \underline{f}^{new} $
22	$signal1 = 1$
23	end
24	end
	<i>%Apply bisection method if no acceptable stepsize is found yet</i>
25	if $signal1 = 0$
26	$signal2 = 0, \alpha^1 = 0, \alpha^2 = \frac{\alpha^3}{2}$
27	$\underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
28	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
29	while $signal2 = 0$
30	if $\alpha^3 - \alpha^1 < tol$
31	$signal2 = 1, m^2 = m^{new}, \underline{f}^2 = \underline{f}^{new}$
32	elseif $m^2 > m^{new} + c^1 \alpha^2 \underline{h}^T \underline{f}^{new}$
33	$\alpha^3 = \alpha^2, m^3 = m^2, \underline{f}^3 = \underline{f}^2, \alpha^2 = \frac{\alpha^1 + \alpha^2}{2}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
34	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
35	else
36	if $\underline{h}^T \underline{f}^2 < c^2 \underline{h}^T \underline{f}^{new}$
37	$\alpha^1 = \alpha^2, \alpha^2 = \frac{\alpha^2 + \alpha^3}{2}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
38	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
39	elseif $\underline{h}^T \underline{f}^2 > -c^2 \underline{h}^T \underline{f}^{new}$
40	$\alpha^3 = \alpha^2, m^3 = m^2, \underline{f}^3 = \underline{f}^2, \alpha^2 = \frac{\alpha^1 + \alpha^2}{2}, \underline{u}^x = \underline{u} + \alpha^2 \underline{h}$
41	Compute $m^2 = m(\underline{u}^x)$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}^x}$
42	else
43	$signal2 = 1$
44	end
45	end
46	end
47	end
	<i>%Complete iteration</i>
48	$\Delta \underline{u} = \underline{u}^x - \underline{u}, \underline{u} = \underline{u}^x, cnt = cnt + 1$
49	if $signal1 = 1$
50	$m^{new} = m^3, \underline{f}^2 = \underline{f}^3$
51	else
52	$m^{new} = m^2$
53	end
54	end
	<i>%COMPLETION</i>
55	$m^* = m^{old}, \underline{u}^* = \underline{u} - \Delta \underline{u}$

In Tables 6.1 and 6.2, possible pseudo-implementations are presented for the BFGS method, where the line search of Table 6.1 satisfies the Wolfe conditions and that of Table 6.2 the strong Wolfe conditions. Because the bisection method requires at least one acceptable domain to be located inside its initial search domain, lines 13-24 in Tables 6.1 and 6.2 are used to ensure that the initial search domain contains at least one acceptable domain. However, if an acceptable domain is already found during the quest for an initial search domain (i.e. in lines 13-24 in Tables 6.1 and 6.2), the bisection method is skipped.

In Fig. 6.2, updates of the minimizer are shown for the quasi-Newton method applied to the Rosenbrock function, together with those of the conjugate gradient method and Newton's method. The number of updates is clearly smaller than for the conjugate gradient method and larger than for Newton's method. In turn, each BFGS update takes more time than an update in the conjugate gradient method and less time than an update in Newton's method.

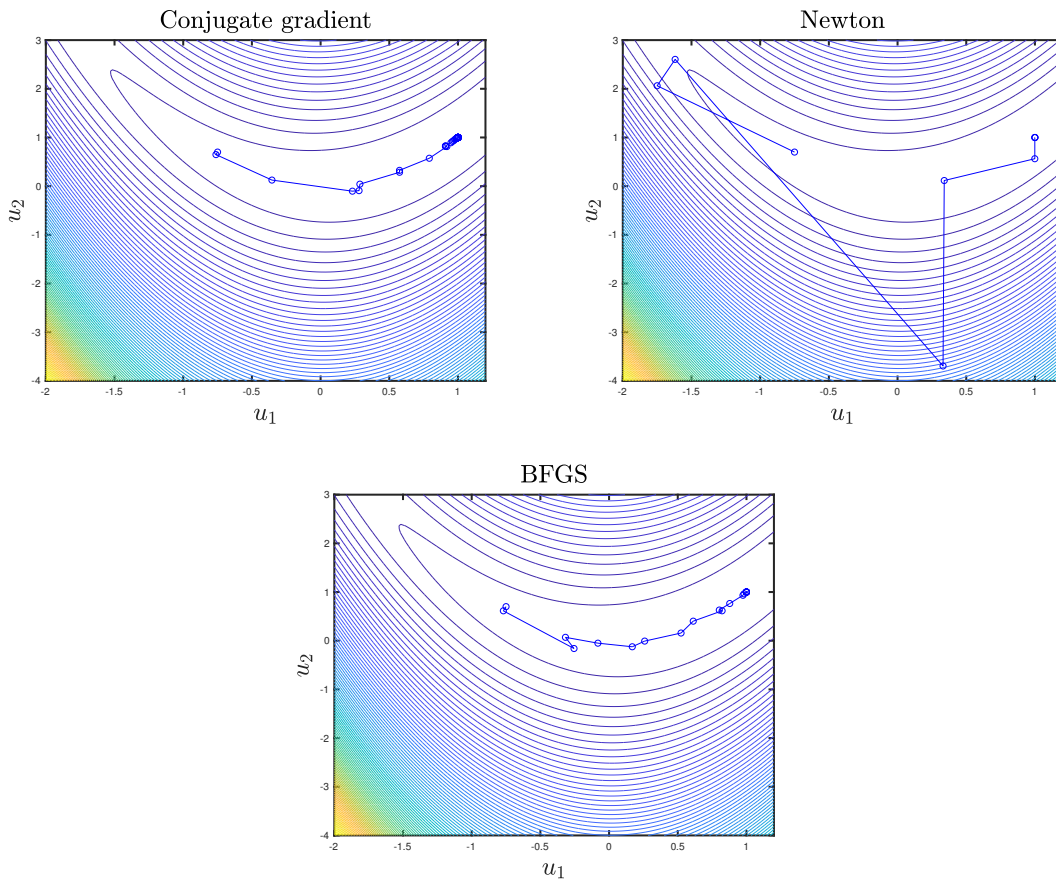


Figure 6.2: The updates of the quasi-Newton method employing the BFGS update for the Rosenbrock function starting at $\underline{u} = [-0.75 \ 0.7]^T$ compared to the conjugate gradient method and Newton's method.

6.2 Background

In the current subsection, the foundation of the BFGS method is discussed in more detail. The discussion focuses on two points: (1) why is the curvature rule required for the BFGS update

(next to the Armijo rule), and (2) what is the origin of the BFGS update of the approximated Hessian's inverse.

Secant equation and curvature condition

In the second half of the previous chapter, we have seen that each update in Newton's method is based on a second-order Taylor expansion of the objective function around the current estimate of the minimizer. The same approximation also forms the basis to determine the search direction in quasi-Newton methods, except that the Hessian ($\underline{\underline{K}}^i$) is replaced by an approximation of the Hessian ($\tilde{\underline{\underline{K}}}^i$):

$$\tilde{m}(\underline{u}^i + \underline{h}) \approx m(\underline{u}^i) + \underline{h}^T \underline{f}^i + \frac{1}{2} \underline{h}^T \tilde{\underline{\underline{K}}}^i \underline{h}. \quad (6.7)$$

Practically, all quasi-Newton methods are based on two conditions for the above approximation of the objective function. The first condition states that the gradient of the approximation at $\underline{h} = \underline{0}$ must be the same as the gradient of the true objective function at $\underline{h} = \underline{0}$. Eq. (6.7) automatically fulfils this condition:

$$\left. \frac{\partial \tilde{m}}{\partial \underline{h}} \right|_{\underline{h}=\underline{0}} = \underline{f}^i. \quad (6.8)$$

The second condition again involves the gradients of the approximation and the true objective function. It states that the gradient of the approximation at the previous estimate of the minimizer must be the same as the gradient of the true objective function at the previous estimate of the minimiser. In other words, the gradient of the approximation at $\underline{h} = -\alpha^{i-1} \underline{h}^{i-1}$ (where α^{i-1} denotes the previous step size and $\underline{h}^{i-1}$ the previous search direction) must be the same as the gradient of the true objective function at $\underline{h} = -\alpha^{i-1} \underline{h}^{i-1}$. This can be written as:

$$\left. \frac{\partial \tilde{m}}{\partial \underline{h}} \right|_{\underline{h}=-\alpha^{i-1} \underline{h}^{i-1}} = \underline{f}^{i-1}. \quad (6.9)$$

Working out the left-hand side yields:

$$\underline{f}^i - \tilde{\underline{\underline{K}}}^i \alpha^{i-1} \underline{h}^{i-1} = \underline{f}^{i-1}, \quad (6.10)$$

which can be rewritten as follows:

$$\tilde{\underline{\underline{K}}}^i \alpha^{i-1} \underline{h}^{i-1} = \underline{f}^i - \underline{f}^{i-1}, \quad (6.11)$$

and by abbreviating $\Delta \underline{u} = \underline{u}^i - \underline{u}^{i-1} = \alpha^{i-1} \underline{h}^{i-1}$ and $\Delta \underline{f} = \underline{f}^i - \underline{f}^{i-1}$, it can be expressed as:

$$\underline{\tilde{K}}^i \Delta \underline{u} = \Delta \underline{f}, \quad (6.12)$$

which is called the *secant equation* and forms the basis of most, if not all quasi-Newton methods. In other words, pretty much all quasi-Newton methods state that the approximation in Eq. (6.7) must abide the conditions in Eqs. (6.8) and (6.9).

In order to ensure that the approximated Hessian is invertible, we can make sure that it is positive definite (every positive definite matrix is invertible, not every invertible matrix is positive definite). Matrix $\underline{\underline{A}}$ is positive definite if:

$$\underline{x}^T \underline{\underline{A}} \underline{x} > 0, \quad (6.13)$$

for all columns $\underline{x} \neq \underline{0}$.

If we now premultiply both sides of Eq. (6.12) with $\Delta \underline{u}$, we obtain:

$$\Delta \underline{u}^T \underline{\tilde{K}}^i \Delta \underline{u} = \Delta \underline{u}^T \Delta \underline{f}. \quad (6.14)$$

If we now want the approximated Hessian to be positive definite (and hence, invertible), we write:

$$0 < \Delta \underline{u}^T \Delta \underline{f}. \quad (6.15)$$

Instead of expanding $\Delta \underline{u}$ and $\Delta \underline{f}$ in terms of the previous update ($\alpha^{i-1} \underline{h}^{i-1}$) and associated gradients ($\underline{f}^{i-1}$ and $\underline{f}^i$), we now expand the above inequality in terms of the *current* update ($\alpha^i \underline{h}^i$) and associated gradients ($\underline{f}^i$ and $\underline{f}^{i+1}$):

$$0 < \alpha^i \underline{h}^{iT} \left(\underline{f}^{i+\alpha^i \underline{h}^i} - \underline{f}^i \right). \quad (6.16)$$

Since $\alpha^i > 0$ and with some rewriting, we can also write:

$$\underline{h}^{iT} \underline{f}^i < \underline{h}^{iT} \underline{f}^{i+\alpha^i \underline{h}^i}. \quad (6.17)$$

In practise, we can relax this statement somewhat and instead write ($0 < c^2 < 1$):

$$c^2 \underline{h}^{iT} \underline{f}^i \leq \underline{h}^{iT} \underline{f}^{i+\alpha^i \underline{h}^i}, \quad (6.18)$$

and we have retrieved the curvature condition (Eq. (curvature)).

Thus, the curvature condition is not only necessary to ensure that the update of the approximated Hessian (and its inverse) is invertible for the BFGS method, it is necessary for *any* quasi-Newton method (that is based on the secant equation).

BFGS update of the approximated Hessian's inverse

The assumption in the BFGS method is that the approximated Hessian's inverse of the current iteration must be as similar as possible to that of the previous iteration. This statement can be expressed as the following minimization problem:

$$\underline{\tilde{L}}^i = \underset{\underline{\tilde{L}}}{\operatorname{argmin}} \quad \|\underline{\tilde{L}} - \underline{\tilde{L}}^{i-1}\|, \quad (6.19)$$

where $\|\bullet\|$ denotes the Frobenius norm, which is defined as follows for real $m \times n$ matrix $\underline{A}$:

$$\|\underline{A}\| = \sqrt{\sum_{i=1}^m \sum_{j=1}^n \left((\underline{A})_{ij} \right)^2}. \quad (6.20)$$

The minimization problem of Eq. (6.19) would simply yield $\underline{\tilde{L}}^i = \underline{\tilde{L}}^{i-1}$. Besides the fact that this result would not yield any update of the approximated Hessian's inverse, it also does not abide the (dual of) secant equation (Eq. (6.12)):

$$\underline{\tilde{L}}^i \Delta \underline{f} = \Delta \underline{u}, \quad (6.21)$$

which is therefore included as a constraint. As the true Hessian and its inverse are generally symmetric, it also makes sense to include the condition that the approximated Hessian's inverse must be symmetric:

$$\underline{\tilde{L}}^i = \left(\underline{\tilde{L}}^i \right)^T. \quad (6.22)$$

As we furthermore obtain most up-to-date information about the approximated Hessian's inverse at $\underline{u}^i + \alpha^i \underline{h}^i$ and at $\underline{u}^i$, the BFGS update uses this as a weight for the objective function of Eq. (6.19). Consequently, the actual minimization problem defining the update of the approximated Hessian's inverse in the BFGS method reads:

$$\underline{\tilde{L}}^i = \underset{\underline{\tilde{L}}}{\operatorname{argmin}} \quad \|\underline{W}^{\frac{1}{2}} (\underline{\tilde{L}} - \underline{\tilde{L}}^{i-1}) \underline{W}^{\frac{1}{2}}\| = \underset{\underline{\tilde{L}}}{\operatorname{argmin}} \quad \|\underline{W}^{\frac{1}{2}} \underline{\tilde{L}} \underline{W}^{\frac{1}{2}} - \underline{W}^{\frac{1}{2}} \underline{\tilde{L}}^{i-1} \underline{W}^{\frac{1}{2}}\|, \quad (6.23)$$

$$\text{such that} \quad \underline{\tilde{L}} = \left(\underline{\tilde{L}}\right)^T, \quad \underline{\tilde{L}} \Delta \underline{f} = \Delta \underline{u}. \quad (6.24)$$

Furthermore, the symmetric weighing matrix is some type of average Hessian, that maps $\Delta \underline{u}$ to $\Delta \underline{f}$, according to:

$$\underline{W} \Delta \underline{u} = \Delta \underline{f}. \quad (6.25)$$

Although complicated, this minimization problem has a solution. To this end, we introduce two new matrices:

$$\underline{\check{L}} = \underline{W}^{\frac{1}{2}} \underline{\tilde{L}} \underline{W}^{\frac{1}{2}}, \quad \underline{\check{L}}^{i-1} = \underline{W}^{\frac{1}{2}} \underline{\tilde{L}}^{i-1} \underline{W}^{\frac{1}{2}}, \quad (6.26)$$

so that we can now rewrite the minimization problem of Eq. (6.23) as follows:

$$\underline{\check{L}}^i = \underset{\underline{\check{L}}}{\operatorname{argmin}} \quad \|\underline{\check{L}} - \underline{\check{L}}^{i-1}\|. \quad (6.27)$$

The constraints in Eq. (6.24) must still be valid and must be rewritten in terms of our new optimization variable: matrix $\underline{\check{L}}$. The first constraint is simple, but the second constraint in Eq. (6.24) requires some algebra. If we assume that we want to rewrite the second constraint in the form of:

$$\underline{\check{L}} \Delta \underline{\check{f}} = \Delta \underline{\check{u}}, \quad (6.28)$$

the question is what are the expressions for $\Delta \underline{\check{f}}$ and $\Delta \underline{\check{u}}$. The answers are the following expressions:

$$\Delta \underline{\check{f}} = \underline{W}^{-\frac{1}{2}} \Delta \underline{f}, \quad \Delta \underline{\check{u}} = \underline{W}^{\frac{1}{2}} \Delta \underline{u}. \quad (6.29)$$

After all, if we insert these expressions together with the expression for $\underline{\check{L}}$ in Eq. (6.28), we obtain:

$$\underline{W}^{\frac{1}{2}} \underline{\tilde{L}} \underline{W}^{\frac{1}{2}} \underline{W}^{-\frac{1}{2}} \Delta \underline{f} = \underline{W}^{\frac{1}{2}} \Delta \underline{u}, \quad (6.30)$$

$$\underline{W}^{\frac{1}{2}} \underline{\tilde{L}} \Delta \underline{f} = \underline{W}^{\frac{1}{2}} \Delta \underline{u}. \quad (6.31)$$

$$\underline{\tilde{L}} \Delta \underline{f} = \Delta \underline{u}, \quad (6.32)$$

and the second constraint in Eq. (6.24) is indeed retrieved. Before continuing with the minimization problem at hand, we must realize that the expressions for $\Delta \underline{\check{f}}$ and $\Delta \underline{\check{u}}$ entail another relation. To this purpose, we take Eq. (6.25), and write:

$$\underline{\underline{W}} \Delta \underline{u} = \Delta \underline{f}, \quad (6.33)$$

$$\underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{W}}^{\frac{1}{2}} \Delta \underline{u} = \Delta \underline{f}, \quad (6.34)$$

$$\underline{\underline{W}}^{\frac{1}{2}} \Delta \underline{\check{u}} = \Delta \underline{f}, \quad (6.35)$$

$$\Delta \underline{\check{u}} = \underline{\underline{W}}^{-\frac{1}{2}} \Delta \underline{f}, \quad (6.36)$$

$$\Delta \underline{\check{u}} = \Delta \underline{\check{f}}. \quad (6.37)$$

Because of this equality, Eq. (6.28) can also be written as:

$$\underline{\underline{\check{L}}} \Delta \underline{\check{u}} = \Delta \underline{\check{u}}, \quad (6.38)$$

which indicates that $\Delta \underline{\check{u}}$ (and hence, $\Delta \underline{\check{f}}$) is a (non-normalized) eigenvector of $\underline{\underline{\check{L}}}$. A true (normalized) eigenvector of $\underline{\underline{\check{L}}}$ can then be defined as:

$$\Delta \underline{\check{n}} = \frac{\Delta \underline{\check{u}}}{\|\Delta \underline{\check{u}}\|}, \quad (6.39)$$

and we may also define $n_{var} - 1$ normalized vectors, $\Delta \underline{\check{m}}$, that are orthogonal to $\Delta \underline{\check{n}}$ and to each other, such that:

$$\Delta \underline{\check{n}}^T \Delta \underline{\check{n}} = 1, \quad \Delta \underline{\check{n}}^T \Delta \underline{\check{m}}^k = 0, \quad \Delta \underline{\check{m}}^{kT} \Delta \underline{\check{m}}^l = \begin{cases} 1 & \text{if } k = l \\ 0 & \text{else,} \end{cases} \quad (6.40)$$

with integers $1 \leq k \leq n_{var} - 1$ and $1 \leq l \leq n_{var} - 1$.

If we now collect all these columns in an $n_{var} \times n_{var}$ matrix, $\underline{\underline{N}}$, as follows:

$$\underline{\underline{N}} = [\Delta \underline{\check{n}} \quad \Delta \underline{\check{m}}^1 \quad \Delta \underline{\check{m}}^2 \quad \dots \quad \Delta \underline{\check{m}}^{n_{var}-1}], \quad (6.41)$$

the product of this matrix with itself reads:

$$\underline{\underline{N}} \underline{\underline{N}}^T = \underline{\underline{N}}^T \underline{\underline{N}} = \underline{\underline{I}} = \Delta \underline{\check{n}} \Delta \underline{\check{n}}^T + \sum_{k=1}^{n_{var}-1} \Delta \underline{\check{m}}^k \Delta \underline{\check{m}}^{kT}. \quad (6.42)$$

If we now regard that the Frobenius-norm (in the minimization problem of Eq. (6.43)) effectively only depends on the singular values of the matrices to which it is applied, we may multiply the matrices inside the Frobenius-norm with normalized vectors, so that we obtain:

$$\underline{\check{L}}^i = \underset{\underline{\check{L}}}{\operatorname{argmin}} \quad \|\underline{N}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \underline{N}\|. \quad (6.43)$$

If we expand the objective function, we can write:

$$\begin{aligned} \|\underline{N}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \underline{N}\| &= \left\| \begin{bmatrix} \Delta \check{n}^T \\ \Delta \check{m}^{1T} \\ \Delta \check{m}^{2T} \\ \dots \\ \Delta \check{m}^{(n_{var}-1)T} \end{bmatrix} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) [\Delta \check{n} \quad \Delta \check{m}^1 \quad \Delta \check{m}^2 \quad \dots \Delta \check{m}^{n_{var}-1}] \right\|, \\ &= \left\| \begin{bmatrix} \Delta \check{n}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{n} & \Delta \check{n}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{n}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{n}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \\ \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{n} & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \\ \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{n} & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \\ \dots & \dots & \dots & \dots & \dots \\ \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{n} & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \end{bmatrix} \right\| \\ &= \left\| \begin{bmatrix} 1 - \Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{n} & -\Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{m}^1 & -\Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{m}^2 & \dots & -\Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{m}^{n_{var}-1} \\ -\Delta \check{m}^{1T} \underline{\check{L}}^{i-1} \Delta \check{n} & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{1T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \\ -\Delta \check{m}^{2T} \underline{\check{L}}^{i-1} \Delta \check{n} & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{2T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \\ \dots & \dots & \dots & \dots & \dots \\ -\Delta \check{m}^{(n_{var}-1)T} \underline{\check{L}}^{i-1} \Delta \check{n} & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^1 & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^2 & \dots & \Delta \check{m}^{(n_{var}-1)T} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^{n_{var}-1} \end{bmatrix} \right\|, \end{aligned} \quad (6.44)$$

where the optimization variable, $\underline{\check{L}}$, is denoted in green and we have used the fact that $\Delta \check{n}$ is an eigenvector of $\underline{\check{L}}$ and that $\Delta \check{n}$ is orthonormal to $\Delta \check{m}^k$, to include the following relations:

$$\Delta \check{n}^T \underline{\check{L}} \Delta \check{n} = \Delta \check{n}^T \Delta \check{n} = 1, \quad (6.45)$$

$$\Delta \check{n}^T \underline{\check{L}} \Delta \check{m}^k = \Delta \check{n}^T \Delta \check{m}^k = 0. \quad (6.46)$$

The result of Eq. (6.44) can also be written as:

$$\begin{aligned} \underline{\check{L}}^i = \underset{\underline{\check{L}}}{\operatorname{argmin}} \quad \|\underline{N}^T (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \underline{N}\| &= \left\{ \left(1 - \Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{n} \right)^2 + \right. \\ &\quad \left. 2 \sum_{k=1}^{n_{var}-1} \left(\Delta \check{n}^T \underline{\check{L}}^{i-1} \Delta \check{m}^k \right)^2 + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \left(\Delta \check{m}^{kT} (\underline{\check{L}} - \underline{\check{L}}^{i-1}) \Delta \check{m}^l \right)^2 \right\}^{\frac{1}{2}}. \end{aligned} \quad (6.47)$$

It may be clear that in order to minimize the objective function, we only need to minimize the terms that include the (green) optimization variable. For any k and l , this means that we must ensure that:

$$\Delta \check{m}^{kT} \underline{\check{L}} \Delta \check{m}^l = \Delta \check{m}^{kT} \underline{\check{L}}^{i-1} \Delta \check{m}^l, \quad (6.48)$$

and the question is now if we can find an expression for $\underline{\check{L}}$ as a function of $\underline{\check{L}}^{i-1}$, $\Delta\check{n}$ and $\Delta\check{m}^k$ ($1 \leq k \leq n_{var} - 1$) for which Eq. (6.48) holds (for any k and l), as well as the previous equation:

$$\underline{\check{L}}\Delta\check{n} = \Delta\check{n}. \quad (6.49)$$

The answer is the following:

$$\underline{\check{L}}^i = \Delta\check{n}\Delta\check{n}^T + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \Delta\check{m}^k \Delta\check{m}^{kT} \underline{\check{L}}^{i-1} \Delta\check{m}^l \Delta\check{m}^{lT}. \quad (6.50)$$

After all, if we post-multiply this expression by $\Delta\check{n}$, we obtain:

$$\underline{\check{L}}^i \Delta\check{n} = \left(\Delta\check{n}\Delta\check{n}^T + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \Delta\check{m}^k \Delta\check{m}^{kT} \underline{\check{L}}^{i-1} \Delta\check{m}^l \Delta\check{m}^{lT} \right) \Delta\check{n}, \quad (6.51)$$

$$= \Delta\check{n}\Delta\check{n}^T \Delta\check{n} + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \Delta\check{m}^k \Delta\check{m}^{kT} \underline{\check{L}}^{i-1} \Delta\check{m}^l \Delta\check{m}^{lT} \Delta\check{n}, \quad (6.52)$$

$$= \Delta\check{n}. \quad (6.53)$$

Furthermore, if we pre-multiply and post-multiply the expression for $\underline{\check{L}}^i$ with $\Delta\check{m}^p$ and $\Delta\check{m}^q$ (with $1 \leq p \leq n_{var} - 1$ and $1 \leq q \leq n_{var} - 1$), we obtain:

$$\Delta\check{m}^{pT} \underline{\check{L}}^i \Delta\check{m}^q = \Delta\check{m}^{pT} \left(\Delta\check{n}\Delta\check{n}^T + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \Delta\check{m}^k \Delta\check{m}^{kT} \underline{\check{L}}^{i-1} \Delta\check{m}^l \Delta\check{m}^{lT} \right) \Delta\check{m}^q, \quad (6.54)$$

$$= \Delta\check{m}^{pT} \Delta\check{n}\Delta\check{n}^T \Delta\check{m}^q + \sum_{k=1}^{n_{var}-1} \sum_{l=1}^{n_{var}-1} \Delta\check{m}^{pT} \Delta\check{m}^k \Delta\check{m}^{kT} \underline{\check{L}}^{i-1} \Delta\check{m}^l \Delta\check{m}^{lT} \Delta\check{m}^q, \quad (6.55)$$

$$= \Delta\check{m}^{pT} \underline{\check{L}}^{i-1} \Delta\check{m}^q, \quad (6.56)$$

which is indeed the same as we have obtained from the minimization. Just to be clear, we have derived this results amongst others using the relation:

$$\sum_{k=1}^{n_{var}-1} \Delta\check{m}^k \Delta\check{m}^{qT} = \Delta\check{m}^q \Delta\check{m}^{qT} = 1. \quad (6.57)$$

Hence, the relation in Eq. (6.50) indeed minimizes the objective function and abides the relation in Eq. (6.49). The only issue is remaining is to express the relation in terms of $\underline{\check{L}}^{i-1}$, $\Delta\check{u}$ and $\Delta\check{f}$.

To this purpose, we first exploit the relation in Eq. (6.42), so that we can write Eq. (6.50) as follows:

$$\underline{\check{L}}^i = \Delta \check{n} \Delta \check{n}^T + (\underline{I} - \Delta \check{n} \Delta \check{n}^T) \underline{\check{L}}^{i-1} (\underline{I} - \Delta \check{n} \Delta \check{n}^T). \quad (6.58)$$

Subsequently, we use the expressions in Eq. (6.26), to write:

$$\underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^i \underline{\underline{W}}^{\frac{1}{2}} = \Delta \check{n} \Delta \check{n}^T + (\underline{I} - \Delta \check{n} \Delta \check{n}^T) \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^{i-1} \underline{\underline{W}}^{\frac{1}{2}} (\underline{I} - \Delta \check{n} \Delta \check{n}^T). \quad (6.59)$$

The question is now which expression for each $\Delta \check{n}$ we should use, since we can choose from:

$$\Delta \check{n} = \frac{\underline{\underline{W}}^{\frac{1}{2}} \Delta u}{\|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\|}, \quad \Delta \check{n} = \frac{\underline{\underline{W}}^{\frac{-1}{2}} \Delta f}{\|\underline{\underline{W}}^{\frac{-1}{2}} \Delta f\|}. \quad (6.60)$$

Let's pick a combination and see what happens:

$$\begin{aligned} \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^i \underline{\underline{W}}^{\frac{1}{2}} = \frac{\underline{\underline{W}}^{\frac{1}{2}} \Delta u \Delta u^T \underline{\underline{W}}^{\frac{1}{2}}}{\|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\| \|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\|} + \left(\underline{I} - \frac{\underline{\underline{W}}^{\frac{1}{2}} \Delta u \Delta f^T \underline{\underline{W}}^{\frac{-1}{2}}}{\|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\| \|\underline{\underline{W}}^{\frac{-1}{2}} \Delta f\|} \right) \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^{i-1} \underline{\underline{W}}^{\frac{1}{2}} \\ \left(\underline{I} - \frac{\underline{\underline{W}}^{\frac{-1}{2}} \Delta f \Delta u^T \underline{\underline{W}}^{\frac{1}{2}}}{\|\underline{\underline{W}}^{\frac{-1}{2}} \Delta f\| \|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\|} \right), \end{aligned} \quad (6.61)$$

Now we include the following two relations (be aware that $\underline{\underline{W}}$ is symmetric):

$$\|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\| \|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\| = \left(\underline{\underline{W}}^{\frac{1}{2}} \Delta u \right)^T \underline{\underline{W}}^{\frac{1}{2}} \Delta u = \Delta u^T \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{W}}^{\frac{1}{2}} \Delta u = \Delta u^T \underline{\underline{W}} \Delta u = \Delta u^T \Delta f, \quad (6.62)$$

$$\|\underline{\underline{W}}^{\frac{1}{2}} \Delta u\| \|\underline{\underline{W}}^{\frac{-1}{2}} \Delta f\| = \left(\underline{\underline{W}}^{\frac{1}{2}} \Delta u \right)^T \underline{\underline{W}}^{\frac{-1}{2}} \Delta f = \Delta u^T \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{W}}^{\frac{-1}{2}} \Delta f = \Delta u^T \underline{I} \Delta f = \Delta u^T \Delta f, \quad (6.63)$$

so that we can write:

$$\begin{aligned} \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^i \underline{\underline{W}}^{\frac{1}{2}} = \frac{\underline{\underline{W}}^{\frac{1}{2}} \Delta u \Delta u^T \underline{\underline{W}}^{\frac{1}{2}}}{\Delta f^T \Delta u} + \left(\underline{I} - \frac{\underline{\underline{W}}^{\frac{1}{2}} \Delta u \Delta f^T \underline{\underline{W}}^{\frac{-1}{2}}}{\Delta f^T \Delta u} \right) \underline{\underline{W}}^{\frac{1}{2}} \underline{\underline{L}}^{i-1} \underline{\underline{W}}^{\frac{1}{2}} \\ \left(\underline{I} - \frac{\underline{\underline{W}}^{\frac{-1}{2}} \Delta f \Delta u^T \underline{\underline{W}}^{\frac{1}{2}}}{\Delta f^T \Delta u} \right), \end{aligned} \quad (6.64)$$

It is not difficult to see that the final result for $\underline{\underline{L}}^i$ thus reads:

$$\underline{\underline{L}}^i = \frac{\Delta \underline{u} \Delta \underline{u}^T}{\Delta \underline{f}^T \Delta \underline{u}} + \left(\underline{\underline{I}} - \frac{\Delta \underline{u} \Delta \underline{f}^T}{\Delta \underline{f}^T \Delta \underline{u}} \right) \underline{\underline{L}}^{i-1} \left(\underline{\underline{I}} - \frac{\Delta \underline{f} \Delta \underline{u}^T}{\Delta \underline{f}^T \Delta \underline{u}} \right), \quad (6.65)$$

and it is easy to show that this expression is the same as the one in Eq. (6.3).

6.3 L-BFGS: Avoiding storage problems

The update of the inverse Hessian approximation in the BFGS method is always a full matrix. This entails that its storage involves $n_{var}^2/2 + n_{var}/2$ independent scalars, which limits the method's applicability with respect to the number of variables, i.e. only problems with a moderate number of variables can be handled. To avoid this issue of storage, the limited-storage BFGS (L-BFGS) was introduced. It overcomes the problem of storage by only storing the initially selected Hessian, the difference between the gradients (i.e. $\Delta \underline{f}$) and the difference between the updated minimizers (i.e. $\Delta \underline{u}$) of the last n_{li} iterations. If the initial Hessian is diagonal, this reduces the number of independent scalars to store to $(2n_{li} + 1)n_{var}$.

To explain the workings of the limited-storage BFGS method, we again look at the update of the inverse Hessian approximation as expressed in Eq. (6.65):

$$\underline{\underline{L}}^i = (\underline{\underline{I}} - \rho \Delta \underline{u} \Delta \underline{f}^T) \underline{\underline{L}}^{i-1} (\underline{\underline{I}} - \rho \Delta \underline{f} \Delta \underline{u}^T) + \rho \Delta \underline{u} \Delta \underline{u}^T, \quad (6.66)$$

with the abbreviation:

$$\rho = \frac{1}{\Delta \underline{f}^T \Delta \underline{u}}. \quad (6.67)$$

Since ρ , $\Delta \underline{u}$ and $\Delta \underline{f}$ change in each iteration, and are taken from the previous iteration, it is actually more precise to include superscript $i-1$:

$$\underline{\underline{L}}^i = \left(\underline{\underline{I}} - \rho^{i-1} \Delta \underline{u}^{i-1} \Delta \underline{f}^{(i-1)T} \right) \underline{\underline{L}}^{i-1} \left(\underline{\underline{I}} - \rho^{i-1} \Delta \underline{f}^{i-1} \Delta \underline{u}^{(i-1)T} \right) + \rho^{i-1} \Delta \underline{u}^{i-1} \Delta \underline{u}^{(i-1)T}. \quad (6.68)$$

This expression can be abbreviated as follows:

$$\underline{\underline{L}}^i = \underline{\underline{V}}^{(i-1)T} \underline{\underline{L}}^{i-1} \underline{\underline{V}}^{i-1} + \rho^{i-1} \Delta \underline{u}^{i-1} \Delta \underline{u}^{(i-1)T}, \quad (6.69)$$

where:

$$\underline{\underline{V}}^{i-1} = \underline{\underline{I}} - \rho^{i-1} \Delta \underline{f}^{i-1} \Delta \underline{u}^{(i-1)T}. \quad (6.70)$$

If we work this expression out for the second iteration, the first update for the inverse Hessian approximation (that takes place in iteration 2) reads:

$$\begin{aligned}\underline{\tilde{L}}^2 &= \underline{V}^{1T} \underline{\tilde{L}}^0 \underline{V}^1 \\ &\quad + \rho^1 \Delta \underline{u}^1 \Delta \underline{u}^{1T},\end{aligned}\tag{6.71}$$

where $\underline{\tilde{L}}^0$ denotes the initial Hessian's inverse as selected by the user and $\underline{V}^1$, ρ^1 and $\Delta \underline{u}^1$ are all computed in iteration 1. If we continue expanding the update for subsequent iterations, we obtain:

$$\begin{aligned}\underline{\tilde{L}}^3 &= \underline{V}^{2T} \underline{V}^{1T} \underline{\tilde{L}}^0 \underline{V}^1 \underline{V}^2 \\ &\quad + \underline{V}^{2T} \rho^1 \Delta \underline{u}^1 \Delta \underline{u}^{1T} \underline{V}^2 \\ &\quad + \rho^2 \Delta \underline{u}^2 \Delta \underline{u}^{2T},\end{aligned}\tag{6.72}$$

for iteration 3, and:

$$\begin{aligned}\underline{\tilde{L}}^4 &= \underline{V}^{3T} \underline{V}^{2T} \underline{V}^{1T} \underline{\tilde{L}}^0 \underline{V}^1 \underline{V}^2 \underline{V}^3 \\ &\quad + \underline{V}^{3T} \underline{V}^{2T} \rho^1 \Delta \underline{u}^1 \Delta \underline{u}^{1T} \underline{V}^2 \underline{V}^3 \\ &\quad + \underline{V}^{3T} \rho^2 \Delta \underline{u}^2 \Delta \underline{u}^{2T} \underline{V}^3 \\ &\quad + \rho^3 \Delta \underline{u}^3 \Delta \underline{u}^{3T},\end{aligned}\tag{6.73}$$

for iteration 4. Skipping some iterations, we obtain the following update for iteration 7:

$$\begin{aligned}\underline{\tilde{L}}^7 &= \underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \underline{V}^{3T} \underline{V}^{2T} \underline{V}^{1T} \underline{\tilde{L}}^0 \underline{V}^1 \underline{V}^2 \underline{V}^3 \underline{V}^4 \underline{V}^5 \underline{V}^6 \\ &\quad + \underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \underline{V}^{3T} \underline{V}^{2T} \rho^1 \Delta \underline{u}^1 \Delta \underline{u}^{1T} \underline{V}^2 \underline{V}^3 \underline{V}^4 \underline{V}^5 \underline{V}^6 \\ &\quad + \underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \underline{V}^{3T} \rho^2 \Delta \underline{u}^2 \Delta \underline{u}^{2T} \underline{V}^3 \underline{V}^4 \underline{V}^5 \underline{V}^6 \\ &\quad + \underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \rho^3 \Delta \underline{u}^3 \Delta \underline{u}^{3T} \underline{V}^4 \underline{V}^5 \underline{V}^6 \\ &\quad + \underline{V}^{6T} \underline{V}^{5T} \rho^4 \Delta \underline{u}^4 \Delta \underline{u}^{4T} \underline{V}^5 \underline{V}^6 \\ &\quad + \underline{V}^{6T} \rho^5 \Delta \underline{u}^5 \Delta \underline{u}^{5T} \underline{V}^6 \\ &\quad + \rho^6 \Delta \underline{u}^6 \Delta \underline{u}^{6T}.\end{aligned}\tag{6.74}$$

As we have mentioned before, only the information of the last n_{li} iterations is incorporated in the update in the L-BFGS approach. This entails for iteration 7 in case $n_{li} = 3$, that the update reads:

$$\begin{aligned}
\underline{\tilde{L}}^7 = & \underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \underline{\tilde{L}}^0 \underline{V}^4 \underline{V}^5 \underline{V}^6 \\
& + \underline{V}^{6T} \underline{V}^{5T} \rho^4 \Delta \underline{u}^4 \Delta \underline{u}^{4T} \underline{V}^5 \underline{V}^6 \\
& + \underline{V}^{6T} \rho^5 \Delta \underline{u}^5 \Delta \underline{u}^{5T} \underline{V}^6 \\
& + \rho^6 \Delta \underline{u}^6 \Delta \underline{u}^{6T},
\end{aligned} \tag{6.75}$$

instead of Eq. (6.74). The search direction for iteration 7 thus reads:

$$\begin{aligned}
\underline{h}^7 = & - \left(\underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T} \underline{\tilde{L}}^0 \underline{V}^4 \underline{V}^5 \underline{V}^6 \right. \\
& + \underline{V}^{6T} \underline{V}^{5T} \rho^4 \Delta \underline{u}^4 \Delta \underline{u}^{4T} \underline{V}^5 \underline{V}^6 \\
& + \underline{V}^{6T} \rho^5 \Delta \underline{u}^5 \Delta \underline{u}^{5T} \underline{V}^6 \\
& \left. + \rho^6 \Delta \underline{u}^6 \Delta \underline{u}^{6T} \right) \underline{f}^6,
\end{aligned} \tag{6.76}$$

where $\underline{f}^6$ denotes the gradient computed in iteration 6.

The issue with calculating the search direction using the matrices of Eq. (6.76) is that matrices $\underline{V}$ and $\Delta \underline{u} \Delta \underline{u}^T$ are full matrices. To avoid storing any matrix, the algorithm to compute the search direction only involves column-column products (involving columns $\Delta \underline{f}$ and $\Delta \underline{u}$ that make up matrices $\underline{V}$). A pseudo-algorithm to determine the search direction in the L-BFGS approach is presented in Table 6.3. The comments in the pseudo-algorithm are related to letters A to F in the following expression (for iteration 7 in case $n_{li} = 3$, which is the same as in (Eq. 6.76)):

$$\underline{h}^7 = \underbrace{-}_{\text{F}} \left(\underbrace{\underline{V}^{6T} \underline{V}^{5T} \underline{V}^{4T}}_{\text{D}} \underbrace{\underline{\tilde{L}}^0}_{\text{C}} \underbrace{\underline{V}^4 \underline{V}^5 \underline{V}^6}_{\text{B}} + \underbrace{\underline{V}^{6T} \underline{V}^{5T} \rho^4 \Delta \underline{u}^4 \Delta \underline{u}^{4T} \underline{V}^5 \underline{V}^6 + \underline{V}^{6T} \rho^5 \Delta \underline{u}^5 \Delta \underline{u}^{5T} \underline{V}^6 + \rho^6 \Delta \underline{u}^6 \Delta \underline{u}^{6T}}_{\text{E}} \right) \underbrace{\underline{f}^6}_{\text{A}}. \tag{6.77}$$

Table 6.4 presents a pseudo-algorithm of the entire L-BFGS approach, in which the algorithm of Table 6.3 to determine the search direction is incorporated. In Fig. 6.3 the updates of the L-BFGS are compared to those of others for the Rosenbrock function.

Positive definiteness of the L-BFGS Hessian's inverse

Finally, we will show that, similar to the Hessian's inverse of the BFGS approach, the Hessian's inverse of the L-BFGS approach is also positive definite (which entails that the Hessian approximated by the L-BFGS exists). To this purpose, we write the L-BFGS update in a general format:

$$\underline{L}^i = \underline{G}^T \underline{L}^0 \underline{G} + \sum_{j=i-n_{li}-1}^{i-1} \underline{H}^{jT} \rho^j \Delta \underline{u}^j \Delta \underline{u}^{jT} \underline{H}^j, \tag{6.78}$$

Table 6.3: A pseudo-algorithm to compute the search direction in the L-BFGS method.
The algorithm assumes that $i > n_{li}$, where i denotes the iteration number.

INPUT: $\underline{\underline{L}}^0, \Delta \underline{f}^{i-n_{li}}$ to $\Delta \underline{f}^{i-1}, \Delta \underline{u}^{i-n_{li}}$ to $\Delta \underline{u}^{i-1}, \rho^{i-n_{li}}$ to $\rho^{i-1}, \underline{f}^{i-1}, i, n_{li}$	
1	$\underline{h} = \underline{f}^{i-1}$ % A in Eq. (6.77): set the search direction to the gradient of the previous iteration
	% Evaluate the influence of B on A in Eq. (6.77): multiply B with A
2	for $j = i - 1 : i - n_{li}$
3	$\gamma^j = \rho^j \Delta \underline{u}^{jT} \underline{h}$
4	$\underline{h} = \underline{h} - \gamma^j \Delta \underline{f}^j$
5	end
6	$\underline{h} = \underline{\underline{L}}^0 \underline{h}$ % Evaluate the influence of C on the result of B times A in Eq. (6.77)
	% Evaluate the influence of D and E on the result of C times B times A in Eq. (6.77)
7	for $j = i - n_{li} : i - 1$
8	$\eta = \rho^j \Delta \underline{f}^{jT} \underline{h}$
9	$\underline{h} = \underline{h} + (\gamma^j - \eta) \Delta \underline{u}^j$ % γ accounts for D and η accounts for E in Eq. (6.77)
10	end
11	$\underline{h}^i = -\underline{h}$ % F in Eq. (6.77): account for minus sign
OUTPUT: $\underline{h}^i$	

with:

$$\underline{\underline{G}} = \prod_{j=i-n_{li}-1}^{i-1} \underline{\underline{V}}^j, \quad (6.79)$$

$$\underline{\underline{H}}^j = \begin{cases} \underline{\underline{I}} & \text{if } j = i - 1, \\ \prod_{k=j}^{i-1} \underline{\underline{V}}^k & \text{else,} \end{cases} \quad (6.80)$$

where $\prod_{k=a}^b \underline{\underline{V}}^k = \underline{\underline{V}}^a \underline{\underline{V}}^{a+1} \underline{\underline{V}}^{a+2} \dots \underline{\underline{V}}^b$.

Considering first $\underline{\underline{G}}^T \underline{\underline{L}}^0 \underline{\underline{G}}$ and inserting this in the definition of positive definite matrix, $\underline{\underline{A}}$:

$$\underline{x}^T \underline{\underline{A}} \underline{x} > 0, \quad (6.81)$$

we obtain:

$$\underline{x}^T \underline{\underline{G}}^T \underline{\underline{L}}^0 \underline{\underline{G}} \underline{x} > 0, \quad (6.82)$$

which, with the help of $\underline{y} = \underline{\underline{G}} \underline{x}$, can be written as:

$$\underline{y}^T \underline{\underline{L}}^0 \underline{y} > 0, \quad (6.83)$$

Table 6.4: A pseudo-algorithm for the L-BFGS method. To save space, the line search is not presented. The iteration number is now denoted by cnt .

	<i>%INITIALISATION</i>
1	Set $\underline{a}, \underline{g}, n_{var}$ <i>% set parameters of the objective function</i>
2	Set $\alpha_{init}, 0 < c^1 < c^2 < 1, 0 < r < 1, \underline{u}, \underline{\tilde{L}}^0 = \text{eye}(n_{var}), n_{li}, \underline{\Delta F} = \underline{\Delta U} = \text{zeros}(n_{var}, n_{li}), \underline{\rho} = \text{zeros}(n_{li}, 1)$ <i>% set parameters of the algorithm</i>
	<i>%QUASI-NEWTON ALGORITHM</i>
3	Compute $m^{new} = m(\underline{u})$ and $\underline{f}^2 = \frac{\partial m}{\partial \underline{u}} _{\underline{u}}$
4	$m^{old} = 10^{100}, cnt = 0$ <i>% dummy values and counter</i>
5	while $m^{new} < m^{old}$
6	$m^{old} = m^{new}, \underline{f}^{new} = \underline{f}^2, cnt = cnt + 1$
	<i>%Determine search direction</i>
7	if $cnt = 1$
8	$\underline{h} = -\underline{\tilde{L}}^0 \underline{f}^{new}$
9	else
10	$\underline{\gamma} = \text{zeros}(n_{li}, 1)$
11	$\underline{h} = \underline{f}^{new}$
12	for $j = n_{li} : -1 : \max([1, n_{li} - cnt + 2])$
13	$\underline{\rho} = \underline{\rho}(j), \underline{\Delta u} = \underline{\Delta U}(:, j), \underline{\Delta f} = \underline{\Delta F}(:, j)$
14	$\underline{\gamma} = \underline{\rho} \underline{\Delta u}^T \underline{h}$
15	$\underline{h} = \underline{h} - \underline{\gamma} \underline{\Delta f}$
16	$\underline{\gamma}(j) = \underline{\gamma}$
17	end
18	$\underline{h} = \underline{\tilde{L}}^0 \underline{h}$
19	for $j = \max([1, n_{li} - cnt + 2]) : n_{li}$
20	$\underline{\rho} = \underline{\rho}(j), \underline{\Delta u} = \underline{\Delta U}(:, j), \underline{\Delta f} = \underline{\Delta F}(:, j), \underline{\gamma} = \underline{\gamma}(j)$
21	$\underline{\eta} = \underline{\rho} \underline{\Delta f}^T \underline{h}$
22	$\underline{h} = \underline{h} + (\underline{\gamma} - \underline{\eta}) \underline{\Delta u}$
23	end
24	$\underline{h} = -\underline{h}$
25	end
	<i>%Line search</i>
	TAKE LINE SEARCH FROM TABLE 6.1 OR TABLE 6.2
	<i>%Complete iteration</i>
c.1	$\underline{\Delta u} = \underline{u}^x - \underline{u}, \underline{u} = \underline{u}^x$
c.2	if $signal = 1$
c.3	$m^{new} = m^3, \underline{f}^2 = \underline{f}^3$
c.4	else
c.5	$m^{new} = m^2$
c.6	end
c.7	$\underline{\Delta f} = \underline{f}^2 - \underline{f}^{new}$
c.8	$\underline{\Delta U}(:, 1 : n_{li} - 1) = \underline{\Delta U}(:, 2 : n_{li}), \underline{\Delta U}(:, n_{li}) = \underline{\Delta u}$ <i>%Make space for $\underline{\Delta u}$ in $\underline{\Delta U}$ and store it</i>
c.9	$\underline{\Delta F}(:, 1 : n_{li} - 1) = \underline{\Delta F}(:, 2 : n_{li}), \underline{\Delta F}(:, n_{li}) = \underline{\Delta f}$ <i>%Make space for $\underline{\Delta f}$ in $\underline{\Delta F}$ and store it</i>
c.10	$\underline{\rho}(1 : n_{li} - 1) = \underline{\rho}(2 : n_{li}), \underline{\rho}(n_{li}) = 1/(\underline{\Delta f}^T \underline{\Delta u})$ <i>%Make space for $1/(\underline{\Delta f}^T \underline{\Delta u})$ in $\underline{\rho}$ and store it</i>
c.11	end
	<i>%COMPLETION</i>
c.12	$m^* = m^{old}, \underline{u}^* = \underline{u} - \underline{\Delta u}$

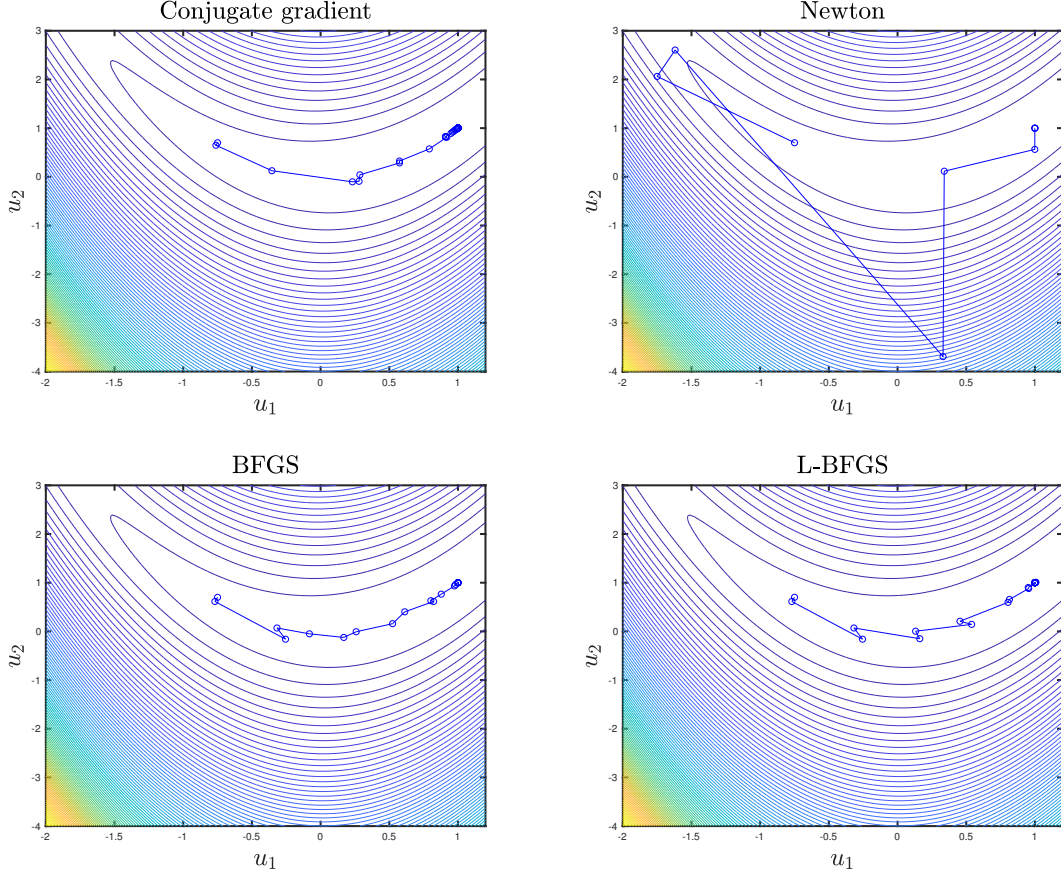


Figure 6.3: The updates of the L-BFGS method with the Wolfe conditions ($n_{li} = 2$) for the Rosenbrock function starting at $\underline{u} = [-0.75 \ 0.7]^T$ compared to other methods.

which is indeed true, because $\underline{\underline{L}}^0$ must be selected to be positive definite. In other words, $\underline{\underline{G}}^T \underline{\underline{L}}^0 \underline{\underline{G}}$ is positive definite. If we now focus on $\underline{\underline{H}}^{jT} \rho^j \Delta \underline{u}^j \Delta \underline{u}^{jT} \underline{\underline{H}}^j$ and insert this in the definition of a positive definite matrix, we obtain:

$$\underline{x}^T \underline{\underline{H}}^{jT} \rho^j \Delta \underline{u}^j \Delta \underline{u}^{jT} \underline{\underline{H}}^j \underline{x} > 0. \quad (6.84)$$

Incorporating that $\Delta \underline{u}^{jT} \underline{\underline{H}}^j \underline{x}$ is a scalar, which we for instance denote by a , we obtain:

$$\rho^j a^2 > 0, \quad (6.85)$$

which is indeed true if $\rho^j (= \Delta \underline{u}^{jT} \Delta \underline{f}^j)$ is positive. In other words, just like the BFGS update, the L-BFGS update also yields an invertible Hessian, provided that the line search abides the Wolfe conditions.

6.4 Questions

Question 6.1 Implement the BFGS method with the Wolfe conditions to determine the minima of (using $\underline{u}^0 = \underline{0}$):

Model 1: All a 's are 1, all g 's are 1.

Model 2a: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 2b: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 3a: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 3b: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 4a: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{79} = 1$.

Question 6.2 Implement the L-BFGS method with the Wolfe conditions to determine the minima of (using $\underline{u}^0 = \underline{0}$):

Model 1: All a 's are 1, all g 's are 1.

Model 2a: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 2b: All a 's are 1, all g 's are 0 except $g_2 = g_3 = 1$.

Model 3a: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 3b: All a 's are 1, all g 's are 0 except $g_2 = g_{19} = 1$.

Model 4a: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{62} = g_{79} = 1$.

Model 4b: All a 's are 1, all g 's are 0 except $g_{79} = 1$.

Constrained minimisation

In the previous chapters, we have discussed *unconstrained* numerical minimisation for continuous objective functions. Often however, we have to perform the minimisation under one or more *constraints*. One example is the optimisation of the geometry of a machine part, where the geometry is not allowed to be in the space that is used by other machine parts. Another example is that the optimisation of material parameters, since many parameters are bounded by the laws of physics. Another example considers the purchasing of stocks: one constraint is that the amount of money to purchase stocks is fixed. In summary, it can be stated that constraints are present in all the domains in which optimisation itself is present.

7.1 Equality constraints

Let us return to a two dimensional objective function that we have seen before:

$$m(\underline{u}) = u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20}(u_1^2 + u_2^2), \quad (7.1)$$

but now we want to minimise it with the following constraint present:

$$c(\underline{u}) = u_2 - (u_1 + 0.1)^3 - 1 = 0. \quad (7.2)$$

This is commonly expressed as follows:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) \quad (7.3)$$

$$\text{such that } \underline{c}(\underline{u}) = \underline{0}, \quad (7.4)$$

where the bar below c indicates that constraints are stored in a column, and hence, multiple constraints may be present. For the objective function of Eq. (7.1) with the constraint of Eq. (7.2), we can thus write:

$$\underline{u}^* = \arg \min_{\underline{u}} u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20}(u_1^2 + u_2^2) \quad (7.5)$$

$$\text{such that } u_2 - (u_1 + 0.1)^3 - 1 = 0. \quad (7.6)$$

In the top-left diagram of Fig. 7.1, the isolines of the objective function are presented with the constraint as a black line. In effect, the minimisation of the objective function under the presence of the constraint means that the admissible minimum has to be located on top of the constraint. A very logical thing to do is therefore to write the constraint as:

$$u_2 = (u_1 + 0.1)^3 + 1, \quad (7.7)$$

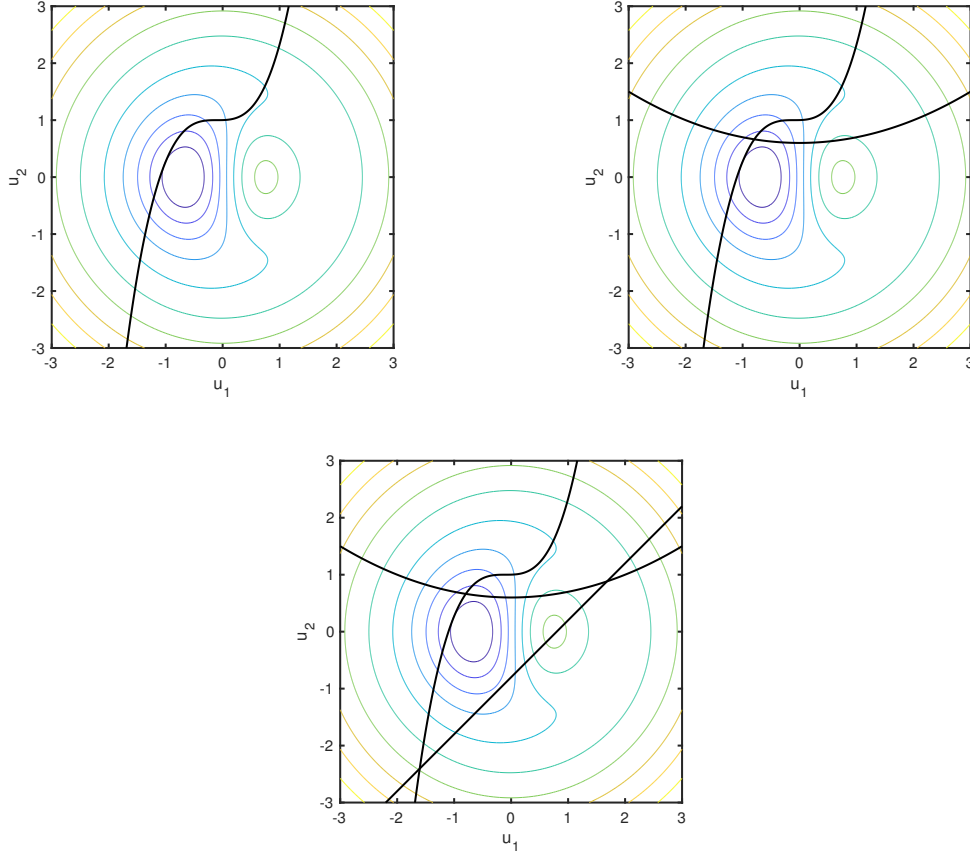


Figure 7.1: Isolines of m according to Eq. (7.1), with different constraints presented as black lines.

and substitute this expression in the original objective function, which yields a one dimensional objective function:

$$u_1^* = \arg \min_{u_1} u_1 e^{-u_1^2 - ((u_1 + 0.1)^3 + 1)^2} + \frac{u_1^2}{20} + \frac{1}{20} ((u_1 + 0.1)^3 + 1)^2. \quad (7.8)$$

This substitution is quite convenient, because it reduces the number of variables of the objective function and as we have seen in the previous chapters, the less variables are present in the objective function, the faster the minimisation.

However, substitution becomes less practical if constraints depending on many more variables are considered. However, the real difficulties appear if *inequality* constraints must be incorporated.

So just to be clear: **substitution** is used to incorporate equality constraints in minimisation problems, but mostly for *equality* constraints that can easily be substituted in the objective function. For more complex *equality* and *inequality* constraints, other methods are considered, such as penalty methods (Chapter 7) or methods using Lagrange multipliers (Chapter 8).

7.2 Constraint counting

Above we have considered an objective function of two variables with one constraint, but what happens if more constraints are presents? Let us consider now one additional constraint, as shown in the top-right diagram of Fig. 7.1. If we now want to minimise the same objective function, but with the two illustrated constraints present, we cannot truly speak of a minimisation problem anymore. After all, only one location is admissible: the one where the two constraints meet each other. The value of the objective function at this location is completely irrelevant.

Now let us add another constraint, as shown in the bottom diagram of Fig. 7.1. In this case, the constrained minimisation problem has no solution at all, since not a single location can be found for which all three constraints hold.

So a crude approximation, that is nevertheless quite useful and often employed, is to count the number of constraints (n_c) and the number of variables of the objective function (n_{var}) and determine their ratio. Then,

- $\frac{n_{var}}{n_c} > 1$; a solution exists.
- $\frac{n_{var}}{n_c} = 1$; the solution is completely determined by the constraints.
- $\frac{n_{var}}{n_c} < 1$; no solution exists.

That this is only a *crude* rule of thumb is illustrated in Fig. 7.2. On the left for instance, we see two constraints (for a two dimensional objective function) and hence, $\frac{n_{var}}{n_c} = 1$. However, no solution exists because the constraints are never equal to each other (i.e. no point can be found at which both constraints meet). However, according to the rule of thumb above, one solution should exist.

In the right diagram of Fig. 7.2 on the other hand, three constraints are present (for a two dimensional objective function) and hence, $\frac{n_{var}}{n_c} < 1$. One would therefore expect that no solution exists. In this case however, one solution exists because all three constraints meet in the same point.

In other words, constraint counting is not always accurate, but it often provides a good idea if the constrained minimisation problem has a solution or not.

7.3 Inequality constraints

So far we have only considered *equality* constraints, which are constraints that must hold under *all circumstances*. An example of an equality constraint in a minimisation problem in which the fastest route between points A and B must be found, is that the route must follow roads (and cannot cross fields, forests, etcetera).

Inequality constraints on the other hand only indicate an inadmissible domain. An example of an inequality constraint in a minimisation problem in which the mass density of an object is a variable, is that the mass density must be positive. Another example of an inequality constraint

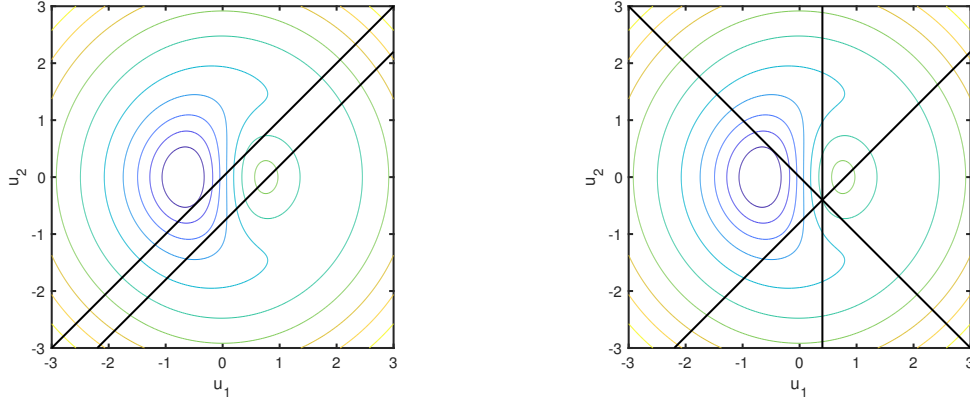


Figure 7.2: Isolines of m according to Eq. (7.1), with different constraints presented as black lines.

in a minimisation problem in which the velocity of an object is a variable, is that the velocity cannot be larger than the speed of light.

Now let us consider the same minimisation problem as in the beginning of this chapter, except that the equality constraint is replaced by the following inequality constraint:

$$\underline{u}^* = \arg \min_{\underline{u}} u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20}(u_1^2 + u_2^2) \quad (7.9)$$

$$\text{such that } u_2 - (u_1 + 0.1)^3 - 1 \leq 0. \quad (7.10)$$

This minimisation problem is graphically illustrated on the left in Fig. 7.3. If we turn the inequality constraint around:

$$\underline{u}^* = \arg \min_{\underline{u}} u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20}(u_1^2 + u_2^2) \quad (7.11)$$

$$\text{such that } u_2 - (u_1 + 0.1)^3 - 1 \geq 0, \quad (7.12)$$

the situation as presented as on the right in Fig. 7.3 occurs. **Substitution** is not the most convenient way to incorporate inequality constraints, since we do not know in advance which, if any, inequality constraints will be active. (For the two-dimensional cases in Fig. 7.3 however, this is relatively trivial).

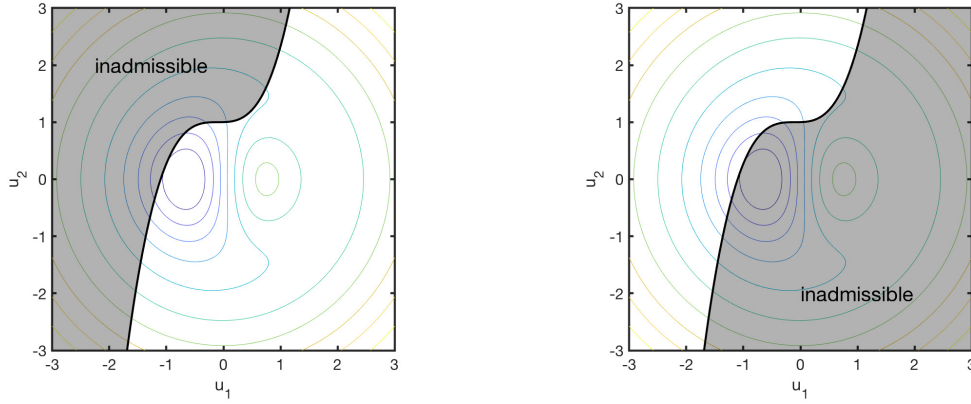


Figure 7.3: Isolines of m according to Eq. (7.1), with the inequality constraint of Eq. (7.10) in the left diagram and with the inequality constraint of Eq. (7.12) in the right diagram.

7.4 Questions

Question 7.1 Find the minimum and minimisers of the unconstrained minimisation problem of Eqs. (7.1) using Newton's method. Use a random initial guess in domain $(-1, 1) \times (-1, 1)$ and $tol = 10^{-12}$. Include a maximum number of iterations in your code, as well as a check that if the maximum number of iterations is reached, no stationary point is found. Also include a check that if a stationary point is found, a notification is outputted mentioning what type of stationary point is found.

Question 7.2 Find the minimum and minimisers of the constrained minimisation problem of Eqs. (7.1) and (7.2) using Newton's method and by incorporating the constraint using substitution. Use a random initial guess in domain $(-1, 1)$ and $tol = 10^{-12}$. Include a maximum number of iterations in your code, as well as a check that if the maximum number of iterations is increased, no stationary point is found. Also include a check that if a stationary point is found, a notification is outputted mentioning what type of stationary point is found.

Question 7.3 Use the implementations you have written to solve Questions 6.1 and 6.2, together with Fig. 7.3 to determine the minimum and minimisers of the constrained minimisation problem of Eqs (7.1) and inequality constrained (7.10). Do not change *anything* in your implementations!

Question 7.4 Use the implementations you have written to solve Questions 6.1 and 6.2, together with Fig. 7.3 to determine the minimum and minimisers of the constrained minimisation problem of Eqs (7.1) and inequality constrained (7.12). Do not change *anything* in your implementations!

Constrained minimisation: the penalty method

In the previous chapter, we have seen that constraints in minimisation problems can be incorporated by substituting them in the objective function. If this is possible, it is convenient because the number of unknown variables reduces, which makes the solution procedure faster. For many cases however, the difference in speed is negligible. For instance, reducing the number of variables from 10,000 to 9,995 by means of substitution is not going to make the minimisation noticeably faster. In many cases furthermore, the constraints cannot be substituted in a useful manner. In case of *inequality* constraints moreover, one does not know if the constraint will be ‘active’ or not and then substitution is also not a very convenient way to incorporate constraints, especially not if numerous of them may be present.

Equality and inequality constraints in minimisation problems are often incorporated using the penalty method, which is the topic of the current chapter.

8.1 Equality constraints

In the penalty method, the constrained minimisation problem:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) \quad (8.1)$$

$$\text{such that } \underline{c}(\underline{u}) = \underline{0}, \quad (8.2)$$

is replaced by an unconstrained minimisation problem of the following form:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) + \frac{1}{2} \sum_{i=1}^{n_c} k_i (c_i(\underline{u}))^2, \quad (8.3)$$

where n_c denotes the number of constraints (i.e. the length of column $\underline{c}$) and $k_i \in \mathbb{R}$ denote penalty parameters, the values of which must be selected by the user (it makes sense to start with the same value for all penalty parameters).

In order to explain why this is a good way to incorporate constraints, let us again consider the constrained minimisation problem of the previous chapter:

$$\underline{u}^* = \arg \min_{\underline{u}} u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20} (u_1^2 + u_2^2) \quad (8.4)$$

$$\text{such that } u_2 - (u_1 + 0.1)^3 - 1 = 0. \quad (8.5)$$

If we replace this with:

$$\underline{u}^* = \arg \min_{\underline{u}} u_1 e^{-u_1^2 - u_2^2} + \frac{1}{20} (u_1^2 + u_2^2) + \frac{1}{2} k (u_2 - (u_1 + 0.1)^3 - 1)^2, \quad (8.6)$$

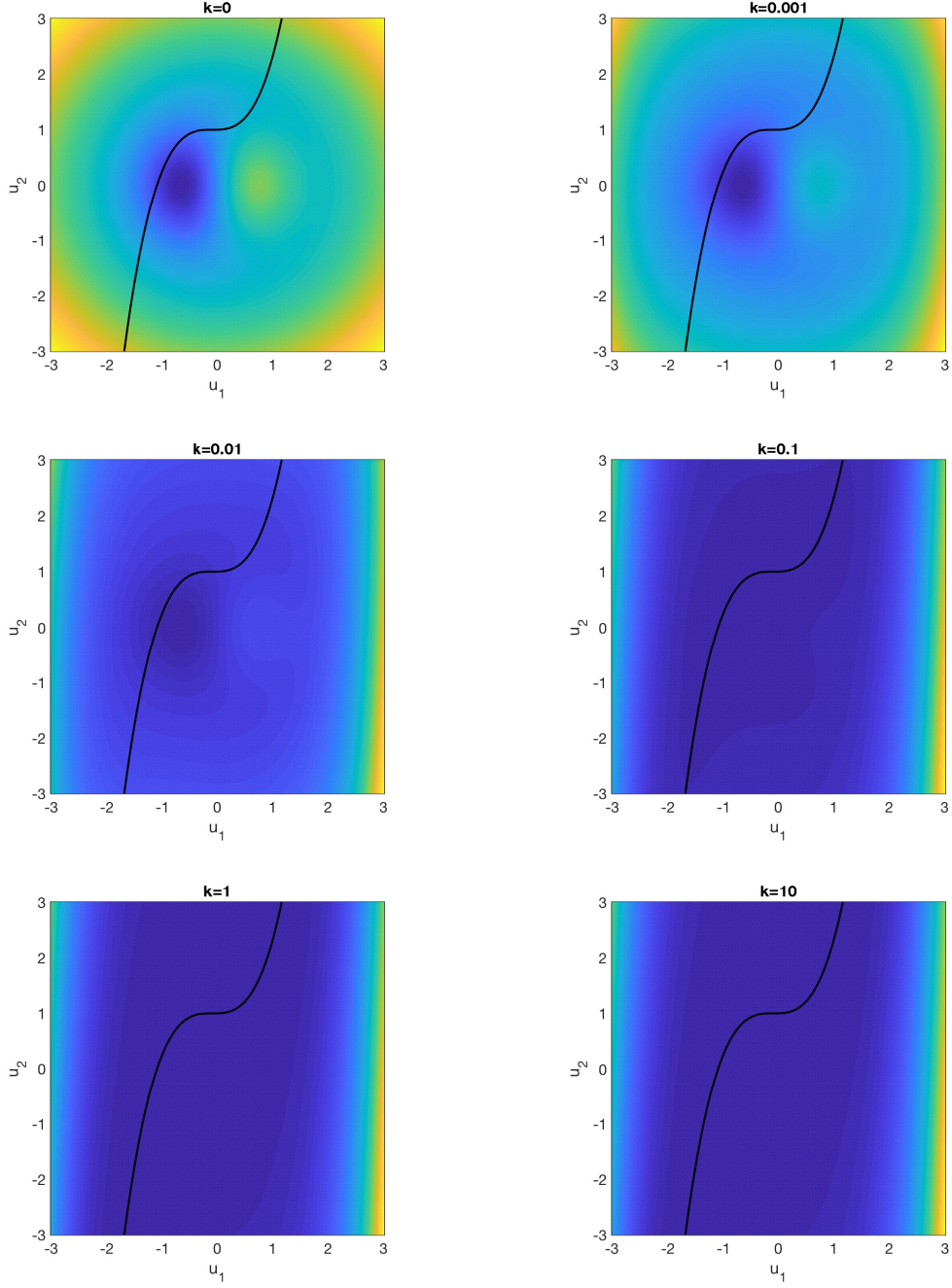


Figure 8.1: The objective function of Eq. (8.4) adjusted using the penalty method with the equality constraint of Eq. (8.5) for different values of the penalty parameter.

we can observe two things. First, the objective function *at* the constraint remains unchanged. In that case after all, $u_2 - (u_1 + 0.1)^3 - 1$ can be replaced by 0, entailing that the contribution of the penalty term is also zero.

Second, the further a combination of u_1 and u_2 is located away from the constraint, the higher the objective function (see Fig. 8.1). In other words, combinations of u_1 and u_2 that are far away from the objective function result in a large value for $(c(\underline{u}))^2$ and hence, the further away

from the constraint, the larger the value of the objective function - i.e. the objective function is increasingly more *penalised*. The fact that a square is incorporated furthermore ensures that the objective function is not only increased if $c > 0$, but also if $c < 0$.

The advantage of the penalty method over the use of substitution for the implementation is that the first-order derivatives and Hessian of the unconstrained minimisation problem ($m(\underline{u})$) remain unaltered. The first-order derivatives and the Hessian of the unconstrained minimisation problem can thus first be implemented and when it is certain that those are correct, the first order derivatives and Hessian of the constraints can simply be added to those of the unconstrained minimisation problem. In the exercises regarding Model 4a below for instance, the first-order derivatives and Hessian of the unconstrained Model 4a that were derived in the previous chapters can be used and a 'simple' *for*-loop incorporating the constraints can subsequently be included below the derivatives and Hessian (according to the sixth programming tip in Chapter 1).

The question is now what the value(s) of the penalty parameter(s) must be. In theory, we want to set the penalty parameter infinitely high, because then the constraint will be enforced perfectly. However, if we select $k = \infty$, the objective function will equal ∞ everywhere except where the constraint is perfectly zero. As the minimisation approaches we have seen in Chapter 3, 4 and 5 need the objective function to smoothly decrease towards critical points (i.e. the objective functions must be C_1 -continuous, i.e. the derivatives of the objective functions can only be zero at critical points), the minimisation approaches will not work if we choose $k = \infty$. We therefore want to choose k high. However, if the value is too large, other computational problems arise. If Newton's method is employed for instance, the Hessian may become ill-posed and hence, the systems of linear equations (in Eq. (5.5)) cannot be solved anymore.

8.2 Inequality constraints

In case of inequality constraints the story becomes slightly more difficult. To repeat, we are now interested in:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) \quad (8.7)$$

$$\text{such that } \underline{c}(\underline{u}) \leq \underline{0}, \quad (8.8)$$

or alternatively,

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) \quad (8.9)$$

$$\text{such that } \underline{c}(\underline{u}) \geq \underline{0}, \quad (8.10)$$

(or a combination of the two types, possibly also with equality constraints, but for simplicity we will not consider those, because the approaches remain the same.)

In the first case (i.e. Eqs. (8.7) and (8.8)), we can replace the constrained minimisation problem by the following unconstrained minimisation problem:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) + \frac{1}{2} \sum_{i=1}^{n_c} k_i (c_i(\underline{u}))^2 H(c_i(\underline{u})), \quad (8.11)$$

where $H(\bullet)$ denotes the Heaviside function. This function is defined as follows:

$$H(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 & \text{if } x \geq 0. \end{cases} \quad (8.12)$$

In the second case (i.e. Eqs. (8.9) and (8.10)), we can replace the constrained minimisation problem by the following unconstrained minimisation problem:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) + \frac{1}{2} \sum_{i=1}^{n_c} k_i (c_i(\underline{u}))^2 H(-c_i(\underline{u})). \quad (8.13)$$

The problem is that we do not know in advance which of the constraints must be activated or not and even worse, for different estimates of the minimisers, different constraints may be active.

To illustrate this, we will consider the objective function of Model 4a, with all a 's set to 1, $\underline{g} = \underline{0}$ and initial guess $\underline{u}_0 = \underline{0}$, subject to the following inequality constraints:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}), \quad (8.14)$$

$$\text{such that } \sqrt{(x_R - j + 31 - u_{2j-1})^2 + (y_R - 4 - u_{2j})^2} - R \geq 0 \quad j \in \{31, 32, \dots, 40\}, \quad (8.15)$$

where R denotes the radius of a circle (see Fig. 8.2) and x_R and y_R the horizontal and vertical location of the circle's center, respectively. Although Eqs. (8.14) and (8.15) look kind of complicated, their meaning is much more trivial. It means that none of the top locations where springs meet are allowed to be in the domain of the circle (see Fig. 8.2).

The technicality in Fig. 8.2 is that for the considered initial guess ($\underline{u}_0 = \underline{0}$), six of the top locations violate the inequality constraint. Hence, Eq. (8.15) must be 'activated' for six top locations. For the minimum however (right in Fig. 8.2), only four of the top locations violate the inequality constraint. Hence, for each new estimate of the minimisers, the algorithm has to determine which constraints must be activated and which not.

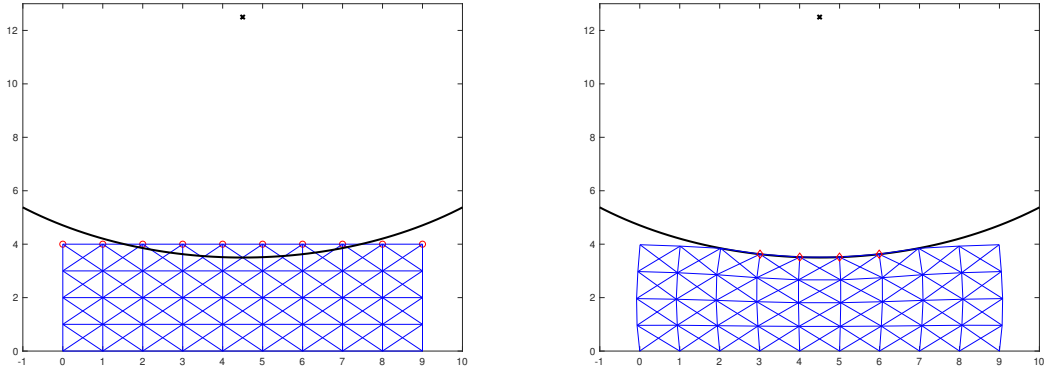


Figure 8.2: Left: Model 4a with initial guess $\underline{u}_0 = \underline{0}$, $\underline{a} = \underline{1}$ and $\underline{g} = \underline{0}$. The top locations at which springs meet are presented as red dots. The circle is presented using a black line and its center with a black cross. Right: The configuration that matches the minimum (i.e. $\underline{u}^*$). The top locations at which springs meet *and* constraints are prescribed are presented as red diamonds.

8.3 Final comment

A dangerous issue of the penalty method is that if incompatible constraints are present, such as in the bottom diagram of Fig. 7.1, each of the presented numerical methods for minimisation works perfectly well and indeed converges to a minimum and minimisers (provided the right numerical parameters are used). The problem is that even though the numerical methods behave perfectly well, the constrained minimisation problem in the bottom diagram of Fig. 7.1 is complete nonsense. Unfortunately, this cannot be spotted in the convergence behaviour of the three presented numerical methods if the penalty method is employed.

8.4 Questions

As a basis for your implementations to answer the questions below, use the implementations of the unconstrained objective functions that you have developed in Chapters 4 and 5.

Question 8.1 Find the minimisers of the minimisation problem given by the objective function of Model 4a with the following equality constraints:

$$\text{such that } \sqrt{(x_R - j + 31 - u_{2j-1})^2 + (y_R - 4 - u_{2j})^2} - R = 0 \quad j \in \{31, 32, \dots, 40\}, \quad (8.16)$$

using a penalty approach and the conjugate gradient approach for three different values of the penalty parameters: $k = 1$, $k = 100$ and $k = 10^4$. Set all values of $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ and $\underline{u}_0 = \underline{0}$,

$\alpha_{init}=0.25$, $r = 0.5$, $c = 0.5$, $n_{reset} = 25$, β according to Eq. (4.6), $R = 9$, $x_R = 4.5$ and $y_R = 12.5$.

Question 8.2 Solve the constrained minimisation problem of Question 8.1 but use Newton's method with $tol = 10^{-12}$ and $\underline{u}_0 = \underline{0}$.

Question 8.3 Find the minimisers of the minimisation problem given by the objective function of Model 4a with the following inequality constraints:

$$\text{such that } \sqrt{(x_R - j + 31 - u_{2j-1})^2 + (y_R - 4 - u_{2j})^2} - R \leq 0 \quad j \in \{31, 32, \dots, 40\}, (8.17)$$

using a penalty approach and the conjugate gradient approach for $k = 100$, $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ and $\underline{u}_0 = \underline{0}$, $\alpha_{init}=0.25$, $r = 0.5$, $c = 0.5$, $n_{reset} = 25$, β according to Eq. (4.6), $R = 9$, $x_R = 4.5$ and $y_R = 12.5$.

Question 8.4 Solve the constrained minimisation problem of Question 8.3 but use Newton's method with $tol = 10^{-12}$ and $\underline{u}_0 = \underline{0}$.

Question 8.5 Find the minimisers of the minimisation problem given by the objective function of Model 4a with the following inequality constraints:

$$\text{such that } \sqrt{(x_R - j + 31 - u_{2j-1})^2 + (y_R - 4 - u_{2j})^2} - R \geq 0 \quad j \in \{31, 32, \dots, 40\}, (8.18)$$

using a penalty approach and the conjugate gradient approach for $k = 100$, $\underline{a} = \underline{1}$, $\underline{g} = \underline{0}$ and $\underline{u}_0 = \underline{0}$, $\alpha_{init}=0.25$, $r = 0.5$, $c = 0.5$, $n_{reset} = 25$, β according to Eq. (4.6), $R = 9$, $x_R = 4.5$ and $y_R = 12.5$.

Question 8.6 Solve the constrained minimisation problem of Question 8.5 but use Newton's method with $tol = 10^{-12}$ and $\underline{u}_0 = \underline{0}$.

Constrained minimisation: the method of Lagrange multipliers

The penalty approach of the previous chapter has two disadvantages. First, the user must select appropriate values for the penalty parameters in order to ensure that the constraints hold sufficiently accurately. The problem is that if too small values are selected, the constraints are not accurately enforced, whereas if too large values are selected, Newton's method (our favorite approach because of its speed) will not converge. Second, if incompatible constraints are enforced (yielding the constrained minimisation problem nonsensical, such as in the bottom diagram of Fig. 7.1), the convergence behaviour of the employed numerical method remains unchanged. Hence, based on the behaviour of the numerical method one cannot notice if the constrained minimisation problem itself is nonsensical. Believe it or not, this often leads to a complete misinterpretation of the results.

In this chapter, an alternative approach is presented which overcomes the two mentioned disadvantages of the penalty method. Instead of user-defined penalty values, in the method described in this chapter the values necessary to enforce the constraints are part of the solution of the minimisation process. Hence, they automatically become the appropriate values during the minimisation process. These additional variables are called Lagrange multipliers. Second, if incompatible constraints are present, the method described in this chapter does not converge. This is convenient, because this is a sign that the constrained minimisation problem is intrinsically incorrect.

The method of Lagrange multipliers, described in this chapter, also has disadvantages. The first disadvantage is that it can only be applied in combination with Newton's approach. Hence, it cannot be used in combination with GAs or the conjugate gradient approach. This is a true disadvantage if the Hessian is invertible. The second disadvantage is that if inequality constraints are present, the method cannot activate or de-activate constraints during Newton's process. Instead, the inequality constraints that are assumed to be active and inactive at the beginning of Newton's process must remain active and inactive throughout the entire Newton process. (Positive is that after Newton's method has converged, the signs of the resulting Lagrange multipliers can be used to make an educated guess which constraints must be changed and which constraints can remain active and inactive.) Another disadvantage is that the number of unknowns is increased, as each Lagrange multiplier is to be computed during Newton's process. This is not really an issue if the number of constraints is low compared to the number of variables, but if the number of constraints is of a similar order as the number of variables, and the number of variables is large, it can become noticeable.

9.1 The approach in a nutshell

We start with considering the following constrained minimisation problem:

$$\underline{u}^* = \arg \min_{\underline{u}} m(\underline{u}) \quad (9.1)$$

$$\text{such that } \underline{c}(\underline{u}) = \underline{0}, \quad (9.2)$$

with n_{var} variables ($\underline{u}$ is of length n_{var}) and n_c constraints ($\underline{c}$ is of length n_c). In the method of

Lagrange multipliers, this problem is rewritten in the following saddle point problem:

$$\begin{bmatrix} \underline{u}^* \\ \underline{\lambda}^* \end{bmatrix} = \arg \min_{\underline{u}} \arg \max_{\underline{\lambda}} \mathcal{L}(\underline{u}, \underline{\lambda}), \quad (9.3)$$

where

$$\mathcal{L}(\underline{u}, \underline{\lambda}) = m(\underline{u}) + \sum_{i=1}^{n_c} \lambda_i c_i(\underline{u}) = m(\underline{u}) + \underline{\lambda}^T \underline{c}(\underline{u}). \quad (9.4)$$

Since this is a saddle-point problem instead of a minimisation problem, we cannot use genetic algorithms and descent approaches. After all, these approaches are formulated to find a minimum. However, Newton's method is able to find *stationary points*: minima, maxima and saddle points. We are thus obliged to use Newton's method. If we consider Newton's method as a root finding method, we start with the interior extremum theorem for the Lagrangian:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}^*, \underline{\lambda}^*} = \underline{0}, \quad (9.5)$$

where $[(\underline{u}^*)^T (\underline{\lambda}^*)^T]^T$ denotes a stationary point. As we do not know the stationary point, we need to make some initial guess, $[(\underline{u}^i)^T (\underline{\lambda}^i)^T]^T$, and develop a 1st-order Taylor expansion around the initial guess:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i} + \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial (\underline{u})^2} & \frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \\ \frac{\partial^2 \mathcal{L}}{\partial \underline{u} \partial \underline{\lambda}} & \frac{\partial^2 \mathcal{L}}{\partial (\underline{\lambda})^2} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i} \begin{bmatrix} \Delta \underline{u} \\ \Delta \underline{\lambda} \end{bmatrix} = \underline{0}, \quad (9.6)$$

which we can simplify a bit:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i} + \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial (\underline{u})^2} & \frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \\ \left(\frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \right)^T & \underline{0} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i} \begin{bmatrix} \Delta \underline{u} \\ \Delta \underline{\lambda} \end{bmatrix} = \underline{0}. \quad (9.7)$$

Corrections $\Delta \underline{u}$ and $\Delta \underline{\lambda}$ to estimates $\underline{u}^i$ and $\underline{\lambda}^i$ can then be computed by solving the following system of linear equations:

$$\begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial (\underline{u})^2} & \frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \\ \left(\frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \right)^T & \underline{0} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i} \begin{bmatrix} \Delta \underline{u} \\ \Delta \underline{\lambda} \end{bmatrix} = - \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i}, \quad (9.8)$$

which, even though it seems complicated, is completely standard, since it can also be expressed as:

$$\underline{\underline{A}} \underline{x} = \underline{b}, \quad (9.9)$$

with a known, symmetric matrix of size $(n_{var} + n_c) \times (n_{var} + n_c)$:

$$\underline{\underline{A}} = \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial(\underline{u})^2} & \frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \\ \left(\frac{\partial^2 \mathcal{L}}{\partial \underline{\lambda} \partial \underline{u}} \right)^T & \underline{\underline{0}} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i}, \quad (9.10)$$

a known column of length $n_{var} + n_c$:

$$\underline{b} = - \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}^i, \underline{\lambda}^i}, \quad (9.11)$$

and a column with $n_{var} + n_c$ unknowns:

$$\underline{x} = \begin{bmatrix} \Delta \underline{u} \\ \Delta \underline{\lambda} \end{bmatrix}. \quad (9.12)$$

After the system is solved, the update takes place according to:

$$\begin{bmatrix} \underline{u}^{i+1} \\ \underline{\lambda}^{i+1} \end{bmatrix} = \begin{bmatrix} \underline{u}^i \\ \underline{\lambda}^i \end{bmatrix} + \begin{bmatrix} \Delta \underline{u} \\ \Delta \underline{\lambda} \end{bmatrix}, \quad (9.13)$$

and the process is repeated if the L^2 -norm of the gradient of the Lagrangian is insufficiently close to zero.

It may be clear that after the Lagrangian is formulated and both $\underline{u}$ and $\underline{\lambda}$ are together considered the variables of the Lagrangian, the mathematical development of Newton's method is exactly the same as presented in Chapter 5.

9.2 The ansatz

The method of Lagrange multipliers is based on the observation that *a constrained minimum (or maximum) occurs where a contour line of $m(\underline{u})$ is parallel to constraint $c(\underline{u})$* , in the space of the objective function's variables. This entails that at the constrained minimum, the gradient of $m(\underline{u})$ (which is orthogonal to a contour line of $m(\underline{u})$) points in the same or opposite direction

as the gradient of constraint $c(\underline{u})$ (which is orthogonal to $c(\underline{u})$). This is graphically illustrated in Fig. 9.1 for two constrained minimisation problems. This can be expressed as:

$$\left. \frac{\partial m}{\partial \underline{u}} \right|_{\underline{u}^*} = -\lambda^* \left. \frac{\partial c}{\partial \underline{u}} \right|_{\underline{u}^*}, \quad (9.14)$$

where $\frac{\partial m}{\partial \underline{u}}$ denotes the gradient of the objective function and hence, the gradient of a contour line of $m(\underline{u})$. $\frac{\partial c}{\partial \underline{u}}$ denotes the gradient of constraint c and $\lambda^* \in \mathbb{R}$ denotes an unknown scalar (the Lagrange multiplier). Lagrange multiplier λ can both be positive and negative, since at the constrained minimum $\frac{\partial m}{\partial \underline{u}}$ and $\frac{\partial c}{\partial \underline{u}}$ can be pointing in the same or in opposite directions. The magnitude of Lagrange multiplier λ at the constrained minimum can take any value, since the magnitudes of $\frac{\partial m}{\partial \underline{u}}$ and $\frac{\partial c}{\partial \underline{u}}$ at the constrained minimum are by no means necessarily the same.

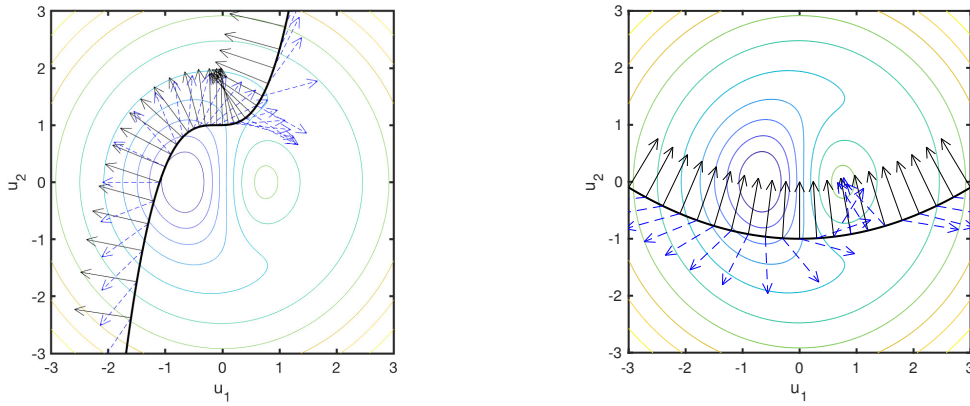


Figure 9.1: Derivatives $\frac{\partial c}{\partial \underline{u}}$ (black arrows) and $\frac{\partial m}{\partial \underline{u}}$ (blue dashed arrows) presented along two different constraints. Constrained minima or maxima occur where the two derivatives point in the same or opposite directions, but are not necessarily of the same magnitude.

If we look at Eq. (9.14) again, we can deduce that for the 2D objective function of Fig. 9.1, both $\frac{\partial m}{\partial \underline{u}}$ and $\frac{\partial c}{\partial \underline{u}}$ depend on two variables (u_1 and u_2), which we need to find. However, Eq. (9.14) depends in total on three variables (u_1 , u_2 and λ), but only involves two equations. Hence, we need one more equation in order to solve for the three unknowns. The additional equation is constraint c itself. Hence, we must solve the following system of three non-linear equations simultaneously:

$$\begin{cases} \frac{\partial m}{\partial \underline{u}} + \lambda \frac{\partial c}{\partial \underline{u}} = \underline{0}, \\ c(\underline{u}) = 0, \end{cases} \quad (9.15)$$

for u_1 , u_2 and λ .

It can quite easily be shown that Eq. (9.15) is the result of setting the gradient of the following function to zero:

$$\mathcal{L}(\underline{u}, \lambda) = m(\underline{u}) + \lambda c(\underline{u}). \quad (9.16)$$

After all,

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \underline{u}} = \frac{\partial m}{\partial \underline{u}} + \lambda \frac{\partial c}{\partial \underline{u}} = \underline{0}, \\ \frac{\partial \mathcal{L}}{\partial \lambda} = c(\underline{u}) = 0. \end{cases} \quad (9.17)$$

9.3 Inequality constraints

In the penalty method of the previous chapter, we have seen that the penalty contribution of an equality constraint can straightforwardly be adjusted to an inequality constraint by making the penalty contribution active or inactive depending on the sign of $c(\underline{u})$. We have seen that for each new estimate of the minimisers (i.e. for each iteration), the activation/deactivation of the penalty contribution must be reconsidered in case of inequality constraints.

The method of Lagrange multipliers does not allow to reconsider whether or not constraints need to be activated or deactivated for each new estimate of the minimisers (i.e. for each iteration). Instead, one must make an assumption which constraints are active and inactive and then finish Newton's method entirely with the same assumption. Afterwards, one must for each constraint evaluate if it was correctly activated or not. If only one or more of the constraints was incorrectly active or inactive, Newton's method must be repeated, with the new activation and de-activation of the constraints. It is thus possible that it Newton's method must be repeated several times.

In case of an inequality constraint of the following form:

$$c(\underline{u}) \geq 0, \quad (9.18)$$

and the constraint was activated and its Lagrange multiplier turns out to *positive* at the saddle-point, the constraint must be de-activated and Newton's method must be repeated. If the constraint was inactive, but $c(\underline{u}^*) < 0$, Newton's method must also be repeated in which the constraint is activated.

In case of an inequality constraint of the following form:

$$c(\underline{u}) \leq 0, \quad (9.19)$$

and the constraint was activated and its Lagrange multiplier turns out to *negative* at the saddle-point, the constraint must be de-activated and Newton's method must be repeated. If the constraint was inactive, but $c(\underline{u}^*) > 0$, Newton's method must also be repeated, but the constraint must be activated.

9.4 Final comment

Although this chapter mentions at length that the only stationary point of the Lagrangian is a saddle-point and hence, only Newton's method can be used in combination with the method of

Lagrange multipliers, there is a way to ensure that other minimisation approaches, such as GAs and the conjugate gradient method, can also be employed. Although this is not considered in the questions below, nor in the remainder of this book, it is briefly introduced in this subsection.

The workaround starts with the case if the interior extremum theorem does not hold:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u} \neq \underline{u}^*, \underline{\lambda} \neq \underline{\lambda}^*} \neq \underline{0}. \quad (9.20)$$

If we now multiply each component of the column on the left-hand side with itself and sum all of them together, and we do the same for the right-hand side, we obtain:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u} \neq \underline{u}^*, \underline{\lambda} \neq \underline{\lambda}^*}^T \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u} \neq \underline{u}^*, \underline{\lambda} \neq \underline{\lambda}^*} = a, \quad (9.21)$$

where $a \in \mathbb{R}^+$ denotes a positive real number. This is only true if $\underline{u} \neq \underline{u}^*$ and/or $\underline{\lambda} \neq \underline{\lambda}^*$, resulting in the following expression:

$$\begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}, \underline{\lambda}}^T \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}, \underline{\lambda}} = \begin{cases} 0 & \text{if } \underline{u} = \underline{u}^* \text{ and } \underline{\lambda} = \underline{\lambda}^*, \\ \mathbb{R}^+ & \text{otherwise.} \end{cases} \quad (9.22)$$

Clearly, this expression looks very much like an objective function:

$$\tilde{m}(\underline{u}, \underline{\lambda}) = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}, \underline{\lambda}}^T \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \underline{u}} \\ \frac{\partial \mathcal{L}}{\partial \underline{\lambda}} \end{bmatrix}_{\underline{u}, \underline{\lambda}}, \quad (9.23)$$

with minimum $\tilde{m}^* = 0$. Note that although $\tilde{m}(\underline{u}, \underline{\lambda})$ is under most circumstances C^1 -continuous (meaning that both $\tilde{m}$ and its gradient are continuous) - which is required for many numerical methods, the above explanation does not prove this.

9.5 Questions

Question 9.1 Repeat question 7.2, but use the method of Lagrange multipliers instead of the penalty method.

Question 9.2 Repeat question 7.4, but use the method of Lagrange multipliers instead of the penalty method.

Question 9.3 Repeat question 7.6, but use the method of Lagrange multipliers instead of the penalty method.

Solutions to the exercises

Solutions to questions of Chapter 1

1.1

$$\nabla^2 m = \begin{bmatrix} 4u_1^3 e^{-u_1^2 - u_2^2} - 6u_1 e^{-u_1^2 - u_2^2} + \frac{1}{10} & 4u_1^2 u_2 e^{-u_1^2 - u_2^2} - 2u_2 e^{-u_1^2 - u_2^2} \\ 4u_1^2 u_2 e^{-u_1^2 - u_2^2} - 2u_2 e^{-u_1^2 - u_2^2} & 4u_1 u_2^2 e^{-u_1^2 - u_2^2} - 2u_1 e^{-u_1^2 - u_2^2} + \frac{1}{10} \end{bmatrix}. \quad (10.1)$$

1.2 500500 independent components.

Solutions to questions of Chapter 3

3.1 (B) Adding mutations ($R_{rand} = 0.5$) makes the GA converge more accurately to the true minimum, but also increases the number of generations necessary to reach convergence. The minimiser reads: $\underline{u}^* = [0.5707 \quad 0.3955]^T$.

3.2 $m^* = -0.573403660$, $\underline{u}^* = [0.5882 \quad 0.3263 \quad 0.6400 \quad -0.2937]^T$.

3.3 Particular about the minimiser is that it is outside the initial search domain, which is possible thanks to the use of mutations. $m^* = -0.872311139$, $\underline{u}^* = [0.8864 \quad 0.1698 \quad 1.0807 \quad -0.3680]^T$.

3.4 $m^* = -0.439029346$, $\underline{u}^* = [0.1637 \quad 0.3596 \quad 0.0321 \quad -0.0112 \quad 0.0128 \quad 0.0001 \quad 0.0062 \quad -0.0000 \quad 0.0059 \quad -0.0000 \quad 0.0115 \quad -0.0000 \quad 0.0285 \quad -0.0001 \quad 0.0740 \quad -0.0020 \quad 0.1933 \quad -0.0278 \quad 0.4925 \quad -0.2306]^T$.

3.5 $m^* = -1.26201090$, $\underline{u}^* = [0.3273 \quad 0.4289 \quad 0.3463 \quad -0.0207 \quad 0.4600 \quad -0.1102 \quad 0.5755 \quad -0.1851 \quad 0.6882 \quad -0.2798 \quad 0.7956 \quad -0.4035 \quad 0.8941 \quad -0.5651 \quad 0.9824 \quad -0.7597 \quad 1.0880 \quad -0.9125 \quad 1.2934 \quad -0.9624]^T$.

3.7 Increasing the values of n_{par0} and n_{par} (e.g. $n_{par0} = 10^4$ and $n_{par} = 5 \cdot 10^3$) increases the chance that a parent or child is placed nearby the global minimum in the first few generations, which is required to converge towards the global minimum.

Solutions to questions of Chapter 4

4.1 Model 1: β according to Eq. (4.6) requires the least iterations. $m^* = -0.492320397$, $\underline{u}^* = [0.5707 \quad 0.3955]^T$.

Model 2a: β according to Eq. (4.9) requires the least iterations. $m^* = -0.573403660$, $\underline{u}^* = [0.5882 \quad 0.3263 \quad 0.6400 \quad -0.2937]^T$.

Model 2b: β according to Eq. (4.9) requires the least iterations. $m^* = -0.872311139$, $\underline{u}^* = [0.8864 \quad 0.1698 \quad 1.0807 \quad -0.3680]^T$.

Model 3a: β according to Eq. (4.7) requires the least iterations. $m^* = -0.439029346$, $\underline{u}^* = [0.1637 \quad 0.3596 \quad 0.0321 \quad -0.0112 \quad 0.0128 \quad 0.0001 \quad 0.0062 \quad -0.0000 \quad 0.0059 \quad -0.0000 \quad 0.0115 \quad -0.0000 \quad 0.0285 \quad -0.0001 \quad 0.0740 \quad -0.0020 \quad 0.1933 \quad -0.0278 \quad 0.4925 \quad -0.2306]^T$.

Model 3b: β according to Eq. (4.9) requires the least iterations. $m^* = -1.26201090$, $\underline{u}^* =$

$[0.3273 \quad 0.4289 \quad 0.3463 \quad -0.0207 \quad 0.4600 \quad -0.1102 \quad 0.5755 \quad -0.1851 \quad 0.6882 \quad -0.2798 \quad 0.7956 \quad -$
 $0.4035 \quad 0.8941 \quad -0.5651 \quad 0.9824 \quad -0.7597 \quad 1.0880 \quad -0.9125 \quad 1.2934 \quad -0.9624]^T$.

Model 4a: β according to Eq. (4.8) requires the least iterations. $m^* = -0.97803880$, $\underline{u}^* =$
 $[0.1148 \quad 0.1673 \quad 0.0489 \quad 0.1065 \quad 0.0329 \quad 0.0685 \quad 0.0436 \quad 0.0390 \quad 0.0639 \quad 0.0226 \quad 0.0815 \quad 0.0161$
 $0.0904 \quad 0.0102 \quad 0.0876 \quad -0.0087 \quad 0.0851 \quad -0.0522 \quad 0.1221 \quad -0.1484 \quad 0.2031 \quad 0.3337 \quad 0.1275$
 $0.2305 \quad 0.1032 \quad 0.1254 \quad 0.1197 \quad 0.0553 \quad 0.1459 \quad 0.0254 \quad 0.1750 \quad 0.0166 \quad 0.2054 \quad 0.0099 \quad 0.2329$
 $-0.0173 \quad 0.2451 \quad -0.1062 \quad 0.2612 \quad -0.2662 \quad 0.3275 \quad 0.5355 \quad 0.2346 \quad 0.3367 \quad 0.2199 \quad 0.1473$
 $0.2196 \quad 0.0561 \quad 0.2336 \quad 0.0171 \quad 0.2653 \quad 0.0020 \quad 0.3189 \quad -0.0125 \quad 0.3968 \quad -0.0511 \quad 0.4832$
 $-0.1454 \quad 0.4773 \quad -0.3626 \quad 0.5957 \quad 0.8310 \quad 0.4409 \quad 0.3355 \quad 0.3709 \quad 0.1572 \quad 0.3208 \quad 0.0591$
 $0.3065 \quad 0.0093 \quad 0.3326 \quad -0.0211 \quad 0.4031 \quad -0.0557 \quad 0.5233 \quad -0.1183 \quad 0.7062 \quad -0.2429 \quad 0.9915 \quad -$
 $0.5251]^T$.

Model 4b: β according to Eq. (4.7) clearly requires the least iterations. $m^* = -3.47042450$,
 $\underline{u}^* = [0.8293 \quad 0.0653 \quad 0.8007 \quad -0.1833 \quad 0.8038 \quad -0.3113 \quad 0.8171 \quad -0.3851 \quad 0.8339 \quad -$
 $0.4341 \quad 0.8521 \quad -0.4689 \quad 0.8711 \quad -0.4915 \quad 0.8906 \quad -0.5000 \quad 0.9091 \quad -0.4944 \quad 0.9216 \quad -$
 $0.4798 \quad 1.6502 \quad 0.1356 \quad 1.5585 \quad -0.3337 \quad 1.5663 \quad -0.5826 \quad 1.6018 \quad -0.7323 \quad 1.6463 \quad -$
 $0.8391 \quad 1.6946 \quad -0.9235 \quad 1.7454 \quad -0.9893 \quad 1.7992 \quad -1.0314 \quad 1.8534 \quad -1.0458 \quad 1.8951 \quad -$
 $1.0367 \quad 2.3996 \quad 0.2534 \quad 2.2256 \quad -0.4255 \quad 2.2532 \quad -0.7928 \quad 2.3315 \quad -1.0241 \quad 2.4247 \quad -$
 $1.2033 \quad 2.5236 \quad -1.3609 \quad 2.6263 \quad -1.5030 \quad 2.7352 \quad -1.6217 \quad 2.8529 \quad -1.7031 \quad 2.9637 \quad -$
 $1.7399 \quad 2.9257 \quad 0.5065 \quad 2.7001 \quad -0.4346 \quad 2.7970 \quad -0.9172 \quad 2.9597 \quad -1.2336 \quad 3.1390 \quad -$
 $1.5005 \quad 3.3213 \quad -1.7584 \quad 3.5030 \quad -2.0179 \quad 3.6878 \quad -2.2705 \quad 3.8937 \quad -2.4822 \quad 4.1732 \quad -$
 $2.5973]^T$.

Model 4b with only $g_{79} = 1$: β according to Eq. (4.8) clearly requires the least iterations.
 $m^* = -4.33096189$, $\underline{u}^* = [1.0138 \quad -1.0000 \quad 1.0154 \quad -1.0000 \quad 1.0189 \quad -1.0000 \quad 1.0245 \quad -$
 $1.0000 \quad 1.0329 \quad -1.0000 \quad 1.0445 \quad -1.0000 \quad 1.0599 \quad -1.0000 \quad 1.0787 \quad -1.0000 \quad 1.0984 \quad -$
 $1.0000 \quad 1.1128 \quad -1.0000 \quad 2.0260 \quad -2.0000 \quad 2.0291 \quad -2.0000 \quad 2.0356 \quad -2.0000 \quad 2.0464 \quad -$
 $2.0000 \quad 2.0625 \quad -2.0000 \quad 2.0853 \quad -2.0000 \quad 2.1165 \quad -2.0000 \quad 2.1565 \quad -2.0000 \quad 2.2022 \quad -$
 $2.0000 \quad 2.2401 \quad -2.0000 \quad 3.0350 \quad -3.0000 \quad 3.0392 \quad -3.0000 \quad 3.0482 \quad -3.0000 \quad 3.0629 \quad -$
 $3.0000 \quad 3.0853 \quad -3.0000 \quad 3.1178 \quad -3.0000 \quad 3.1641 \quad -3.0000 \quad 3.2286 \quad -3.0000 \quad 3.3137 \quad -$
 $3.0000 \quad 3.4052 \quad -3.0000 \quad 4.0398 \quad -4.0000 \quad 4.0447 \quad -4.0000 \quad 4.0549 \quad -4.0000 \quad 4.0719 \quad -$
 $4.0000 \quad 4.0979 \quad -4.0000 \quad 4.1364 \quad -4.0000 \quad 4.1936 \quad -4.0000 \quad 4.2803 \quad -4.0000 \quad 4.4186 \quad -$
 $4.0000 \quad 4.6619 \quad -4.0000]^T$.

Solutions to questions of Chapter 5

5.1 All minima and minimisers can be found under the answer to Question 4.1. One notable observation is that for Model 2b the Hessian of the first iteration is almost not invertible and therefore, the estimate of the minimiser of the first iteration is truly horrible. However, Newton is still able to minimise the objective function. For Model 4b with $g_{62} = g_{79} = 1$, Newton finds another minimum ($m^* = -2.1613$) and minimiser, which is larger than the minimum found by conjugate gradient and therefore suboptimal. For Model 4b with $g_{79} = 1$, the Hessian is not invertible and therefore Newton's method finds no minimum at all.

5.2 For the conjugate gradient method $\sqrt{\underline{f}^{*T} \underline{f}^*} \approx 10^{-7}$ and for Newton $\sqrt{\underline{f}^{*T} \underline{f}^*} \approx 10^{-14}$. This shows that the gradient found by Newton at the minimiser is closer to zero, and since $\sqrt{\underline{f}^{*T} \underline{f}^*} = 0$ holds at the exact minimiser, it means that the minimiser found by Newton is more accurate.

Solutions to questions of Chapter 6

6.1 All minima and minimisers can be found under the answer to Question 4.1.

6.2 All minima and minimisers can be found under the answer to Question 4.1.

Solutions to questions of Chapter 7

7.1 $m^* = -0.4052368703$, $\underline{u}^* = [-0.6691 \quad 0.0000]^T$.

7.2 $m^* = -0.2887393591$, $\underline{u}^* = [-1.0154 \quad 0.2330]^T$.

7.3 The minimisers of the original objective function are located in the admissible domain and hence, the minimum and minimisers are the same as those of 6.1.

7.4 The minimisers of the original objective function are located in the inadmissible domain and hence, the minimum and minimisers are the same as those of 6.2.

Solutions to questions of Chapter 8

8.1 For $k = 1$: $m^* = 0.386134499$, $\underline{u}^* = [-0.0164 \quad 0.0293 \quad -0.0336 \quad 0.0097 \quad -0.0367 \quad -0.0158 \quad -0.0271 \quad -0.0364 \quad -0.0098 \quad -0.0475 \quad 0.0098 \quad -0.0475 \quad 0.0271 \quad -0.0364 \quad 0.0367 \quad -0.0158 \quad 0.0336 \quad 0.0097 \quad 0.0164 \quad 0.0293 \quad -0.0009 \quad 0.0884 \quad -0.0291 \quad 0.0277 \quad -0.0365 \quad -0.0366 \quad -0.0281 \quad -0.0859 \quad -0.0104 \quad -0.1116 \quad 0.0104 \quad -0.1116 \quad 0.0281 \quad -0.0859 \quad 0.0365 \quad -0.0366 \quad 0.0291 \quad 0.0277 \quad 0.0009 \quad 0.0884 \quad 0.0780 \quad 0.1712 \quad 0.0325 \quad 0.0493 \quad 0.0091 \quad -0.0606 \quad -0.0012 \quad -0.1386 \quad -0.0016 \quad -0.1792 \quad 0.0016 \quad -0.1792 \quad 0.0012 \quad -0.1386 \quad -0.0091 \quad -0.0606 \quad -0.0325 \quad 0.0493 \quad -0.0780 \quad 0.1712 \quad 0.2564 \quad 0.2646 \quad 0.1766 \quad 0.0697 \quad 0.1093 \quad -0.0765 \quad 0.0573 \quad -0.1813 \quad 0.0176 \quad -0.2360 \quad -0.0176 \quad -0.2360 \quad -0.0573 \quad -0.1813 \quad -0.1093 \quad -0.0765 \quad -0.1766 \quad 0.0697 \quad -0.2564 \quad 0.2646]^T$.

For $k = 100$: $m^* = 0.734450969$, $\underline{u}^* = [-0.0359 \quad 0.0467 \quad -0.0668 \quad 0.0126 \quad -0.0740 \quad -0.0346 \quad -0.0559 \quad -0.0743 \quad -0.0206 \quad -0.0965 \quad 0.0206 \quad -0.0965 \quad 0.0559 \quad -0.0743 \quad 0.0740 \quad -0.0346 \quad 0.0668 \quad 0.0126 \quad 0.0359 \quad 0.0467 \quad -0.0064 \quad 0.1495 \quad -0.0617 \quad 0.0417 \quad -0.0788 \quad -0.0788 \quad -0.0613 \quad -0.1765 \quad -0.0228 \quad -0.2293 \quad 0.0228 \quad -0.2293 \quad 0.0613 \quad -0.1765 \quad 0.0788 \quad -0.0788 \quad 0.0617 \quad 0.0417 \quad 0.0064 \quad 0.1495 \quad 0.1456 \quad 0.2913 \quad 0.0500 \quad 0.0754 \quad 0.0027 \quad -0.1364 \quad -0.0116 \quad -0.2900 \quad -0.0062 \quad -0.3698 \quad 0.0062 \quad -0.3698 \quad 0.0116 \quad -0.2900 \quad -0.0027 \quad -0.1364 \quad -0.0500 \quad 0.0754 \quad -0.1456 \quad 0.2913 \quad 0.4782 \quad 0.4431 \quad 0.3156 \quad 0.0814 \quad 0.1915 \quad -0.1965 \quad 0.1000 \quad -0.3865 \quad 0.0308 \quad -0.4832 \quad -0.0308 \quad -0.4832 \quad -0.1000 \quad -0.3865 \quad -0.1915 \quad -0.1965 \quad -0.3156 \quad 0.0814 \quad -0.4782 \quad 0.4431]^T$.

For $k = 10^4$: $m^* = 0.741062095$, $\underline{u}^* = [-0.0362 \quad 0.0470 \quad -0.0674 \quad 0.0127 \quad -0.0747 \quad -0.0350 \quad -0.0564 \quad -0.0750 \quad -0.0208 \quad -0.0974 \quad 0.0208 \quad -0.0974 \quad 0.0564 \quad -0.0750 \quad 0.0747 \quad -0.0350 \quad 0.0674 \quad 0.0127 \quad 0.0362 \quad 0.0470 \quad -0.0065 \quad 0.1506 \quad -0.0623 \quad 0.0420 \quad -0.0796 \quad -0.0796 \quad -0.0619 \quad -0.1783 \quad -0.0230 \quad -0.2316 \quad 0.0230 \quad -0.2316 \quad 0.0619 \quad -0.1783 \quad 0.0796 \quad -0.0796 \quad 0.0623 \quad 0.0420 \quad 0.0065 \quad 0.1506 \quad 0.1470 \quad 0.2935 \quad 0.0503 \quad 0.0758 \quad 0.0026 \quad -0.1380 \quad -0.0118 \quad -0.2929 \quad -0.0062 \quad -0.3734 \quad 0.0062 \quad -0.3734 \quad 0.0118 \quad -0.2929 \quad -0.0026 \quad -0.1380 \quad -0.0503 \quad 0.0758 \quad -0.1470 \quad 0.2935 \quad 0.4827 \quad 0.4463 \quad 0.3182 \quad 0.0812 \quad 0.1930 \quad -0.1993 \quad 0.1008 \quad -0.3905 \quad 0.0310 \quad -0.4877 \quad -0.0310 \quad -0.4877 \quad -0.1008 \quad -0.3905 \quad -0.1930 \quad -0.1993 \quad -0.3182 \quad 0.0812 \quad -0.4827 \quad 0.4463]^T$.

8.2 For $k = 1$ and $k = 100$, the minima and minimisers are very similar as given for 7.1. They are not exactly the same, but with the number of decimals presented in the solutions for 7.1, the difference cannot be distinguished. For $k = 10^4$, Newton does not converge below the given tolerance of 10^{-12} .

$$\begin{aligned} \mathbf{8.3} \quad m^* &= 0.287799487, \underline{u}^* = [0.0333 \quad 0.0811 \quad -0.0017 \quad 0.0464 \quad -0.0113 \quad 0.0185 \quad - \\ &0.0083 \quad -0.0020 \quad -0.0026 \quad -0.0118 \quad 0.0026 \quad -0.0118 \quad 0.0083 \quad -0.0020 \quad 0.0113 \quad 0.0185 \quad 0.0017 \quad 0.0464 \\ &0.0333 \quad 0.0811 \quad 0.0732 \quad 0.1782 \quad 0.0301 \quad 0.1029 \quad 0.0145 \quad 0.0328 \quad 0.0107 \quad -0.0083 \quad 0.0040 \quad - \\ &0.0238 \quad -0.0040 \quad -0.0238 \quad -0.0107 \quad -0.0083 \quad -0.0145 \quad 0.0328 \quad -0.0301 \quad 0.1029 \quad - \\ &0.0732 \quad 0.1782 \quad 0.1697 \quad 0.3037 \quad 0.1080 \quad 0.1468 \quad 0.0825 \quad 0.0380 \quad 0.0499 \quad -0.0063 \quad 0.0166 \quad - \\ &0.0228 \quad -0.0166 \quad -0.0228 \quad -0.0499 \quad -0.0063 \quad -0.0825 \quad 0.0380 \quad -0.1080 \quad 0.1468 \quad - \\ &0.1697 \quad 0.3037 \quad 0.4314 \quad 0.4668 \quad 0.2705 \quad 0.1513 \quad 0.1731 \quad 0.0611 \quad 0.0946 \quad 0.0149 \quad 0.0300 \quad - \\ &0.0028 \quad -0.0300 \quad -0.0028 \quad -0.0946 \quad 0.0149 \quad -0.1731 \quad 0.0611 \quad -0.2705 \quad 0.1513 \quad - \\ &0.4314 \quad 0.4668]^T. \end{aligned}$$

8.4 The minimum and minimiser are very similar as given for 7.3. They are not exactly the same, but with the number of decimals presented in the solution for 7.3, the difference cannot be distinguished.

$$\begin{aligned} \mathbf{8.5} \quad m^* &= 0.414314994, \underline{u}^* = [-0.0670 \quad -0.0372 \quad -0.0607 \quad -0.0334 \quad -0.0578 \quad -0.0501 \quad - \\ &0.0442 \quad -0.0663 \quad -0.0168 \quad -0.0764 \quad 0.0168 \quad -0.0764 \quad 0.0442 \quad -0.0663 \quad 0.0578 \quad - \\ &0.0501 \quad 0.0607 \quad -0.0334 \quad 0.0670 \quad -0.0372 \quad -0.0796 \quad -0.0359 \quad -0.0857 \quad -0.0604 \quad - \\ &0.0873 \quad -0.1025 \quad -0.0714 \quad -0.1523 \quad -0.0284 \quad -0.1890 \quad 0.0284 \quad -0.1890 \quad 0.0714 \quad - \\ &0.1523 \quad 0.0873 \quad -0.1025 \quad 0.0857 \quad -0.0604 \quad 0.0796 \quad -0.0359 \quad -0.0472 \quad -0.0241 \quad - \\ &0.0545 \quad -0.0676 \quad -0.0729 \quad -0.1417 \quad -0.0690 \quad -0.2521 \quad -0.0262 \quad -0.3333 \quad 0.0262 \quad - \\ &0.3333 \quad 0.0690 \quad -0.2521 \quad 0.0729 \quad -0.1417 \quad 0.0545 \quad -0.0676 \quad 0.0472 \quad -0.0241 \quad 0.0070 \quad - \\ &0.0200 \quad 0.0112 \quad -0.0723 \quad 0.0241 \quad -0.1542 \quad 0.0091 \quad -0.3715 \quad 0.0024 \quad -0.4811 \quad -0.0024 \quad - \\ &0.4811 \quad -0.0091 \quad -0.3715 \quad -0.0241 \quad -0.1542 \quad -0.0112 \quad -0.0723 \quad -0.0070 \quad -0.0200]^T. \end{aligned}$$

8.6 The minimum and minimiser are very similar as given for 7.5. They are not exactly the same, but with the number of decimals presented in the solution for 7.5, the difference cannot be distinguished.

Solutions to questions of Chapter 9

$$\begin{aligned} \mathbf{9.1} \quad \mathcal{L}^* &= 0.741129469, \underline{u}^* = [-0.0362 \quad 0.0470 \quad -0.0674 \quad 0.0127 \quad -0.0747 \quad -0.0350 \quad - \\ &0.0564 \quad -0.0750 \quad -0.0208 \quad -0.0974 \quad 0.0208 \quad -0.0974 \quad 0.0564 \quad -0.0750 \quad 0.0747 \quad - \\ &0.0350 \quad 0.0674 \quad 0.0127 \quad 0.0362 \quad 0.0470 \quad -0.0065 \quad 0.1506 \quad -0.0624 \quad 0.0420 \quad -0.0796 \quad - \\ &0.0796 \quad -0.0619 \quad -0.1783 \quad -0.0230 \quad -0.2316 \quad 0.0230 \quad -0.2316 \quad 0.0619 \quad -0.1783 \quad 0.0796 \quad - \\ &0.0796 \quad 0.0624 \quad 0.0420 \quad 0.0065 \quad 0.1506 \quad 0.1470 \quad 0.2935 \quad 0.0503 \quad 0.0758 \quad 0.0026 \quad -0.1380 \quad - \\ &0.0118 \quad -0.2930 \quad -0.0062 \quad -0.3734 \quad 0.0062 \quad -0.3734 \quad 0.0118 \quad -0.2930 \quad -0.0026 \quad - \\ &0.1380 \quad -0.0503 \quad 0.0758 \quad -0.1470 \quad 0.2935 \quad 0.4827 \quad 0.4463 \quad 0.3183 \quad 0.0812 \quad 0.1930 \quad - \\ &0.1993 \quad 0.1008 \quad -0.3906 \quad 0.0310 \quad -0.4878 \quad -0.0310 \quad -0.4878 \quad -0.1008 \quad -0.3906 \quad - \\ &0.1930 \quad -0.1993 \quad -0.3183 \quad 0.0812 \quad -0.4827 \quad 0.4463]^T, \underline{\lambda}^* = [0.4947 \quad 0.0726 \quad -0.2363 \quad - \\ &0.3925 \quad -0.4625 \quad -0.4625 \quad -0.3925 \quad -0.2363 \quad 0.0726 \quad 0.4947]^T. \end{aligned}$$

$$\begin{aligned} \mathbf{9.2} \quad \mathcal{L}^* &= 0.290088033, \underline{u}^* = [0.0336 \quad 0.0817 \quad -0.0017 \quad 0.0467 \quad -0.0114 \quad 0.0186 \quad -0.0084 \quad - \\ &0.0020 \quad -0.0026 \quad -0.0119 \quad 0.0026 \quad -0.0119 \quad 0.0084 \quad -0.0020 \quad 0.0114 \quad 0.0186 \quad 0.0017 \quad 0.0467 \quad - \\ &0.0336 \quad 0.0817 \quad 0.0738 \quad 0.1795 \quad 0.0304 \quad 0.1038 \quad 0.0146 \quad 0.0331 \quad 0.0107 \quad -0.0083 \quad 0.0041 \quad - \\ &0.0239 \quad -0.0041 \quad -0.0239 \quad -0.0107 \quad -0.0083 \quad -0.0146 \quad 0.0331 \quad -0.0304 \quad 0.1038 \quad - \end{aligned}$$

0.0738 0.1795 0.1712 0.3060 0.1089 0.1480 0.0831 0.0383 0.0503 -0.0063 0.0167 -
0.0230 -0.0167 -0.0230 -0.0503 -0.0063 -0.0831 0.0383 -0.1089 0.1480 -
0.1712 0.3060 0.4350 0.4703 0.2725 0.1523 0.1744 0.0615 0.0953 0.0150 0.0302 -
0.0028 -0.0302 -0.0028 -0.0953 0.0150 -0.1744 0.0615 -0.2725 0.1523 -
0.4350 0.4703]^T, $\underline{\lambda}^* = [0.4804 \quad 0.4804]^T$.

9.3 $\mathcal{L}^* = 0.418671486$, $\underline{u}^* = [-0.0677 \quad -0.0376 \quad -0.0613 \quad -0.0338 \quad -0.0584 \quad -0.0506 \quad -$
 $0.0447 \quad -0.0670 \quad -0.0170 \quad -0.0772 \quad 0.0170 \quad -0.0772 \quad 0.0447 \quad -0.0670 \quad 0.0584 \quad -$
 $0.0506 \quad 0.0613 \quad -0.0338 \quad 0.0677 \quad -0.0376 \quad -0.0804 \quad -0.0362 \quad -0.0866 \quad -0.0610 \quad -$
 $0.0883 \quad -0.1036 \quad -0.0722 \quad -0.1539 \quad -0.0288 \quad -0.1911 \quad 0.0288 \quad -0.1911 \quad 0.0722 \quad -$
 $0.1539 \quad 0.0883 \quad -0.1036 \quad 0.0866 \quad -0.0610 \quad 0.0804 \quad -0.0362 \quad -0.0476 \quad -0.0242 \quad -$
 $0.0550 \quad -0.0682 \quad -0.0737 \quad -0.1432 \quad -0.0697 \quad -0.2549 \quad -0.0264 \quad -0.3370 \quad 0.0264 \quad -$
 $0.3370 \quad 0.0697 \quad -0.2549 \quad 0.0737 \quad -0.1432 \quad 0.0550 \quad -0.0682 \quad 0.0476 \quad -0.0242 \quad 0.0073 \quad -$
 $0.0201 \quad 0.0115 \quad -0.0730 \quad 0.0246 \quad -0.1559 \quad 0.0093 \quad -0.3757 \quad 0.0025 \quad -0.4862 \quad -0.0025 \quad -$
 $0.4862 \quad -0.0093 \quad -0.3757 \quad -0.0246 \quad -0.1559 \quad -0.0115 \quad -0.0730 \quad -0.0073 \quad -0.0201]^T$,
 $\underline{\lambda}^* = [-0.4183 \quad -0.5147 \quad -0.5147 \quad -0.4183]^T$.